

Requirements Index — Master Overview

Leish! v2 — Beauty Booking Marketplace

2026-08-29

Leish! v2 — Requirements & Specification Index

Reading Paths

Versioning & Change Control

Quick Link: The Booking Money Trail (The Hardest Part)

Next Step

Leish! v2 — Requirements & Specification Index

Start here. This index maps the full requirements corpus and tells any role where to go.

DOC	FILE	AUDIENCE	PURPOSE
BRS	01-BRS-Business-Requirements-Specification.md	Founders, PM, Ops, Investors	Why we build — objectives, scope, business model (hybrid deposit/balance + payouts), rules BRL-01..15, processes, KPIs, roadmap
SRS	02-SRS-Software-Requirements-Specification.md	Architects, Eng Lead, QA Lead	What the system shall do — IEEE 830 system features SRS-F-01..13, external interfaces, data & verification strategy
FRS	03-FRS-Functional-Requirements-Specification.md	Engineers, QA, UAT	How each function works — 48 FR-xxx with pre/post, flows, source files (src/server/*:line), and Given/When/Then acceptance
NFR	04-NFR-Non-Functional-Requirements.md	Eng, SRE, Security, Compliance	How well — measurable latency, availability 99.5%, headers/CSP/HMAC, coverage ≥80%, WCAG AA etc.
Use Cases	05-Use-Cases-and-User-Stories.md	PM, Design, QA	20 use cases UC-01..20 + 24 INVEST stories — step-by-step actor flows for every journey
Data Model	06-Data-Model-and-ERD.md	DBAs, Eng	ERD + 13 table dictionaries + money logic + seeding/migration + retention
API Spec	07-API-Specification.md	Frontend, Mobile, Integrators	40+ endpoints with request/response examples, status codes, pagination, SSE, webhook HMAC contract
RTM	08-Traceability-Matrix.md	QA, Auditors, Release Mgr	Bidirectional BR→SRS→FRS→API→DB→test + BRL→test + orphan check
Glossary	09-Glossary.md	Everyone	Canonical term definitions (Deposit/Balance/Payout/Claim/etc.)
Index	00-Requirements-Index.md	Everyone	This file

Companion docs (existing):

- [AGENTS.md](#) — repo conventions and “active memory” for agents (DB facade, auth, catalog 404 gotchas, security).
- [ARCHITECTURE.md](#) — single-backend data flow narrative (booking→fee→webhook→confirm).
- [PDPA_RETENTION_GUIDELINES.md](#) — retention policy per entity.

- `CODEBASE-AUDIT-FINAL.md` — closure of prior audit findings.
- `DEPLOY.md` / `HANDOVER.md` / `ROADMAP-TO-DEPLOYMENT.md` — operational runbooks.

Reading Paths

ROLE	READ IN ORDER
Founder / Investor	BRS (01) → NFR §9 Success metrics → Use Cases intro → API index (scan)
Product Manager	BRS (01) → Use Cases (05) → FRS (03) §12–14 dashboard/compliance → RTM (08) for coverage
Engineering (backend)	SRS (02) §2.1 → FRS (03) §4–11 → Data Model (06) → API Spec (07) → NFR (04) §4 Security
Engineering (frontend)	SRS (02) §4.1 UI + §4.3 supabase header → API Spec (07) §3–7 → Use Cases (05) for flows
QA / Release	FRS (03) acceptance criteria → RTM (08) §1 → NFR (04) §11 gate matrix → <code>npm run</code> checks
Design	BRS (01) §8 journeys → Use Cases (05) flows → SRS (02) §4.1 design system notes
Security / Compliance	NFR (04) §4 Security → FRS §4,8,14 → Data Model (06) retention → <code>09-Glossary</code> PDPA term
Ops / SRE	NFR (04) §2–3, §9 Ops → SRS (02) §4.3 software interfaces → API §14 health/cron → <code>AGENTS.md</code> deploy checklist

Versioning & Change Control

- Documents follow `LEISH-<TYPE>-v2.0` with date. Next increment is `2.1` for studio-native bookings / balance dunning; `3.0` for e-commerce.
- Change requests: open issue with `type: requirement-change`, update BRS first, then SRS/FRS/RTM in same PR; CI must still pass (`typecheck` proves no code drift).
- Trace orphan scan: before merge, confirm `08-Traceability-Matrix.md §4` lists no untraced prod surface.

Quick Link: The Booking Money Trail (The Hardest Part)

```
Customer → POST /api/bookings (price from catalog)
  → Artist POST /api/bookings/[id]/quotation (total via quotationTotal)
  → Customer POST /api/bookings/[id]/pay-fee (deposit = getBookingFeeSen)
  → Billplz POST /api/payments/webhook (HMAC verify) → handlePaymentPaid (deposit → confirmed)
  → Balance due T-3d → POST /api/bookings/[id]/pay-balance (balance = total - deposit)
  → Webhook (balance → quotation=paid + createPayoutForBooking [gross/commission/net])
  → Admin PATCH /api/admin/payouts {settled, notes}
```

See **BRS §5.1**, **FRS §8–9**, **Data Model §5**, **API Spec §5+7** for each hop's full detail.

Next Step

Open `01-BRS-Business-Requirements-Specification.md` to start at business rationale, or jump to `07-API-Specification.md` if you're integrating against a running deployment.

BRS — Business Requirements Specification

BRS — Business Requirements Specification

Leish! v2 — Beauty Booking Marketplace (Malaysia)

Table of Contents

1. Executive Summary
2. Business Vision & Objectives
3. Stakeholders & Actors
4. Scope Definition
5. Business Model & Monetization
6. Business Requirements (BR-xxx)
7. Business Rules (BRL-xxx)
8. Key Business Processes & Journeys
9. Success Metrics & KPIs
10. Constraints, Assumptions & Dependencies
11. Compliance & Legal
12. Risks & Mitigations
13. Roadmap & Release Strategy
14. Acceptance Criteria (Business Level)
15. Appendix

BRS — Business Requirements Specification

Leish! v2 — Beauty Booking Marketplace (Malaysia)

FIELD	VALUE
Document ID	LEISH-BRS-v2.0
Version	2.0.0
Status	Baseline (Approved)
Date	2026-08-29
Owner	Product & Engineering
Classification	Internal / Investor-Facing
Related Docs	SRS (02), FRS (03), NFR (04), Use Cases (05)

Table of Contents

- [1. Executive Summary](#)
- [2. Business Vision & Objectives](#)
- [3. Stakeholders & Actors](#)
- [4. Scope Definition](#)
- [5. Business Model & Monetization](#)
- [6. Business Requirements \(BR-xxx\)](#)
- [7. Business Rules \(BRL-xxx\)](#)
- [8. Key Business Processes & Journeys](#)
- [9. Success Metrics & KPIs](#)
- [10. Constraints, Assumptions & Dependencies](#)
- [11. Compliance & Legal](#)
- [12. Risks & Mitigations](#)
- [13. Roadmap & Release Strategy](#)
- [14. Acceptance Criteria \(Business Level\)](#)
- [15. Appendix](#)

1. Executive Summary

Leish! v2 is a next-generation, mobile-first **beauty services marketplace** that connects **Clients (customers)** with **MUA (Makeup Artists)** and **Beauty Studios** across Malaysia, starting with the Klang Valley (KL + Selangor) as the launch corridor. v1 ([leish.my](#)) validated demand but incurred architectural debt (dual-DB, fragmented payments, no payout ledger). v2 is a **from-scratch rebuild on Next.js 16 + PostgreSQL** that solves for:

- **Trust:** verified-email gating at payment, reviews gated on completed bookings only, admin audit trail.
- **Money correctness:** hybrid deposit/balance model with server-authoritative pricing, HMAC-verified Billplz webhook, commission/payout ledger.
- **Operations:** admin can manage the full lifecycle without DB access (users, catalog, bookings, payouts, settings).
- **Compliance:** Malaysia PDPA 2010 consent, retention, export/delete.

The platform's **north-star metric** is Completed Bookings per Month — every feature must move this metric or de-risk it.

2. Business Vision & Objectives

2.1 VISION

Book beauty anywhere in Malaysia in under 90 seconds — with guaranteed slot integrity, transparent pricing, and artist-grade economics.

2.2 STRATEGIC OBJECTIVES

ID	OBJECTIVE	MEASURABLE OUTCOME	HORIZON
BO-01	Own Klang Valley bridal discovery	≥10 active MUAs, 500 MAU within 3 months of launch	Q4 2026
BO-02	Payment reliability	≥99.5% webhook-to-confirmation success, <5 min median	Q4 2026
BO-03	Artist retention	≥70% of onboarded MUAs receive ≥2 bookings in first 60 days	H1 2027
BO-04	Unit economics	Take-rate 8–10% net after waivers, contribution margin positive by month 6	H1 2027
BO-05	Compliance	PDPA consent audit 100%, no open critical vulnerabilities (Snyk/Dependabot)	Continuous
BO-06	Platform extensibility	Studio bookings, products, referrals live without re-architecture	2027

2.3 PROBLEM STATEMENT (WHY REBUILD)

PAIN (V1)	IMPACT	V2 SOLUTION
Supabase + Neon dual-write	Data drift, sync cron, on-call burden	Single <code>DATABASE_URL</code> (PG) + SQLite fallback facade <code>src/server/db.ts</code>
Client-trusted pricing	Under-charge risk	Server-derived <code>price / quotation.total</code> only (<code>src/server/quotations.ts</code>)
Single payment type	Cannot charge deposit + balance	Hybrid <code>payments.type IN ('deposit', 'balance')</code> with UNIQUE <code>(booking_id, type)</code>
No payout ledger	Manual spreadsheets	<code>payouts</code> table + <code>/admin/payouts</code> settle/fail
Reviews un-gated	Fake/social proof risk	<code>reviews.booking_id UNIQUE</code> + <code>findReviewableBooking()</code> (COMPLETED only)
No audit trail	Ops risk	<code>admin_audit_log</code> on every admin mutation

3. Stakeholders & Actors

3.1 STAKEHOLDER MAP

STAKEHOLDER	INTEREST	POWER	ENGAGEMENT
Founders / Investors	Revenue, growth, cost control	High	Steering, KPI review
Clients (Customers)	Fast booking, fair price, reliability	High	Discovery research, NPS
MUAs / Studios	Lead flow, fair take-rate, payout speed	High	Onboarding interviews
Platform Ops / Admin	Efficiency, fraud prevention	Medium	Workflow design
Billplz (payment processor)	Volume, compliance	Medium	Integration & SLA
Regulators (PDPA, KPDN)	Consumer protection	Medium	Legal review
Engineering	Maintainability, velocity	Medium	Architecture decisions

3.2 SYSTEM ACTORS (FROM `SRC/LIB/TYPES.TS:1`)

ACTOR	SYSTEM ROLE	AUTH REQUIREMENT
Guest	Unauthenticated visitor	None
Customer	<code>customer</code> — books services	Email + verified for payment
Artist	<code>artist</code> — services a booking, claims <code>artist_profiles</code>	Verified; must claim profile to see jobs
Studio	<code>studio</code> — studio branch of artist (shares <code>artist</code> flows)	Same as artist
Admin	<code>admin</code> — platform operator	<code>requireAdmin()</code> guard + audit log
System/Cron	Internal	<code>CRON_SECRET</code>

4. Scope Definition

4.1 IN SCOPE (V2 LAUNCH)

- Catalog:** Artist & Studio discovery — search, filters (state→area, bridal/non-bridal, budget, date), detail pages, portfolio, reviews.
- Identity:** Register/Login/Logout, JWT session (`leish_session` , 7-day TTL, `src/server/session.ts`), email verification (single-use token), forgot/reset password, consent capture.
- Booking lifecycle:** `requested` → `accepted` → `confirmed` → `completed` | `cancelled` via `src/server/bookings.ts` state machine; slot uniqueness (`uq_bookings_slot` partial index).
- Quotations:** Line-item breakdown (base/travel/early-call/accommodation + up to 10 extras), 24h review window, re-quote supersede.
- Hybrid Payments:** Deposit (configurable `booking_fee_sen` , default RM 50) to accept; Balance (`total - deposit`) due 3 days before event; Billplz + `dev` fallback; HMAC webhook; refund (balance only, deposit non-refundable).
- Payouts & Commission:** `computeCommission()` (basis points + waiver threshold, e.g. 10% above RM100); payout `net = gross - commission - deposit` ; settleable 24h post-event; admin settle/fail.
- Messaging:** Per-booking chat (`messages` table) + SSE live stream (`/api/bookings/[id]/messages/stream`).
- Reviews:** Post-completion, one per booking, atomic `blendAggregate` to rating.
- Admin Panel** (`/admin/*`): Users CRUD, Artists/Studios direct edits, Bookings override, Payments, Payouts, Quotations, Messages, Email Outbox, Audit Log, Settings (`platform_settings`).
- Notifications/Email:** `sendEmail()` abstraction (`dev|resend|postmark|brevo`), preferences per user, retry queue.
- Observability:** pino structured logs, `LOG_WEBHOOK_URL` forwarding, `reportError()` → Sentry/webhook.
- Compliance:** PDPA consent, data export (`/api/me/export`), account deletion (`DELETE /api/me`), retention guidelines.

4.2 OUT OF SCOPE (V2 LAUNCH) — DEFERRED

ITEM	RATIONALE	TARGET RELEASE
Studio as distinct provider with separate commission tier	Complexity; not needed for 10-MUA launch gate	v2.1
Product/e-commerce (<code>products</code> table already scaffolded)	Separate fulfillment flow	v2.2
Loyalty / coupons	Defer until repeat booking rate proven	v2.2
Mobile native apps	PWA/mobile-web first	v2.3
Multi-currency / i18n (BM)	MYR + English only at launch	Backlog

4.3 SCOPE BOUNDARIES (EXPLICIT NON-GOALS)

- The platform **does not** handle physical fulfillment, inventory, or shipping.
- The platform **does not** guarantee artist availability — it guarantees slot atomicity (unique partial index + friendly 409).
- Client-sent amounts are **never** trusted — all money math is server-authoritative.

5. Business Model & Monetization

5.1 REVENUE MODEL

```

Client pays: quotation.total (e.g. RM 1,000)
├ deposit RM 50 (non-refundable, platform-kept, default booking_fee_sen)
└ balance RM 950 (due T-3d)

Platform keeps: deposit + commission
commission = total * rateBps / 10_000 unless total < waiverSen → 0
e.g. total 100,000 sen * 1,000 bps / 10,000 = 10,000 sen (RM100)
Platform keeps 5,000 + 10,000 = 15,000 sen

Artist receives: total - commission - deposit = 100,000 - 10,000 - 5,000 = 85,000 sen
settleable_at = eventDate + 24h (dispute window)

```

- **Settings** are runtime-configurable via `platform_settings` (`booking_fee_sen` , `commission_rate_bps` , `commission_waiver_sen`) — see `src/server/settings.ts:1` .
- **Waiver** protects small bookings (< RM100 by default) from being economically non-viable for artists.

5.2 PRICING RULES

- All amounts stored in **sen** (integer MYR cents) — no floating point for money.
- Quotations are the **sole** pricing authority; `bookings.price` is informational (catalog price of selected service at booking time).
- Client always pays **exactly** the quoted `total` ; commission is artist-side.

5.3 TAKE-RATE GUARDRAILS

KNOB	DEFAULT	CLAMP	PURPOSE
<code>booking_fee_sen</code>	5,000 (RM50)	≥ 0	Non-refundable slot commitment
<code>commission_rate_bps</code>	1,000 (10%)	0–5,000 (0–50%)	Typo-protection clamp in <code>getCommissionRateBps()</code>
<code>commission_waiver_sen</code>	10,000 (RM100)	≥ 0	Small-booking protection

5.4 COST STRUCTURE

COST	DRIVER	MITIGATION
Billplz fees	Per-transaction	Passed through or absorbed via commission
Email (Resend/Postmark/Brevo)	Per-send	<code>dev</code> fallback, outbox batching
Hosting (Vercel + PG)	Traffic + connections	<code>PG_MAX</code> , pool thresholds, <code>s-maxage=300</code> on public catalog
Support	Bookings volume	Self-serve dashboard + chat reduces tickets

6. Business Requirements (BR-xxx)

6.1 CUSTOMER & DISCOVERY

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-01	As a Guest, I can discover artists/studios without logging in.	Must	Catalog loads unauthenticated; CDN <code>s-maxage=300</code> .
BR-02	As a Customer, I can filter artists by state, area, bridal/non-bridal tags, budget, date, and free-text query.	Must	<code>ListArtists(filters)</code> + <code>artistsQuerySchema</code> .
BR-03	As a Visitor, I can view artist/studio detail (bio, services, portfolio, reviews, availability).	Must	<code>getArtistBySlug()</code> resolves both slug and legacy id.
BR-04	As a Customer, I can leave a review only after my booking is completed, once per booking.	Must	<code>findReviewableBooking()</code> + <code>reviews.booking_id UNIQUE</code> .
BR-05	As a Platform, ratings are blended atomically to prevent lost-update races.	Must	<code>blendAggregate()</code> single-statement UPDATE.

6.2 IDENTITY & TRUST

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-06	As a User, I can register with role choice (customer/artist/studio) and PDPA consent.	Must	<code>registerSchema</code> , <code>users.consent</code> .
BR-07	As a User, I must verify my email before payments become effective (trust gate).	Must	Verification token single-use, expiry; gate at <code>pay-fee</code> / <code>pay-balance</code> .
BR-08	As a User, I can reset my password via a single-use, expiring link with no user enumeration.	Must	<code>password_resets.token_hash</code> sha256, 1h expiry.
BR-09	As a User, my session lasts 7 days and is revocable on logout.	Must	<code>SESSION_TTL_SECONDS = 604800</code> , JTI <code>sessions.revoked</code> .
BR-10	As a User, I can export my data and delete my account (PDPA).	Must	<code>GET /api/me/export</code> , <code>DELETE /api/me?confirm=1</code> .

6.3 BOOKING & QUOTATIONS

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-11	As a Customer, I can submit a booking request with event details (type, venue, guests, notes).	Must	<code>bookingSchema</code> with <code>eventType/venue/guestCount</code> .
BR-12	As a System, a slot (artist+date+time) can have only one active booking.	Must	<code>uq_bookings_slot</code> partial index + 409 friendly error.
BR-13	As an Artist, I can accept/reject a <code>requested</code> booking, or complete a <code>confirmed</code> one.	Must	<code>applyBookingTransition()</code> state machine.
BR-14	As an Artist, I can build and send a line-item quotation (24h review window); re-quote supersedes.	Must	<code>createQuotation()</code> + <code>quotationTotal()</code> + <code>status='superseded'</code> .
BR-15	As a Customer, I can see the quotation breakdown, total, expiry countdown, balance due date (T-3d), and balance amount.	Must	<code>serializeBooking()</code> computes <code>balanceDueDate/balanceAmount</code> .

6.4 PAYMENTS & PAYOUTS

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-16	As a Customer, I can pay a flat non-refundable deposit to confirm my accepted booking.	Must	<code>POST /pay-fee</code> → <code>type='deposit'</code> Billplz bill.
BR-17	As a Customer, I can pay the remaining balance after confirmation (due T-3d).	Must	<code>POST /pay-balance</code> → <code>type='balance'</code> .
BR-18	As a System, Billplz webhook signature (HMAC-SHA256 of raw body) must be verified timing-safe before mutating state.	Must	<code>verifyBillplzSignature()</code> + <code>POST /api/payments/webhook</code> .
BR-19	As a System, a deposit-paid webhook auto-confirms an <code>accepted</code> booking; balance-paid creates the artist payout.	Must	<code>handlePaymentPaid()</code> routed by <code>payment.type</code> .
BR-20	As a Platform, commission is computed artist-side from <code>platform_settings</code> ; waived below threshold.	Must	<code>computeCommission()</code> + <code>getCommissionRateBps()</code> clamp 0–50%.
BR-21	As an Admin, I can settle or fail a payout (24h post-event dispute window tracked).	Must	<code>updatePayoutStatus()</code> + <code>payouts.settleable_at</code> .
BR-22	As a Customer, I can request refund of the balance on a cancelled confirmed booking (deposit never refunded).	Should	<code>refundBalancePayment()</code> — deposit non-refundable.

6.5 COMMUNICATIONS

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-23	As a Participant, I can chat per-booking with the counterparty; messages stream live.	Must	<code>messages</code> table + <code>/messages/stream</code> SSE.
BR-24	As a User, I control email preferences (booking_created, quotation_sent, balance_reminder, etc.).	Must	<code>email_preferences</code> + <code>/api/email/preferences</code> .
BR-25	As a System, emails are retried (up to 3 attempts) and viewable in <code>/dev/emails</code> in dev.	Must	<code>email_outbox</code> + <code>email_retries</code> .

6.6 ADMINISTRATION & OPS

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-26	As an Admin, I can manage users, artists/studios (direct DB edits), bookings (status override), payments, payouts, quotations, messages, and settings.	Must	<code>/admin/*</code> pages + <code>/api/admin/*</code> under <code>requireAdmin()</code> .
BR-27	As an Admin, every mutation is audit-logged with admin identity, target, and details.	Must	<code>admin_audit_log</code> with FK to <code>users(id)</code> .
BR-28	As an Admin, I can tune booking fee / commission / waiver at runtime without deploy.	Must	<code>platform_settings</code> + <code>/admin/settings</code> .
BR-29	As an Admin, I can see dashboard metrics (users, bookings by status, revenue, recent activity).	Should	<code>GET /api/admin</code> aggregated stats.

6.7 PLATFORM & COMPLIANCE

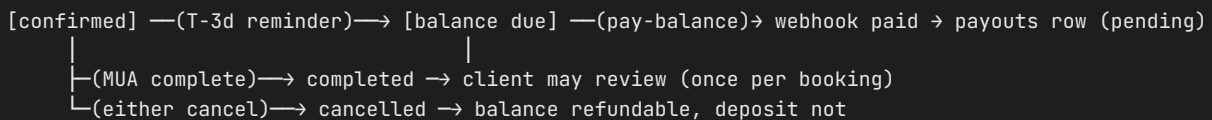
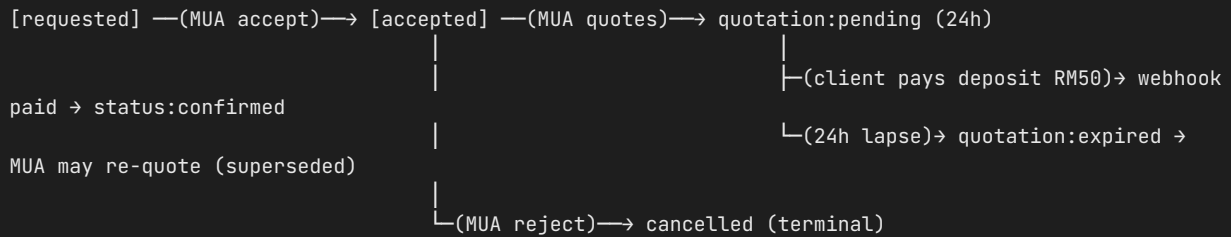
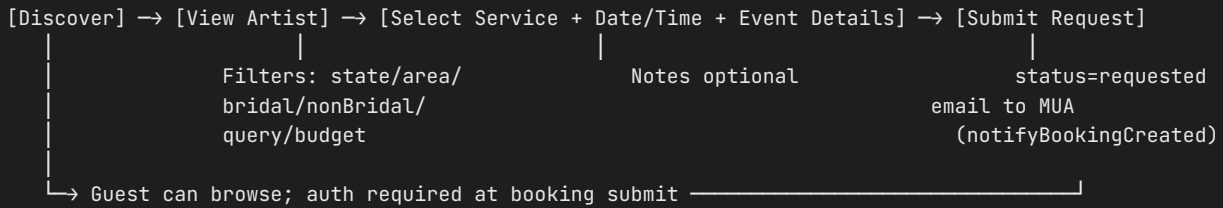
ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-30	As a System, logs are structured (pino) and forwardable to a webhook (LOG_WEBHOOK_URL).	Should	<code>src/server/logger.ts</code> batch 20 lines / 50ms debounce.
BR-31	As a System, runtime errors are reported to Sentry (if configured) or ERROR_WEBHOOK_URL.	Should	<code>reportError()</code> envelope.
BR-32	As a System, analytics events (register, login, booking, quotation, payment) are emitted when Agnost is configured.	Could	<code>src/server/agnost.ts</code> + <code>src/lib/agnost-client.ts</code> client tracking.

7. Business Rules (BRL-xxx)

ID	RULE	EXPRESSION	ENFORCEMENT
BRL-01	Booking slot atomicity	<code>UNIQUE (artist_id,date,time) WHERE status IN ('requested','accepted','confirmed')</code>	DB partial index + pre-check 409
BRL-02	Price authority	<code>bookings.price</code> and <code>quotations.total</code> computed server-side from catalog/inputs only	<code>getArtistById()</code> → <code>serviceDef.price</code> ; <code>quotationTotal()</code>
BRL-03	Amounts in sen	All monetary columns <code>INTEGER</code> (sen), never float	Schema + <code>Math.round()</code> in settings/quotations
BRL-04	Deposit non-refundable	On <code>cancelled</code> , only <code>payments.type='balance' AND status='paid'</code> is refundable	<code>refundBalancePayment()</code> guard
BRL-05	Confirmation only via payment	<code>requested → accepted</code> (artist) → <code>accepted → confirmed</code> (webhook only)	<code>confirmOnFeePaid()</code> called only from <code>handlePaymentPaid()</code> /webhook
BRL-06	Quotation TTL	<code>expires_at = created_at + 24h</code> ; lazy-checked on read + cron sweep	<code>isQuotationExpired()</code> + <code>findExpiredQuotations()</code>
BRL-07	Review gating	<code>reviews</code> requires <code>bookings.status='completed' AND reviews.booking_id IS NULL</code>	<code>findReviewableBooking()</code> + <code>UNIQUE booking_id</code>
BRL-08	Commission waiver	<code>if total < waiverSen → commission=0</code> <code>else round(total*rateBps/10000)</code>	<code>computeCommission()</code>
BRL-09	Commission cap	<code>0 ≤ rateBps ≤ 5,000</code>	Clamp in <code>getCommissionRateBps()</code>
BRL-10	Email verification gate	Payment creation requires <code>users.email_verified=1</code> (configurable strictness)	<code>pay-fee</code> / <code>pay-balance</code> handler
BRL-11	Profile claim scoping	<code>artist</code> / <code>studio</code> only see bookings for <code>artist_profiles/studio_profiles</code> they claimed	<code>getClaimedArtistIds()</code> filter in <code>GET /api/bookings</code>
BRL-12	Settlement window	<code>payouts.settleable_at = eventDate + 24h</code>	<code>createPayoutForBooking()</code>
BRL-13	Audit completeness	Every <code>PATCH/POST/DELETE /api/admin/*</code> writes <code>admin_audit_log</code>	<code>logAdminAction()</code> wrapper
BRL-14	Rate limiting	Auth + booking + webhook routes use sliding window (Upstash if configured, in-memory fallback)	<code>src/server/rate-limit.ts</code>
BRL-15	Session TTL	JWT expires in 7 days; JTI revocation respected; revoked=1 always wins; lookup failure is fail-open	<code>src/server/session.ts</code>

8. Key Business Processes & Journeys

8.1 CLIENT BOOKING JOURNEY (PRIMARY FLOW)



8.2 ARTIST JOURNEY

1. Register as `artist` → verify email → `/onboarding` → **claim** catalog profile (`POST /api/artist-profiles {artistId}`) — scopes bookings.
2. Dashboard: `GET /api/bookings` filtered to claimed `artist_id` (s) → act: `accept|reject|complete|cancel` .
3. On `accepted` : `POST /api/bookings/[id]/quotation` (base/travel/early-call/accommodation + extras ≤10, total computed server-side, `expires_at = now+24h` , supersedes previous `pending`) .
4. On `confirmed` : `POST /api/bookings/[id]/remind` (balance reminder), chat, invoice (`/invoice` + `/invoice.pdf`) .
5. After balance paid: payout appears `pending` (settleable 24h post-event) → admin settles via `/admin/payouts` .

8.3 PAYMENT & WEBHOOK FLOW

```

POST /pay-fee → createBookingPayment('deposit', bookingFeeSen)
├─ billplz: POST /bills (auth Basic API_KEY:) → {id, url} → INSERT
payments(type=deposit,status=required)
├─ dev: INSERT payments(type=deposit,status=required, provider_ref=dev_*)
└─ (dev) auto-settlement: markBillPaid + handlePaymentPaid → booking confirmed

Billplz -POST→ /api/payments/webhook (raw body + X-Billplz-Signature)
→ verifyBillplzSignature(rawBody, sig, API_KEY) [timingSafeEqual]
→ markBillPaid(billId) → handlePaymentPaid(payment) [routes by type]
├─ deposit: confirmOnFeePaid(status) → UPDATE bookings SET status='confirmed'
└─ balance: UPDATE quotations SET status='paid', createPayoutForBooking(...)

```


9. Success Metrics & KPIs

CATEGORY	METRIC	TARGET (LAUNCH + 90D)	SOURCE
Acquisition	Visitors → Artist view CTR	≥25%	Web analytics / Agnost
Conversion	Request → Accepted	≥60%	<code>bookings</code> status counts
Conversion	Accepted → Confirmed (deposit paid within 24h)	≥50%	<code>payments</code> deposit <code>paid</code>
Completion	Confirmed → Completed	≥85%	<code>bookings</code> lifecycle
Revenue	Gross booking value (GBV)	RM 50k cumulative	<code>quotations.total</code> sum
Take-rate	Net / GBV	8–10%	<code>payouts.commission_sen</code> / GBV
NPS	Post-completion survey	≥45	Reviews + survey
Ops	Webhook failure rate	<0.5%	<code>payments.status='failed'</code> + Sentry
Ops	Review fraud rate	~0%	Audit: <code>reviews</code> without valid booking
Engineering	Build + typecheck + lint clean	100% of merges	CI <code>quality-gate.yml</code>

Leading indicators tracked via `src/server/agnost.ts` + `src/lib/agnost-client.ts`: `trackArtistView`, `trackSearch`, `trackBookingForm`, bookings, payments.

10. Constraints, Assumptions & Dependencies

10.1 CONSTRAINTS

ID	CONSTRAINT
C-01	Must run on Vercel (serverless); no long-running processes — crons via <code>VERCEL_CRON</code> or external scheduler.
C-02	<code>DATABASE_URL</code> (PostgreSQL) in production; <code>node:sqlite</code> fallback for local/tests only — never in prod (<code>SKIP_DB_GUARD</code> only for debugging).
C-03	Billplz is the sole gateway at launch; no multi-PSP abstraction needed beyond <code>provider='billplz' 'dev'</code> .
C-04	Single deployable (Next.js); <code>output:"standalone"</code> for Docker parity.
C-05	Node ≥ 22 required (for <code>node:sqlite</code> DatabaseSync).

10.2 ASSUMPTIONS

ID	ASSUMPTION	VALIDATION PLAN
A-01	RM50 flat deposit is acceptable to clients	A/B price sensitivity after 100 bookings
A-02	10% commission + waiver is acceptable to artists	Artist interviews + churn watch
A-03	24h quotation window balances urgency vs deliberation	Funnel analysis (expiry vs payment)
A-04	T-3d balance due date is enforceable (reminder only for now)	Add dunning cron in v2.1 if delinquency >10%
A-05	Email is sufficient for transactional comms at launch	Add WhatsApp in v2.1 if open rate <40%

10.3 DEPENDENCIES

DEPENDENCY	TYPE	RISK IF UNAVAILABLE
Billplz API + Collection ID	External	<code>activePaymentProvider()</code> falls back to <code>dev</code> (no real money); feature-flagged warning
<code>NEXT_PUBLIC_SUPABASE_URL</code> + anon/service keys	External (storage/images)	Catalog images degrade to placeholder; admin Supabase pages gated
RESEND/POSTMARK/BREVO	External	Falls back to <code>email_outbox</code> (no real send, <code>dev</code> provider) — flagged in logs
Upstash Redis	Optional	In-memory rate limiting (single-instance degradation)
Vercel Blob / S3	Optional	Image uploads degrade; local fallback

11. Compliance & Legal

AREA	REQUIREMENT	IMPLEMENTATION
PDPA 2010 (Malaysia)	Consent captured; purpose limitation; access & correction; retention limit	<code>registerSchema.consent</code> , <code>users.consent+consent_timestamp</code> , <code>GET /api/me/export</code> , <code>DELETE /api/me</code> , <code>docs/PDPA_RETENTION_GUIDELINES.md</code>
Consumer Protection (KPDN)	Price transparency; refund policy disclosed	Quotation breakdown, invoice HTML/PDF, non-refundable deposit disclosed at pay-fee
Data Retention	Bookings/payouts retained for audit; PII deletable on request	Retention guideline doc; <code>DELETE /api/me</code> cascades <code>ON DELETE CASCADE</code>
Security disclosures	Headers, CSP, password handling	<code>next.config.ts</code> headers (<code>X-Frame-Options:DENY</code> , <code>X-Content-Type-Options:nosniff</code> , CSP with nonce), scrypt + pepper, httpOnly cookies
Financial	Settlement auditability	<code>payouts</code> + <code>admin_audit_log</code> immutable log

12. Risks & Mitigations

ID	RISK	LIKELIHOOD	IMPACT	MITIGATION
R-01	Double-booking same slot (race)	Medium	High	Partial unique index <code>uq_bookings_slot</code> + optimistic 409; never trust client price
R-02	Webhook forgery (fake paid)	Medium	Critical	<code>verifyBillplzSignature()</code> timing-safe before state change; apiKey as HMAC secret
R-03	Lost-update on rating	Low	Medium	<code>blendAggregate()</code> single SQL (no read-modify-write) + <code>ROUND(...,2)</code>
R-04	SQLite used in prod by misconfig	Low	Critical	<code>getDb()</code> throws in prod when <code>DATABASE_URL</code> missing (unless <code>SKIP_DB_GUARD</code>)
R-05	Email provider downtime	Medium	Medium	Fallback to dev outbox; retry queue <code>email_retries</code> max 3, <code>next_retry</code> backoff
R-06	Artist churn (take-rate too high)	Medium	High	Waiver threshold; configurable <code>platform_settings</code> ; artist dashboard transparency
R-07	Low conversion accepted→confirmed	Medium	Medium	24h expiry urgency + re-quote; analytics on funnel
R-08	PG pool exhaustion	Low	High	<code>PG_MAX</code> , <code>PG_CONNECTION_TIMEOUT_MS</code> , <code>PG_IDLE_TIMEOUT_MS</code> tunable; <code>idleTimeoutMillis</code> for Neon/serverless

13. Roadmap & Release Strategy

PHASE	MILESTONE	SCOPE	GATE
0 — Foundation	DB facade, auth, catalog	Tables, session, seeding, filters	Typecheck + tests pass
1 — Booking Loop	Request → accept → quote → fee → confirm	Bookings, quotations, payments (dev)	E2E (register→book→dashboard) green
2 — Money Hardening	Billplz, webhook, payouts, invoices	<code>verifyBillplzSignature</code> , <code>handlePaymentPaid</code> , <code>payouts</code>	Manual Billplz sandbox test + audit
3 — Comms	Chat, emails, preferences	SSE stream, <code>booking-emails</code> , <code>email_preferences</code>	Mail trap + load test
4 — Admin	Ops panel + audit	<code>/admin/*</code> , <code>admin_audit_log</code> , <code>platform_settings</code>	Admin UAT + seed-admin
5 — Launch	KL+Selangor, 10 MUAs	Seed catalog, health check, deploy	<code>npm run env:check</code> + <code>db:migrate/seed</code>

Post-launch (v2.1+): studio-native bookings, balance dunning cron, WhatsApp, product e-commerce, native loyalty.

14. Acceptance Criteria (Business Level)

☐

A guest can browse artists/studios and filter without authentication.

☐

A customer can register (with PDPA consent), verify email, and log in; sessions last 7 days.

☐

An artist can claim a profile and see only their bookings; `GET /api/bookings` is scoped.

☐

Double-booking a live slot returns a friendly 409 (never 500) and the unique index prevents the anomaly.

☐

Quotations are 24h expiry, re-quotable, and total is server-computed; expired ones are auto-marked.

☐

Deposits confirm bookings only via verified webhook (Billplz) or dev auto-settlement (test).

☐

Commission math matches `computeCommission()` spec with waivers and clamped rate.

☐

Payouts become settleable 24h after event date and are admin-settleable with audit.

☐

Reviews are possible only after `completed`, once per booking, with atomic rating update.

☐

Per-booking chat works live (SSE) and persists.

☐

Admin actions are audit-logged; non-admin cannot access `/admin` or `/api/admin/*`.

☐

PDPA export/delete work as documented; retention guidelines are followed.

☐

Build, lint, typecheck, tests (Vitest + Playwright), and coverage ≥80% all pass in CI.

15. Appendix

15.1 REFERENCES

- `AGENTS.md` — project overview, architecture patterns, security considerations.
- `docs/ARCHITECTURE.md` — single-backend data flow, money logic ownership.
- `docs/CODEBASE-AUDIT-FINAL.md` — enterprise-readiness gaps closed.
- `src/lib/data.ts` — constants (`MALAYSIA_STATES` , `BRIDAL_EVENTS` , `NON_BRIDAL_EVENTS` , `SEED_ARTISTS/STUDIOS`).
- `src/server/db.ts` — `PG_SCHEMA` / `SQLITE_SCHEMA` , `DbFacade` , migration logic.

15.2 GLOSSARY (SHORT)

See `docs/09-Glossary.md` for full glossary. Key terms: **Deposit**, **Balance**, **Quotation**, **Payout**, **Settlement**, **Claim**, **Slot**.

15.3 DOCUMENT CONTROL

VERSION	DATE	AUTHOR	CHANGE
1.0	2026-08-10	Product	Initial BRS covering booking→fee
2.0	2026-08-29	Product+Eng	Added hybrid deposit/ balance, payouts/ commission, admin, compliance

End of BRS — see `02-SRS-Software-Requirements-Specification.md` for system decomposition and `03-FRS-Functional-Requirements-Specification.md` for per-feature functional detail.

02-SRS-Software-Requirements-Specification

SRS — Software Requirements Specification

Leish! v2 — Beauty Booking Marketplace

Table of Contents

1. Introduction
2. Overall Description
3. System Features (Summary)
4. External Interface Requirements
5. System Features — Detailed (SRS-F)
6. Data Requirements
7. Other Requirements
8. Verification & Validation
9. Appendices

SRS — Software Requirements Specification

Leish! v2 — Beauty Booking Marketplace

FIELD	VALUE
Document ID	LEISH-SRS-v2.0
Version	2.0.0
Status	Baseline
Date	2026-08-29
Complies With	IEEE Std 830-1998 (SRS)
Predecessor	BRS (01)
Successor	FRS (03), NFR (04)
Repo	<code>src/</code> (Next.js 16 App Router, React 19, TypeScript)

Table of Contents

- [1. Introduction](#)
- [2. Overall Description](#)
- [3. System Features \(Summary\)](#)
- [4. External Interface Requirements](#)
- [5. System Features — Detailed \(SRS-F\)](#)
- [6. Data Requirements](#)
- [7. Other Requirements](#)
- [8. Verification & Validation](#)
- [9. Appendices](#)

1. Introduction

1.1 PURPOSE

This SRS translates the business requirements (BRS [01](#)) into verifiable software requirements. Its audience is architects, developers, QA, and auditors. It is the single reference for what the system **shall** do, under what constraints, and how compliance will be shown.

1.2 SCOPE

The system is **Leish! v2** — a web platform ([leish_v2](#)) that provides:

- Public catalog (artists/studios) with faceted discovery.
- Authenticated booking request → quotation → two-step payment → completion → review.
- Real-time per-booking messaging.
- Admin operations with audit trail and platform settings.
- Transactional email and observability.

Out of scope is listed in BRS §4.2 and restated in §2.5 below.

1.3 DEFINITIONS, ACRONYMS, ABBREVIATIONS

TERM	MEANING
MUA	Makeup Artist
MYR/sen	Ringgit; amounts stored as integer sen (1 MYR = 100 sen)
Billplz	Malaysian payment gateway (Billplz API v3)
DFA	DbFacade (src/server/db.ts) — PG or SQLite behind same async API
JWT/JTI	JSON Web Token / JWT ID (revocation key)
SSE	Server-Sent Events (chat live stream)
PDPA	Malaysia Personal Data Protection Act 2010
RQ	Requirement

See full glossary in [docs/09-Glossary.md](#).

1.4 REFERENCES

- BRS [01-BRS-Business-Requirements-Specification.md](#)
- [AGENTS.md](#) (project conventions), [docs/ARCHITECTURE.md](#), [docs/PDPA_RETENTION_GUIDELINES.md](#)
- Source of truth code: [src/server/db.ts](#), [src/server/session.ts](#), [src/server/bookings.ts](#), [src/server/payments.ts](#), [src/server/payouts.ts](#), [src/server/quotations.ts](#), [src/server/catalog.ts](#), [src/server/validation.ts](#)
- Standards: IEEE 830, OWASP ASVS L2, WCAG 2.1 AA

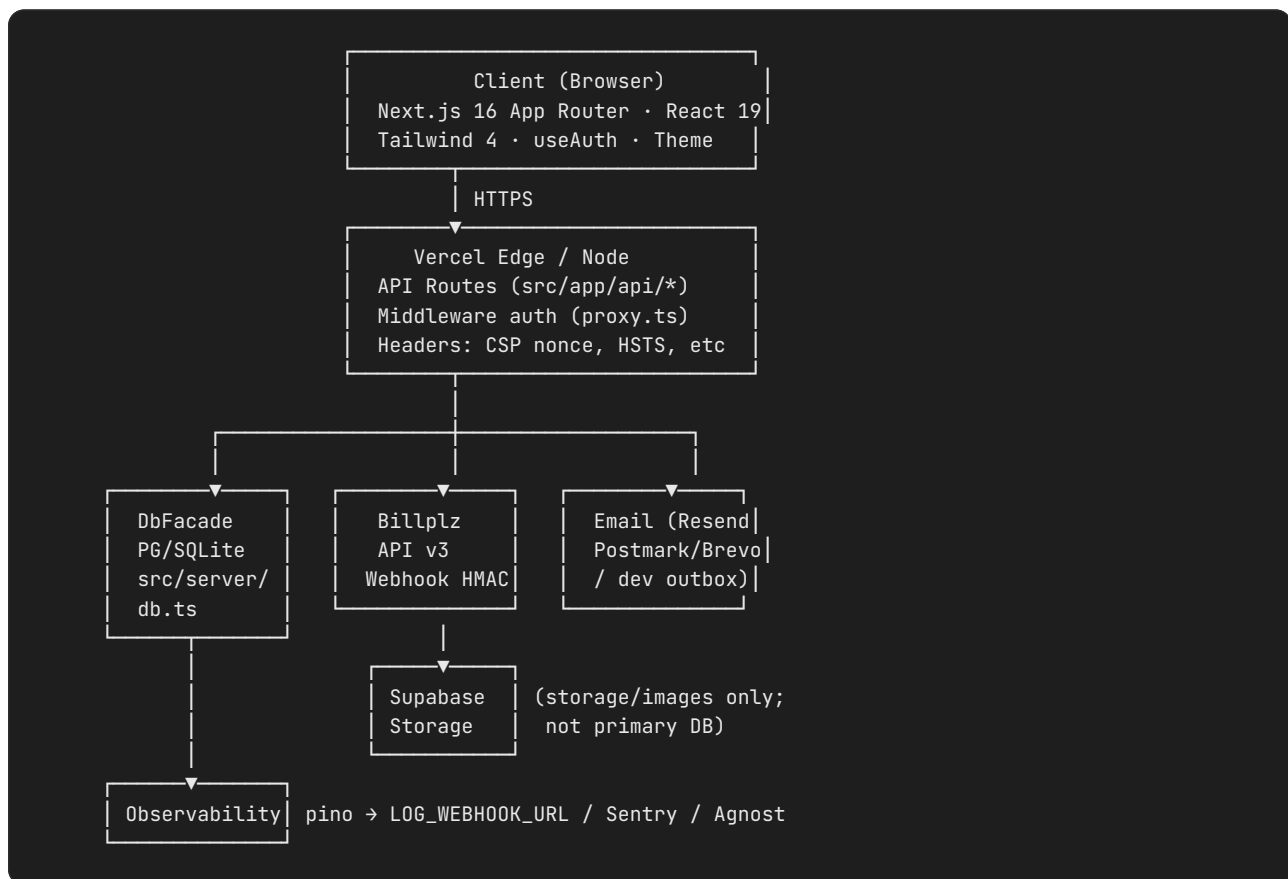
1.5 OVERVIEW

§2 gives the product context; §3 summarizes features; §4 defines interfaces; §5 decomposes each feature into verifiable SRS requirements with IDs; §6 details data; §7 covers adoption; §8 maps verification.

Conformance language: **shall** = mandatory, **should** = recommended, **may** = optional.

2. Overall Description

2.1 PRODUCT PERSPECTIVE



- **Single persistence:** `getDb()` facade (`src/server/db.ts:828`). `isPostgres()` switches between `pg.Pool` (production) and `node:sqlite DatabaseSync` (dev/tests). Schema is defined once per backend (`PG_SCHEMA` / `SQLITE_SCHEMA`) and kept in sync by `extractSchemaTables()` drift detection.
- **Stateless auth:** HS256 JWT (`jose`) signed with `SESSION_SECRET` , stored in `leish_session` httpOnly cookie, `SameSite=lax` , `Secure` in prod, 7-day TTL, JTI revocation table `sessions` .
- **Deployment:** Vercel preferred; Docker `standalone` output also supported; health at `GET /api/health` .

2.2 PRODUCT FUNCTIONS (SUMMARY)

1. Catalog browsing & search (guests and authed users).
2. Account lifecycle (register, login, verify, reset, consent).
3. Booking lifecycle with slot locking.
4. Quotation authoring & expiry.
5. Hybrid payments (deposit + balance) via Billplz/dev.
6. Payout & commission settlement.
7. Reviews (gated, atomic).
8. Real-time chat.
9. Admin CRUD + audit + settings.
10. Notifications, logging, analytics.

FRS (`03`) expands each function into `FR-xxx` with acceptance criteria.

2.3 USER CLASSES AND CHARACTERISTICS

CLASS	COUNT (LAUNCH)	TECHNICAL SKILL	FREQUENCY	PRIVILEGES
Guest	Unbounded	Low	Occasional	Read catalog, register
Customer	~500	Low	1–4/yr	Book, pay, chat, review, export/delete
Artist	~10–30	Low–Med	Daily (dashboard)	Claim profile, manage bookings, quote, complete
Studio	~4–10	Med	Daily	Same as artist (studio_id scoping)
Admin	2–5	High	Daily	Full CRUD + audit + settings + payouts
Cron/ System	—	—	Scheduled	Sweep expired quotes, email retries

Accessibility: WCAG 2.1 AA, responsive (mobile-first), dark/light theme persisted in `localStorage` (`src/lib/theme.tsx`).

2.4 OPERATING ENVIRONMENT

LAYER	REQUIREMENT
Runtime	Node ≥22 (for <code>node:sqlite</code>), Next.js 16.3.1, React 19.2.8
Hosting	Vercel (primary) or Docker <code>node:20-alpine</code> with <code>output:"standalone"</code>
DB — Prod	PostgreSQL (Neon/Supabase pooler) via <code>DATABASE_URL</code>
DB — Dev/Test	<code>node:sqlite</code> (<code>LEISH_DB_PATH</code> or <code>:memory:</code>)
Browsers	Evergreen Chrome/Firefox/Safari/Edge (last 2 versions)
Env	<code>.env.local</code> (prod), <code>.env.example</code> template; <code>src/env.ts</code> validates <code>SESSION_SECRET</code> in prod

2.5 DESIGN AND IMPLEMENTATION CONSTRAINTS

- Shall use **Next.js App Router** (no Pages Router).
- Shall not commit secrets — use Vercel/CI secrets; `SESSION_SECRET` must be 32 random bytes base64 (`openssl rand -base64 32`).
- Shall keep `PG_SCHEMA` and `SQLITE_SCHEMA` in sync (drift warning via `detectSchemaDrift()`).
- Shall never store money as float — `INTEGER sen` only.
- Shall not trust client-sent amounts — server computes `bookings.price` and `quotations.total` .
- Root `loading.tsx` is **forbidden** (breaks `notFound()` 404 semantics — see AGENTS.md §11).
- Absolute imports use `@/` alias to `./src/` .

2.6 ASSUMPTIONS AND DEPENDENCIES

See BRS §10.2/10.3. Additions:

- Assumes `jose` 6.2.8, `pg` 8.23.0, `pino` 10.3.1 remain compatible.
- Depends on Billplz collection setup (`BILLPLZ_COLLECTION_ID`) before live payments.
- Depends on Supabase project for image hosting; graceful fallback to `/images/*` placeholders if unavailable.

3. System Features (Summary)

FEATURE ID	FEATURE	PRIORITY	BRS TRACE
F-01	Authentication & Session	Must	BR-06..BR-10
F-02	Catalog (Artists/Studios)	Must	BR-01..BR-05
F-03	Booking Management	Must	BR-11..BR-13, BR-15
F-04	Quotations	Must	BR-14..BR-15
F-05	Payments (Hybrid)	Must	BR-16..BR-19, BR-22
F-06	Payouts & Commission	Must	BR-20..BR-21
F-07	Reviews & Ratings	Must	BR-04..BR-05
F-08	Messaging (Chat)	Must	BR-23
F-09	Artist/Studio Profile Claim	Must	BRL-11
F-10	Admin Panel & Audit	Must	BR-26..BR-29
F-11	Email & Notifications	Must	BR-24..BR-25
F-12	Observability & Analytics	Should	BR-30..BR-32
F-13	Compliance (PDPA)	Must	BR-10, BRL-13

4. External Interface Requirements

4.1 USER INTERFACES

- Design system:** Tailwind 4, CSS variables `--leish-header-from/to` (`#c9284b` family), `ThemeProvider` dark-by-default with inline anti-CLS script in `layout.tsx`.
- Pages** (`src/app/`): `/`, `/artists`, `/artists/[slug]`, `/artists/[slug]/book`, `/studios`, `/studios/[slug]`, `/login`, `/register`, `/forgot-password`, `/reset-password`, `/verify-email`, `/onboarding`, `/dashboard`, `/booking`, `/contact`, `/help`, `/privacy`, `/terms`, `/dev/emails` (dev only), `/admin/*`.
- Components** (`src/components/`): `Navbar`, `Footer`, `Button`, `ArtistCard`, `StudioCard`, `RatingStars`, `Logo`, `AdminShell`, `StatCard`, `Badge`, `TurnstileWidget`.
- Accessibility:** Keyboard navigation, focus-visible outlines, aria labels on filters/search, color contrast $\geq 4.5:1$, `prefer-reduced-motion` respected.
- Constraints:** No root `loading.tsx`; Suspense boundaries scoped to segments that don't call `notFound()`.

4.2 HARDWARE INTERFACES

None beyond standard web client/server. Server requires PG or local disk for SQLite (`data/leish.db` , WAL mode, `foreign_keys=ON`).

4.3 SOFTWARE INTERFACES

INTERFACE	PROTOCOL	MODULE	ENV VARS
PostgreSQL	<code>pg</code> Pool over TLS	<code>src/server/db.ts</code>	<code>DATABASE_URL</code> , <code>PG_MAX</code> , <code>PG_CONNECTION_TIMEOUT_MS</code> , <code>PG_IDLE_TIMEOUT_MS</code>
SQLite	<code>node:sqlite</code> DatabaseSync	<code>src/server/db.ts</code>	<code>LEISH_DB_PATH</code>
Billplz API v3	HTTPS POST <code>application/x-www-form-urlencoded</code> , Basic Auth	<code>src/server/payments.ts</code> , <code>src/lib/payments/billplz.ts</code>	<code>BILLPLZ_API_KEY</code> , <code>BILLPLZ_COLLECTION_ID</code> , <code>BILLPLZ_BASE_URL</code> , <code>BILLPLZ_X_SIGNATURE_KEY</code> , <code>BILLPLZ_CALLBACK_URL</code>
Billplz Webhook	HTTPS POST raw body, <code>X-Billplz-Signature</code> HMAC-SHA256 hex	<code>src/app/api/payments/webhook/route.ts</code>	same
Email — Resend	HTTPS JSON	<code>src/server/email.ts</code>	<code>EMAIL_PROVIDER=resend</code> , <code>RESEND_API_KEY</code> , <code>EMAIL_FROM</code>
Email — Postmark	HTTPS JSON	<code>src/server/email.ts</code>	<code>EMAIL_PROVIDER=postmark</code> , <code>POSTMARK_SERVER_TOKEN</code>
Email — Brevo	HTTPS JSON	<code>src/server/email.ts</code>	<code>EMAIL_PROVIDER=brevo</code> , <code>BREVO_API_KEY</code>
Supabase Storage	HTTPS (images)	<code>src/lib/supabase/*</code> , <code>next.config.ts</code> <code>images.remotePatterns</code>	<code>NEXT_PUBLIC_SUPABASE_URL</code> , <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>
Upstash Redis	REST	<code>src/server/rate-limit.ts</code>	<code>UPSTASH_REST_URL</code> , <code>UPSTASH_REST_TOKEN</code>
Turnstile	Widget + server verify	<code>src/components/TurnstileWidget</code> , <code>src/lib/turnstile-token.ts</code>	<code>NEXT_PUBLIC_TURNSTILE_SITE_KEY</code> , <code>TURNSTILE_SECRET_KEY</code>
Agnost AI	SDK + client track	<code>src/server/agnost.ts</code> , <code>src/lib/agnost-client.ts</code> , <code>src/instrumentation.ts</code>	<code>AGNOST_ORG_ID</code> , <code>NEXT_PUBLIC_AGNOST_ORG_ID</code>
Vercel Blob / S3	HTTPS	<code>src/app/api/upload/*</code>	<code>BLOB_READ_WRITE_TOKEN</code> , AWS S3 vars

4.4 COMMUNICATIONS INTERFACES

- **HTTP** only (HTTPS in prod). `Strict-Transport-Security` set via `next.config.ts`.
- **Security headers** (`next.config.ts`): `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, CSP with per-request nonce (no `unsafe-inline` for scripts except theme toggle).
- **Email**: via provider abstraction; outbox at `email_outbox` when `EMAIL_PROVIDER=dev`.
- **Chat SSE**: `GET /api/bookings/[id]/messages/stream` — `text/event-stream`, replay + live.
- **Health**: `GET /api/health` → 200 JSON.

5. System Features — Detailed (SRS-F)

Each feature lists its SRS requirements. Full functional decomposition with acceptance criteria lives in FRS (03).

5.1 SRS-F-01 — AUTHENTICATION & SESSION

ID	REQUIREMENT (SHALL)	VERIFICATION
SRS-F-01-01	Provide registration with fields <code>name</code> (2–80 chars), <code>email</code> (RFC), <code>password</code> (8–128 chars, script + optional <code>PEPPER_SECRET</code> HMAC), <code>role</code> in (<code>customer</code> , <code>artist</code> , <code>studio</code>), <code>consent</code> boolean + timestamp.	Unit (<code>src/server/validation.ts</code> <code>registerSchema</code>); e2e <code>register→login</code>
SRS-F-01-02	Hash passwords with <code>script</code> (salted, timing-safe compare); support <code>PEPPER_SECRET</code> pepper rotation via <code>PEPPER_VERSION</code> .	Auth tests; <code>src/server/auth.ts</code>
SRS-F-01-03	Issue HS256 JWT via <code>jose</code> , secret <code>SESSION_SECRET</code> (32-byte base64), payload includes <code>sub</code> , <code>role</code> , <code>jti</code> , <code>exp = now+7d</code> ; set <code>leish_session</code> cookie httpOnly, <code>SameSite=lax</code> , <code>Secure</code> in prod.	<code>src/server/session.ts</code> <code>createSessionToken</code>
SRS-F-01-04	Validate JWT on every authed route; check <code>sessions</code> table: <code>revoked=1</code> wins; missing JTI falls back to signature+expiry (fail-open, logged).	<code>verifySessionToken</code> tests
SRS-F-01-05	Revoke JTI on logout (<code>UPDATE sessions SET revoked=1 WHERE jti=?</code>).	<code>revokeSession</code> tests
SRS-F-01-06	Send single-use email verification link (token hashed sha256 at rest, configurable TTL), banner until <code>email_verified</code> ; gate payments until verified.	<code>src/server/email-verification.ts</code> ; <code>GET /api/auth/verify-email</code>
SRS-F-01-07	Provide forgot/reset password: single-use <code>password_resets</code> rows (hashed token, 1h expiry), rate-limited, no user enumeration (always 200-ish).	<code>src/app/api/auth/forgot-password/</code> *
SRS-F-01-08	Rate-limit auth endpoints (register/login/forgot/verify) with sliding window (Upstash or in-memory). Return 429 + <code>Retry-After</code> .	<code>src/server/rate-limit.ts</code>

5.2 SRS-F-02 — CATALOG

ID	REQUIREMENT	VERIFICATION
SRS-F-02-01	Store artists/studios in DB tables <code>artists</code> / <code>studios</code> (seeded from <code>src/lib/data.ts</code> via <code>src/server/catalog-seed.ts</code> idempotently; slugs == legacy ids).	<code>ensureCatalogSeeded()</code> ; <code>npm run db:seed-catalog</code>
SRS-F-02-02	Provide repository <code>listArtists(filters)</code> : SQL pre-filter <code>state/area/budget</code> (indexed), then <code>filterArtists()</code> for <code>query</code> /tags/date; <code>listAllArtists({limit,offset})</code> capped 500.	<code>src/server/catalog.ts</code>
SRS-F-02-03	Provide <code>getArtistById</code> , <code>getArtistBySlug</code> , <code>resolveArtist(idOrSlug)</code> (slug-first), same for studios.	Catalog tests
SRS-F-02-04	Validate discovery query via <code>artistsQuerySchema</code> (<code>MALAYSIA_STATES</code> , <code>BRIDAL_EVENTS</code> , <code>NON_BRIDAL_EVENTS</code>); support <code>state → area</code> cascading (<code>AREAS_BY_STATE</code>).	<code>src/server/validation.ts</code>
SRS-F-02-05	Allow admin direct column edits and artist self-service <code>PATCH /api/artist-profiles</code> (whitelisted fields only).	<code>updateArtist</code> / <code>updateStudio</code> field maps
SRS-F-02-06	Public catalog routes set <code>s-maxage=300</code> ; catalog pages are <code>force-dynamic</code> so edits surface without redeploy.	<code>src/app/artists/page.tsx</code> dynamic flag

5.3 SRS-F-03 — BOOKING MANAGEMENT

ID	REQUIREMENT	VERIFICATION
SRS-F-03-01	Create booking via <code>POST /api/bookings</code> — zod <code>bookingSchema</code> ; server resolves <code>price</code> from <code>artist.services.find(name)</code> — client price ignored.	Booking tests; <code>src/app/api/bookings/route.ts</code>
SRS-F-03-02	Record <code>event_type</code> , <code>venue</code> , <code>guest_count</code> , <code>notes</code> , <code>date</code> (ISO ≥ today), <code>time</code> string; initial <code>status='requested'</code> .	Schema + <code>bookings</code> table
SRS-F-03-03	Enforce slot uniqueness: partial unique index <code>uq_bookings_slot ON (artist_id,date,time) WHERE status IN (requested,accepted,confirmed)</code> ; friendly 409 on conflict (pre-check + race guard).	DB schema + booking route catch <code>UNIQUE</code>
SRS-F-03-04	Implement state machine <code>applyBookingTransition(current, action, {isOwner,role})</code> with rules per <code>src/server/bookings.ts:1</code> . Payment-only transition <code>confirmOnFeePaid</code> .	State machine unit tests
SRS-F-03-05	Expose <code>PATCH /api/bookings/[id]</code> for <code>accept/reject/complete/cancel</code> ; authz: <code>accept/reject/complete</code> requires claimed <code>artist / studio</code> ; <code>cancel</code> requires <code>isOwner</code> or artist.	Route tests
SRS-F-03-06	Serialize bookings with derived <code>quotation</code> , <code>totalPrice</code> , <code>balanceDueDate</code> (T-3d), <code>balanceAmount = total - bookingFeeSen</code> , <code>payment</code> + <code>balancePayment</code> .	<code>serializeBooking()</code>

5.4 SRS-F-04 — QUOTATIONS

ID	REQUIREMENT	VERIFICATION
SRS-F-04-01	Allow claimed artist to <code>POST /api/bookings/[id]/quotation</code> with <code>baseFee</code> , <code>travelFee</code> , <code>earlyCallFee</code> , <code>accommodationFee</code> , <code>extras[≤10]</code> , <code>artistNote</code> validated via <code>quotationSchema</code> .	<code>src/server/validation.ts</code>
SRS-F-04-02	Compute total server-side: <code>quotationTotal()</code> = sum of all line items (sen).	Unit test
SRS-F-04-03	Set <code>status='pending'</code> , <code>expires_at = now + 24h</code> (<code>QUOTATION_TTL_MS</code>); supersede previous pending (<code>superseded</code>).	<code>createQuotation()</code>
SRS-F-04-04	Lazily expire on read and via sweep: <code>isQuotationExpired()</code> , <code>findExpiredQuotations()</code> , <code>markQuotationExpired()</code> .	<code>src/server/quotations.ts</code>
SRS-F-04-05	Return 404 for already-expired quotations in booking serialization (<code>totalPrice → null</code> when expired).	<code>serializeBooking()</code>

5.5 SRS-F-05 — PAYMENTS

ID	REQUIREMENT	VERIFICATION
SRS-F-05-01	Provide hybrid payments: <code>payments.type IN ('deposit', 'balance') , UNIQUE(booking_id, type) , payments.status IN (required, paid, failed, refunded) .</code>	Schema + <code>src/server/payments.ts</code>
SRS-F-05-02	Create Billplz bill when <code>BILLPLZ_API_KEY</code> + <code>COLLECTION_ID</code> set, else <code>dev</code> payment row; body uses <code>amount</code> sen, <code>collection_id</code> , payer name/email from booking owner, <code>callback_url</code> , <code>redirect_url</code> , <code>reference_1=bookingId</code> .	<code>createBookingPayment()</code>
SRS-F-05-03	Expose <code>POST /api/bookings/[id]/pay-fee</code> (deposit, <code>getBookingFeeSen()</code>) and <code>POST /api/bookings/[id]/pay-balance</code> (quotation total – deposit); both authed, verified-email gated, dev auto-settles synchronously for e2e.	<code>pay-fee</code> / <code>pay-balance</code> routes
SRS-F-05-04	Handle webhook <code>POST /api/payments/webhook</code> — verify <code>X-Billplz-Signature</code> HMAC-SHA256 of raw body with <code>BILLPLZ_API_KEY</code> (timing-safe, hex64 check) before any state change.	<code>verifyBillplzSignature()</code>
SRS-F-05-05	Route settlement by type: <code>deposit</code> → <code>confirmOnFeePaid</code> then <code>UPDATE bookings SET status='confirmed' ; balance</code> → <code>UPDATE quotations SET status='paid' + createPayoutForBooking ;</code> idempotent.	<code>handlePaymentPaid()</code>
SRS-F-05-06	Allow refund of balance only on cancelled bookings when <code>status='paid' ; dev</code> marks refunded, <code>billplz</code> calls <code>POST /bills/{id}/refund</code> ; deposit never refunded.	<code>refundBalancePayment()</code>

5.6 SRS-F-06 — PAYOUTS & COMMISSION

ID	REQUIREMENT	VERIFICATION
SRS-F-06-01	Store platform settings <code>platform_settings(key,value)</code> for <code>booking_fee_sen</code> , <code>commission_rate_bps</code> (0–5000 clamp), <code>commission_waiver_sen</code> ; cached 30s.	<code>src/server/settings.ts</code>
SRS-F-06-02	Compute commission via <code>computeCommission(totalSen, rateBps, waiverSen)</code> — waived when <code>total < waiver</code> ; <code>commission = round(total*rate/10000)</code> .	Unit tests
SRS-F-06-03	On balance paid, create <code>payouts</code> row: <code>gross=total</code> , <code>commission</code> , <code>net = max(0, (total - commission) - deposit)</code> , <code>settleable_at = eventDate+24h</code> , <code>status='pending'</code> ; idempotent per <code>booking_id</code> .	<code>createPayoutForBooking()</code>
SRS-F-06-04	Provide <code>listPayouts(status?)</code> joined with bookings, and <code>updatePayoutStatus(id, settled failed, notes)</code> with <code>settled_at</code> when applicable; admin-only.	<code>src/server/payouts.ts</code> + <code>GET/PATCH /api/admin/payouts</code>

5.7 SRS-F-07 — REVIEWS & RATINGS

ID	REQUIREMENT	VERIFICATION
SRS-F-07-01	Allow adding review via <code>addEntityReview({entityType,entityId,bookingId,userId,authorName,rating 1-5,event,text})</code> ; guard <code>ALREADY_REVIEWED</code> when <code>booking_id</code> duplicate.	<code>src/server/catalog.ts</code> reviews section
SRS-F-07-02	Gate eligibility: <code>findReviewableBooking(userId, artistId)</code> returns most recent <code>status='completed'</code> booking left-joined against <code>reviews</code> (null → not yet reviewed). Studios not yet reviewable (returns null).	Query test
SRS-F-07-03	Blend rating atomically: <code>UPDATE artists SET review_count=review_count+1, rating=ROUND(((rating*review_count)+:rating)/(review_count+1),2)</code> — single statement, no read-modify-write.	<code>blendAggregate()</code>

5.8 SRS-F-08 — MESSAGING

ID	REQUIREMENT	VERIFICATION
SRS-F-08-01	Store <code>messages(id, booking_id FK, sender_id FK, body, created_at)</code> per-booking.	Schema
SRS-F-08-02	Expose <code>GET/POST /api/bookings/[id]/messages</code> (participants only) and <code>GET .../messages/stream</code> SSE (replay history then live via <code>src/server/chat-bus.ts</code>).	Route + e2e

5.9 SRS-F-09 — PROFILE CLAIM

ID	REQUIREMENT	VERIFICATION
SRS-F-09-01	Provide <code>artist_profiles(user_id PK, artist_id UNIQUE, claimed_at)</code> and <code>studio_profiles</code> — one claim per account.	Schema + <code>uq_*_profiles_*</code> indexes
SRS-F-09-02	Expose <code>GET /api/artist-profiles</code> (own profile or null), <code>POST /api/artist-profiles {artistId}</code> (scopes bookings), <code>PATCH /api/artist-profiles</code> (whitelisted self-edits).	<code>src/server/artist-profiles.ts</code>

5.10 SRS-F-10 — ADMIN PANEL & AUDIT

ID	REQUIREMENT	VERIFICATION
SRS-F-10-01	Guard all <code>src/app/api/admin/*</code> via <code>requireAdmin(request)</code> (401 anonymous, 403 non-admin).	<code>src/server/admin-auth.ts</code>
SRS-F-10-02	Guard <code>/admin/*</code> layout server-side redirect for non-admins.	<code>src/app/admin/layout.tsx</code>
SRS-F-10-03	Provide admin APIs: <code>GET /api/admin</code> dashboard aggregates, <code>/api/admin/users</code> CRUD, <code>/api/admin/artists</code> , <code>/api/admin/studios</code> , <code>/api/admin/bookings</code> , <code>/api/admin/payments</code> , <code>/api/admin/payouts</code> , <code>/api/admin/quotations</code> , <code>/api/admin/messages</code> , <code>/api/admin/emails</code> , <code>/api/admin/audit</code> , <code>/api/admin/settings</code> , <code>/api/admin/analytics</code> .	<code>src/app/api/admin/route.ts</code> listing test
SRS-F-10-04	Log every admin mutation to <code>admin_audit_log(id, admin_user_id FK, action, target_table, target_id, details JSON, created_at)</code> via <code>logAdminAction()</code> .	Audit table + tests
SRS-F-10-05	Allow seeding first admin idempotently: <code>ADMIN_EMAIL/ADMIN_PASSWORD npx tsx scripts/seed-admin.ts</code> upgrades existing users.	<code>scripts/seed-admin.ts</code>

5.11 SRS-F-11 — EMAIL & NOTIFICATIONS

ID	REQUIREMENT	VERIFICATION
SRS-F-11-01	Abstract <code>sendEmail()</code> over <code>EMAIL_PROVIDER=dev resend postmark brevo</code> ; missing creds fall back to dev outbox with warning (never silent).	<code>src/server/email.ts</code>
SRS-F-11-02	Compose booking emails in <code>src/server/booking-emails.ts</code> (created, quotation, balance reminder, status).	Module
SRS-F-11-03	Store <code>email_outbox</code> (dev), <code>email_preferences</code> per user (flags per event), <code>email_retries</code> (attempts ≤ 3 , <code>next_retry</code>).	Schema
SRS-F-11-04	Expose <code>/dev/emails</code> viewer (dev only) and <code>/api/email/preferences</code> toggle.	Routes

5.12 SRS-F-12 — OBSERVABILITY & ANALYTICS

ID	REQUIREMENT	VERIFICATION
SRS-F-12-01	Log with <code>pino</code> (<code>src/server/logger.ts</code>): JSON in prod, pretty in dev; <code>LOG_LEVEL</code> override; forward batches (20 lines / 50ms debounce) to <code>LOG_WEBHOOK_URL</code> .	Logger tests
SRS-F-12-02	Report errors via <code>reportError(err, context)</code> → pino + Sentry envelope if <code>SENTRY_DSN</code> else <code>ERROR_WEBHOOK_URL</code> .	<code>src/server/errors.ts</code>
SRS-F-12-03	Instrument with Agnost AI (<code>src/instrumentation.ts</code> <code>agnostai</code> init): <code>agnost.begin({userId, agentName, input})</code> → <code>interaction.end(output, success)</code> on register/login/booking/quotation/payment/profile-edit. Client: <code>trackEvent/trackArtistView/trackSearch/trackBookingForm</code> (<code>src/lib/agnost-client.ts</code>).	Agnost dashboard <code>3e27e121-654d-4746-ba55-7996f21bb351</code>

5.13 SRS-F-13 — COMPLIANCE

ID	REQUIREMENT	VERIFICATION
SRS-F-13-01	Implement <code>GET /api/me/export</code> (own data JSON) and <code>DELETE /api/me?confirm=1</code> (cascade delete via FKs).	<code>src/app/api/me/*</code>
SRS-F-13-02	Serve <code>/privacy</code> and <code>/terms</code> with templates from <code>docs/legal/*</code> .	Pages
SRS-F-13-03	Health check <code>GET /api/health</code> returns DB + env status.	<code>src/app/api/health/*</code> + Docker <code>HEALTHCHECK</code>

6. Data Requirements

6.1 LOGICAL DATA MODEL (SUMMARY)

See dedicated `docs/06-Data-Model-and-ERD.md` for full DDL, ER diagram, and field dictionaries. Summary here:

ENTITY GROUP	TABLES	PK/FK	NOTES
Users	users	PK id	role CHECK (customer artist studio admin); consent+consent_timestamp; script password
Catalog	artists, studios, reviews, referrals	PK id, slug UNIQUE for catalog	rating float, review_count, JSON arrays (specialties, services, etc.)
Claims	artist_profiles, studio_profiles	PK user_id → users, UNIQUE on artist_id / studio_id	Scopes bookings per actor
Booking	bookings	PK id, FK user_id → users, partial UNIQUE slot index	uq_bookings_slot WHERE active statuses
Quotation	quotations	PK id, FK booking_id → bookings ON DELETE CASCADE	Fee columns sen, extras JSON
Payments	payments, payouts, platform_settings	UNIQUE(booking_id, type) for payments	status CHECKs; computeCommission drives payouts
Comms	messages, email_outbox, email_preferences, email_retries, email_verifications, password_resets	FKs to users/bookings	Preference flags per event
Ops	sessions, admin_audit_log, catalog_overrides (legacy), platform_settings	FKs	Audit FK to users(id); tests must seed user row first

All tables include created_at (ISO string); updated_at where mutation is expected. Money columns are INTEGER sen.

6.2 DATA RETENTION

Per docs/PDPA_RETENTION_GUIDELINES.md: users/bookings retained for audit period after account deletion where legal hold exists; otherwise cascade delete. Quotations/payments retained ≥7 years for financial audit (configurable).

6.3 DATA INTEGRITY

- Foreign keys enforced (PRAGMA foreign_keys=ON for SQLite, real FKs for PG).
- Placeholder translation ?/@name → \$n tested in src/server/db.ts (compilePlaceholders, resolveParams).
- Drift detection warns when PG_SCHEMA and SQLITE_SCHEMA diverge.

7. Other Requirements

7.1 PERFORMANCE (TARGETS — SEE NFR DOC FOR DETAIL)

METRIC	TARGET
Catalog list (p50)	<250 ms (p95 <600 ms) via SQL pre-filter + index
Booking creation (p95)	<400 ms excluding email send (async)
Webhook handling (p95)	<300 ms
Image delivery	Next.js optimized, <code>*.supabase.co</code> via proxy, CDN

7.2 SAFETY & SECURITY (SUMMARY)

- Passwords: scrypt + pepper, never logged.
- Sessions: httpOnly, secure in prod, JTI revocation.
- Inputs: zod on every API boundary.
- Headers: CSP nonce, `DENY`, `nosniff`, `strict-origin-when-cross-origin`.
- Rate limiting: sliding window, 429.
- Audit: every admin mutation logged.

7.3 SOFTWARE SYSTEM ATTRIBUTES — SUMMARY

Quality attributes are elaborated in `docs/04-NFR-Non-Functional-Requirements.md` (availability, maintainability, portability, etc.).

8. Verification & Validation

8.1 TEST STRATEGY

LEVEL	TOOL	LOCATION	GATE
Unit	Vitest (jsdom)	<code>src/**/*.test.{ts,tsx}</code> (exclude <code>src/lib/data.ts</code>)	All green
Component	Testing Library	<code>src/components/**/*.test.tsx</code>	Green
Integration	Vitest with <code>LEISH_DB_PATH=:memory:</code> SQLite	<code>src/server/**/*.test.ts</code> , API route tests	Green
E2E	Playwright (<code>chromium</code>)	<code>e2e/</code>	Green; browser installed via <code>playwright install --with- deps chromium</code>
Contract	Zod validation tests	<code>src/server/validation.test.ts</code>	Green
Security	ESLint, <code>npm audit</code> , rate- limit tests	CI	0 warnings
Quality Gate	<code>quality-gate.yml</code> : lint, typecheck, test, coverage ≥80%, format	CI	Required per PR
DB Dual Path	<code>vitest.pg.config.mts</code> when <code>DATABASE_URL</code> set	CI <code>database.yml</code> when <code>db.ts migrate.ts</code> changes	Green

8.2 ACCEPTANCE TESTS (TRACE TO SRS)

SRS-F	E2E / TEST SCENARIO
F-01	Register with role → receive verification banner → resend → verify → login → 7d session persists across refresh
F-02	Browse <code>/artists</code> unauthenticated → filter state→area, bridal tag, budget, query → detail page resolves by slug
F-03	Customer creates booking for future date → second concurrent request to same slot returns 409 → artist accepts
F-04	Artist sends quotation → customer sees 24h countdown → expiry auto-marks + re-quote flow
F-05	Customer pays deposit (dev → auto-confirm) → status <code>confirmed</code> → pays balance → payout created
F-07	After <code>completed</code> , customer leaves review → second review for same booking rejected → rating rounded 2dp
F-08	Chat SSE receives history + live messages; only participants can post
F-10	Non-admin blocked from <code>/admin</code> + <code>/api/admin/*</code> ; admin actions audited
F-11	Verification email lands in <code>/dev/emails</code> when provider=dev
F-13	Export returns owned data; delete removes account and cascades bookings

9. Appendices

9.1 FEASIBILITY

Technical feasibility confirmed on Node 22 + pg 8.23 + jose 6.2.8 + pino 10.3. Tested locally via SQLite and in CI via PG. Billplz sandbox tested manually. No blocking unknowns.

9.2 FUTURE ENHANCEMENTS (BACKLOG)

- Studio-native `studio_id` booking path (Option B already scaffolded in `bookings` table) with distinct commission tiers.
- Product catalog (`products` type/table `src/lib/types.ts:89`) with cart/checkout.
- Referral rewards (`referrals` table) activation and commission share.
- WhatsApp notifications, advanced search (Vectorize), promotions engine.

9.3 DOCUMENT CHANGE HISTORY

VERSION	DATE	CHANGE
1.0	2026-08-15	Initial SRS — booking→fee single-path
2.0	2026-08-29	Hybrid deposit/balance, payouts, messaging, admin audit, Agnost

Next: [03-FRS-Functional-Requirements-Specification.md](#) details every FR with actor, pre/post conditions, flow, and acceptance criteria.

03-FRS-Functional-Requirements-Specification

FRS — Functional Requirements Specification

Leish! v2 — Beauty Booking Marketplace

Table of Contents

1. Conventions
2. Actor Definitions
3. Functional Decomposition Map
4. FR-01 Authentication & Session
5. FR-02 Catalog & Discovery
6. FR-03 Booking Lifecycle
7. FR-04 Quotations
8. FR-05 Payments (Hybrid Deposit/Balance)
9. FR-06 Payouts & Commission
10. FR-07 Reviews & Ratings
11. FR-08 Messaging & Chat
12. FR-09 Profile Claim & Self-Service
13. FR-10 Admin Panel & Audit
14. FR-11 Email & Notifications
15. FR-12 Dashboard & UX
16. FR-13 Invoicing & Documents
17. FR-14 Compliance & Data Rights
18. FR-15 Platform Settings & Ops
19. Summary Table & MoSCoW
20. Appendix — State Machines & Flows

FRS — Functional Requirements Specification

Leish! v2 — Beauty Booking Marketplace

FIELD	VALUE
Document ID	LEISH-FRS-v2.0
Version	2.0.0
Date	2026-08-29
Status	Baseline
Predecessor	SRS (02), BRS (01)
Method	Use-case driven, MoSCoW prioritized
Notation	FR-XXX shall; pre/post conditions; primary/alternate flows

Table of Contents

1. [Conventions](#)
2. [Actor Definitions](#)
3. [Functional Decomposition Map](#)
4. [FR-01 Authentication & Session](#)
5. [FR-02 Catalog & Discovery](#)
6. [FR-03 Booking Lifecycle](#)
7. [FR-04 Quotations](#)
8. [FR-05 Payments \(Hybrid Deposit/Balance\)](#)
9. [FR-06 Payouts & Commission](#)
10. [FR-07 Reviews & Ratings](#)
11. [FR-08 Messaging & Chat](#)
12. [FR-09 Profile Claim & Self-Service](#)
13. [FR-10 Admin Panel & Audit](#)
14. [FR-11 Email & Notifications](#)
15. [FR-12 Dashboard & UX](#)
16. [FR-13 Invoicing & Documents](#)

- 17. [FR-14 Compliance & Data Rights](#)
- 18. [FR-15 Platform Settings & Ops](#)
- 19. [Summary Table & MoSCoW](#)
- 20. [Appendix — State Machines & Flows](#)

1. Conventions

- **ID:** `FR-<module>-<seq>` e.g. `FR-AUTH-01`.
- **Priority:** **Must** / **Should** / **Could** / **Won't** (MoSCoW).
- **Shall** = verifiable requirement; **Preconditions** must hold before execution; **Postconditions** hold after.
- **Source file** cites the authoritative implementation; tests must exercise it.
- **Acceptance criteria** written as Given/When/Then or checklist.

2. Actor Definitions

ACTOR	AUTH	ROUTE EXAMPLE	DETAILS
Guest	None	<code>/artists</code> , <code>/studios</code>	Can browse, search, view detail. Redirected to <code>/login?redirect=/dashboard</code> on gated actions.
Customer	JWT <code>customer</code>	<code>POST /api/bookings</code>	Books, pays, chats, reviews.
Artist	JWT <code>artist</code>	<code>PATCH /api/bookings/[id]</code>	Claims <code>artists.id</code> , manages bookings for claimed profiles.
Studio	JWT <code>studio</code>	same	Studio variant; scopes by <code>studio_id</code> when claimed.
Admin	JWT <code>admin</code>	<code>/api/admin/*</code>	Full ops; all mutations audited.
System	<code>CRON_SECRET</code> / internal	<code>POST /api/payments/webhook</code>	Billplz, cron, Agnost.

Role source: `src/lib/types.ts:1` (`Role`), DB `users.role CHECK (customer|artist|studio|admin)` `src/server/db.ts:212`.

3. Functional Decomposition Map

```
Leish v2
├── FR-01 Auth & Session
│   ├── Registration (role, consent, scrypt+pepper)
│   ├── Login / Logout / Session (JWT+JTI)
│   ├── Email verification (single-use, banner)
│   └── Password reset (forgot/reset, no enumeration)
├── FR-02 Catalog
│   ├── List/filter artists & studios
│   ├── Detail (slug→id resolution)
│   └── Admin + self-service edits
├── FR-03 Booking
│   ├── Create (price server-derived, slot-lock)
│   ├── List (scoped: owner vs claimed-artist)
│   └── Transitions (accept/reject/complete/cancel)
├── FR-04 Quotation (line-items, 24h TTL, re-quote)
├── FR-05 Payments (deposit, balance, webhook, refund)
├── FR-06 Payouts & Commission (compute, create, settle/fail)
├── FR-07 Reviews (gated, once-per-booking, atomic blend)
├── FR-08 Messaging (persist + SSE stream)
├── FR-09 Profile Claim (artist/studio scoping)
├── FR-10 Admin & Audit
├── FR-11 Email (provider abstraction, preferences, retries)
├── FR-12 Dashboard
├── FR-13 Invoicing (HTML + PDF)
├── FR-14 Compliance (export/delete)
└── FR-15 Settings & Ops (platform_settings, health, agnost)
```

4. FR-01 Authentication & Session

FR-AUTH-01 — REGISTRATION

FIELD	DETAIL
Pre	Guest, no active session for same email; <code>SESSION_SECRET</code> set in prod; <code>PEPPER_SECRET</code> optional
Input	<code>registerSchema</code> <code>src/server/validation.ts:5</code> — <code>name</code> 2–80, <code>email</code> lowercased RFC, <code>password</code> 8–128, <code>role</code> ∈ {customer,artist,studio}, <code>consent</code> bool, <code>consentTimestamp</code> ISO
Processing	Normalize email lower/trim; check <code>users.email</code> UNIQUE; hash with <code>scrypt</code> (salt 32B, keyLen 64, N=16384, r=8, p=1) + optional <code>PEPPER_SECRET</code> HMAC before hashing; <code>INSERT users (id, email, name, role, password, consent, consent_timestamp, created_at);</code> create <code>email_verifications</code> token (random, sha256 hash at rest); set <code>leish_session</code> cookie (see FR-AUTH-03); emit <code>agnost.begin(agentName="register")</code>
Post	User row created; <code>201</code> with <code>{user: toPublicUser, devVerifyUrl}</code> (dev); session cookie set; verification email queued (or outbox in dev)
Priority	Must
Source	<code>src/app/api/auth/register/route.ts</code> , <code>src/server/validation.ts</code> , <code>src/server/session.ts</code>
Acceptance	✓ Zod rejects weak/short name, invalid email, short password, invalid role. ✓ Duplicate email returns 409 without leaking enumeration (generic message). ✓ Password stored is not plaintext; <code>timingSafeEqual</code> used on verify. ✓ Response includes <code>devVerifyUrl</code> when <code>EMAIL_PROVIDER=dev</code>. ✓ Agnost <code>register</code> event emitted when <code>AGNOST_ORG_ID</code> set.

FR-AUTH-02 — LOGIN

FIELD	DETAIL
Input	<code>loginSchema</code> <code>src/server/validation.ts:14</code> — email + password (1–128)
Processing	Lookup <code>users</code> by lowercased email; script verify with pepper if set; on success rotate JTI: <code>INSERT sessions(jti,user_id,expires_at,created_at)</code> ; sign HS256 JWT <code>{sub,role,emailVerified, jti, iat, exp=iat+7d}</code> via <code>jose</code> ; set cookie; <code>agnost.begin(agentName="login")</code> end(true/false)
Post	<code>200 {user,}</code> + cookie; failed → <code>401</code> generic "Invalid credentials" (no enumeration)
Priority	Must
Source	<code>src/app/api/auth/login/route.ts</code>
Acceptance	✓ Correct creds → cookie present with <code>httpOnly; Path=/; SameSite=lax; Secure@prod</code> . ✓ Wrong password → 401 with no hint which field failed. ✓ Rate-limited after N attempts → 429 + <code>Retry-After</code> .

FR-AUTH-03 — SESSION COOKIE & VERIFICATION

FIELD	DETAIL
Shall	The system shall issue and verify the <code>leish_session</code> cookie as HS256 JWT per <code>src/server/session.ts</code> .
Processing	<code>createSessionToken({sub,role,...})</code> signs with <code>SESSION_SECRET</code> (must be ≥32 bytes in prod or throw). <code>verifySessionToken(token)</code> verifies signature via <code>jose.jwtVerify</code> , checks <code>exp</code> , then looks up <code>sessions</code> <code>WHERE jti=?</code> : if <code>revoked=1</code> → reject; if row missing → accept if sig+expiry pass (fail-open with warning); if <code>expires_at < now</code> → reject.
Priority	Must
Acceptance	✓ Token valid for 7 days (<code>SESSION_TTL_SECONDS = 604800</code>). ✓ Tampered payload → verification fails. ✓ Revoked JTI → rejected even if not expired. ✓ Missing JTI row → accepted on sig+expiry (infra degraded case).

FR-AUTH-04 — LOGOUT / REVOCATION

FIELD	DETAIL
Endpoint	<code>POST /api/auth/logout</code>
Processing	Extract <code>leish_session</code> from <code>Cookie</code> ; decode JTI without failing closed; <code>revokeSession(jti)</code> → <code>UPDATE sessions SET revoked=1 WHERE jti=?</code> ; clear cookie (<code>Max-Age=0</code>).
Priority	Must
Acceptance	✓ After logout, same token is rejected (revoked wins). ✓ Logout without session still clears cookie and returns 200.

FR-AUTH-05 — EMAIL VERIFICATION

FIELD	DETAIL
Endpoints	<code>GET /api/auth/verify-email?token=...</code> (redirect flow), <code>POST /api/auth/resend-verification</code> (authed, rate-limited)
Processing	On register: <pre>email_verifications(id,user_id,token_hash=sha256(token),expires_at~24h,used_at null) ; link {NEXT_PUBLIC_SITE_URL}/verify-email?token=raw</pre> On verify: hash lookup, check <code>used_at IS NULL && expires_at>now</code> , then <code>UPDATE users SET email_verified=1</code> + <code>UPDATE email_verifications SET used_at=now</code> . Unverified banner shown in <code>/dashboard</code> until verified with resend button (states idle/sending/sent).
Priority	Must
Acceptance	✓ Verification link single-use (second use fails). ✓ Expired token returns friendly error (not stack). ✓ Resend is rate-limited per user. ✓ Banner disappears immediately after verification. ✓ Payment routes can enforce <code>email_verified</code> when strict mode enabled (see FR-PAY-02).

FR-AUTH-06 — FORGOT / RESET PASSWORD

FIELD	DETAIL
Endpoints	POST /api/auth/forgot-password {email, turnstileToken?} , POST /api/auth/reset-password {token,newPassword}
Processing	password_resets(id,user_id,token_hash,expires_at~1h,used_at,created_at) . On forgot: rate-limit by IP; lookup user (always return 200-ish to avoid enumeration); if exists generate token, hash+store, send email (outbox in dev). On reset: verify hash, single-use, expiry, then script new password and mark used. PEPPER_SECRET applied if set.
Priority	Must
Acceptance	✓ Non-existent email returns success-shaped envelope (no enumeration). ✓ Token is single-use and 1h expiry. ✓ New password must meet 8–128 length. ✓ After reset, old password rejected. ✓ Dev response includes devResetUrl when provider is dev.

FR-AUTH-07 — ROLE HANDLING & GUARDS

FIELD	DETAIL
Shall	Every authed route shall resolve user from verified JWT + DB row; admin routes additionally call requireAdmin(request) (401 if unauthenticated, 403 if role $\neq$ admin) src/server/admin-auth.ts . Client AuthProvider (src/lib/auth.tsx) mirrors ROLE_LABELS for UX.
Acceptance	✓ /dashboard and /onboarding middleware redirect unauthenticated to /login . ✓ artist cannot hit /api/admin/* (403). ✓ requireAdmin is called first in every admin handler before any DB read.

5. FR-02 Catalog & Discovery

FR-CAT-01 — PUBLIC ARTIST LISTING

FIELD	DETAIL
Endpoint	<code>GET /api/artists?</code> <code>query=&state=&area=&bridal=&nonBridal=&budget=</code> (plus <code>/api/catalog/artists</code> wrapper for client components)
Processing	<code>artistsQuerySchema</code> <code>src/server/validation.ts:52</code> <code>validates; listArtists(filters)</code> <code>src/server/catalog.ts:198</code> does SQL <code>WHERE state=? AND area=? AND price_from ≤ ?</code> (indexed <code>idx_artists_state_area</code>), then <code>filterArtists()</code> for <code>query</code> (name/tagline/bio/specialties), <code>bridal</code> / <code>nonBridal</code> <code>≠ "any"</code> , and <code>date</code> available. <code>listAllArtists({limit,offset})</code> caps at 500.
Priority	Must
Acceptance	✓ Filter by Selangor returns only Selangor artists. ✓ Budget filters <code>price_from ≤ budget</code> when <code>budget > 0</code>. ✓ Empty result set returns <code>[]</code> not error. ✓ Pagination <code>{limit,offset}</code> respected and clamped (1-500).

FR-CAT-02 — ARTIST/STUDIO DETAIL RESOLUTION

FIELD	DETAIL
Shall	The system shall resolve an artist by slug first then by id: <code>resolveArtist(idOrSlug) = getArtistBySlug </code> <code>getArtistById (src/server/catalog.ts:256)</code> . Same for studios (<code>resolveStudio</code>). <code>slug</code> equals legacy id for seeded rows (e.g. <code>aisha-azman</code>) so old links keep working.
Routing	<code>src/app/artists/[slug]/page.tsx</code> and <code>src/app/studios/[slug]/page.tsx</code> are <code>force-dynamic</code> (no stale static).
Acceptance	✓ <code>/artists/aisha-azman</code> resolves seeded artist. ✓ Unknown slug returns 404 via <code>notFound()</code> (not 200 shell — no root loading boundary). ✓ Admin-created artists with UUID <code>id</code> resolve by <code>slug</code> and <code>id</code>.

FR-CAT-03 — STATE→AREA CASCADING & VALIDATION

FIELD	DETAIL
Constants	<code>MALAYSIA_STATES</code> (16 entries), <code>AREAS_BY_STATE</code> mapping <code>src/lib/data.ts:3</code> , <code>BRIDAL_EVENTS</code> (5), <code>NON_BRIDAL_EVENTS</code> (6).
Processing	Client (<code>ArtistsBrowser</code>) cascades area list when state selected; server validates <code>state ∈ MALAYSIA_STATES</code> and <code>area string ≤80</code> ; <code>bridal ∈ BRIDAL_EVENTS.id</code> , <code>nonBridal ∈ NON_BRIDAL_EVENTS.id</code> .
Priority	Must
Acceptance	✓ Invalid <code>state</code> returns 400 zod error; area not required when state absent.

FR-CAT-04 — CATALOG MUTATIONS (ADMIN & SELF-SERVICE)

FIELD	DETAIL
Admin	<code>PATCH/POST /api/admin/artists</code> and <code>/api/admin/studios</code> via <code>updateArtist()/updateStudio()</code> with whitelists <code>ARTIST_UPDATE_FIELDS</code> / <code>STUDIO_UPDATE_FIELDS</code> <code>src/server/catalog.ts:305</code> ; JSON fields stringified; <code>updated_at=now</code> . <code>createArtist()/createStudio()</code> derive de-duplicated <code>slug</code> via <code>slugifyName()</code> , assign referral code, optionally link <code>referralCode</code> .
Self-service	<code>PATCH /api/artist-profiles</code> for claimed artists — same field whitelist minus <code>verified</code> / <code>referralEarnings</code> (audit).
Acceptance	✓ Unauthorized field names are ignored (not error, not written). ✓ Empty src uses <code>/images/hero.jpg</code> placeholder (prevents next/image crash). ✓ Mutations appear immediately due to <code>force-dynamic</code> (no CDN stale).

6. FR-03 Booking Lifecycle

FR-BOOK-01 — CREATE BOOKING

FIELD	DETAIL
Endpoint	POST /api/bookings — auth required
Input	bookingSchema src/server/validation.ts:19 — artistId (min1), service (min1), date YYYY-MM-DD regex, date ≥ today, time min1, notes ≤2000, eventType 1-80, venue ≤200, guestCount 0-1000
Processing	Resolve artist via getArtistById(artistId) else 404; verify service ∈ artist.services[].name else 400; construct BookingRow with artist_name=artist.name, price=serviceDef.price (server-derived), status='requested'; pre-check SELECT id FROM bookings WHERE artist_id=? AND date=? AND time=? AND status IN (requested,accepted,confirmed) → 409 if exists; then INSERT with race guard catching UNIQUE / duplicate key → 409; notifyBookingCreated() emails; Agnost booking- create begin/end; return serializeBooking()
Pre	User authed; if !email_verified, log info but allow (gate moved to payment); artistId exists; slot free
Post	Row bookings.id=randomUUID() created; response 201 {booking,user}
Priority	Must
Acceptance	✓ Client-sent price/duration ignored; server resolves from catalog. ✓ Past date rejected (zod refine). ✓ Concurrent double-book race returns 409 (never 500) — unique partial index is hard guarantee. ✓ Cancelled/completed slots don't block (WHERE status IN (...)).

FR-BOOK-02 — LIST BOOKINGS (SCOPED)

FIELD	DETAIL
Endpoint	GET /api/bookings?limit=&offset= — auth required
Processing	If <code>role ∈ (artist,studio)</code> : fetch <code>getClaimedArtistIds(user.id)</code> + <code>getClaimedStudioIds(user.id)</code> ; <code>claimed =</code> <code>studioIds.length>0 ? studioIds : artistIds</code> when <code>role=studio</code> ; <code>column =</code> <code>studioIds.length>0?"studio_id":"artist_id"</code> ; if <code>claimed.length==0</code> → <code>bookings=[]</code> ; else <code>SELECT * FROM bookings WHERE \${column} IN (?) ORDER</code> <code>BY date DESC,time DESC LIMIT ? OFFSET ?</code> plus count query. If customer: <code>WHERE user_id=?</code> . Serialize each via <code>serializeBooking()</code> (fetch deposit/balance payments, active quotation, balanceDueDate/balanceAmount). Pagination envelope <code>{bookings, pagination:</code> <code>{total,limit,offset,hasMore}}</code> . <code>limit</code> default 20 max 100.
Acceptance	✓ Artist sees only bookings for profiles they claimed (empty when none claimed). ✓ Customer sees only own <code>user_id</code> bookings. ✓ Studio with <code>studio_id</code> claim sees studio bookings; fallback to <code>artist_id</code> for legacy claims.

FR-BOOK-03 — BOOKING STATE TRANSITIONS

FIELD	DETAIL
Endpoint	PATCH /api/bookings/[id] {action: "accept" "reject" "complete" "cancel"} — auth required
State Machine	<pre>applyBookingTransition(current, action, {isOwner, role})</pre> <code>src/server/bookings.ts:40</code> : • <code>accept</code> : only artists, only from <code>requested</code> → <code>accepted</code> • <code>reject</code> : only artists, only from <code>requested</code> → <code>cancelled</code> • <code>complete</code> : only artists, only from <code>confirmed</code> → <code>completed</code> • <code>cancel</code> : from non-terminal (<code>!completed && !cancelled</code>), requires <code>isOwner artist</code> → <code>cancelled</code> <code>confirmOnFeePaid(current)</code> : only from <code>accepted</code> → <code>confirmed</code> (webhook only; not exposed via this PATCH).
Acceptance	✓ Artist <code>reject</code> on <code>accepted</code> returns error "Only requested..." ✓ Customer <code>accept</code> returns "Only artists..." ✓ Double <code>cancel</code> on <code>cancelled</code> returns "no longer be cancelled" ✓ Admin cannot bypass machine (admin uses separate <code>/api/admin/bookings</code> override).

FR-BOOK-04 — BOOKING SERIALIZATION & DERIVED FIELDS

FIELD	DETAIL
Shall	Every booking read shall be serialized via <code>serializeBooking(b: BookingRow)</code> <code>src/app/api/bookings/route.ts:32</code> : <code>getActiveQuotation(b.id)</code> , <code>getBookingFeeSen()</code> , <code>balanceDueDate = eventDate - 3d</code> (ISO), <code>totalPrice = quo</code> <code>max(0, total - fee)</code> . Return <code>{id, artistId, studioId, artistName, service, price, date, time, notes, status, eventType, venue, guestCount, quotat</code>
Acceptance	✓ <code>balanceAmount</code> never negative. ✓ Expired quotation yields <code>totalPrice=null</code> (not stale total). ✓ When no quotation, <code>totalPrice=null</code>.

7. FR-04 Quotations

FR-QUO-01 — CREATE / RE-QUOTE

FIELD	DETAIL
Endpoint	<code>POST /api/bookings/[id]/quotation</code> — claimed artist only; booking must be <code>accepted</code>
Input	<code>quotationSchema</code> <code>src/server/validation.ts:34</code> — <code>baseFee</code> 0–10M sen int, <code>travelFee/earlyCallFee/accommodationFee</code> optional 0–10M, <code>extras</code> ≤10 items <code>{label 1-80, amount 0-10M}</code> , <code>artistNote</code> ≤1000
Processing	Validate ownership via claimed ids; check booking <code>status==='accepted'</code> ; <code>quotationTotal(input)</code> sums all line items; supersede: <code>UPDATE quotations SET status='superseded' WHERE booking_id=? AND status='pending'</code> ; create row <code>id=randomUUID()</code> , <code>status='pending'</code> , <code>created_at=now</code> , <code>expires_at=now+24h</code> ; <code>bind(row)</code> insert; notify <code>notifyQuotationSent</code> email; Agnost <code>quotation</code> event.
Post	<code>201 {quotation: serializeQuotation(row)}</code>
Acceptance	✓ Amounts outside 0–10M rejected. ✓ 11th extra rejected (max 10). ✓ Previous <code>pending</code> becomes <code>superseded</code> atomically before insert. ✓ Unknown booking 404; not-claimed 403; wrong status 400.

FR-QUO-02 — EXPIRY SEMANTICS

FIELD	DETAIL
Shall	Per <code>src/server/quotations.ts:13</code> , <code>QUOTATION_TTL_MS=24h</code> . <code>isQuotationExpired(row)</code> checks <code>status==='pending' && expires_at < now</code> . <code>getActiveQuotation(bookingId)</code> lazily marks expired to <code>'expired'</code> on read; <code>findExpiredQuotations()</code> + <code>markQuotationExpired()</code> support cron sweep. Client shows countdown from <code>createdAt/expiresAt</code> ; <code>totalPrice</code> nulled when expired; artist sees "Send new quotation" when <code>accepted && expired</code> .
Acceptance	✓ Quotation older than 24h reads as <code>expired</code> and stays <code>expired</code>. ✓ Re-quote after expiry succeeds (supersede not needed when previous is <code>expired</code>).

8. FR-05 Payments (Hybrid Deposit/Balance)

FR-PAY-01 — PAYMENT TYPES & SCHEMA

FIELD	DETAIL
Schema	<code>payments(id TEXT PK, booking_id FK ON DELETE CASCADE, type TEXT CHECK(deposit,balance) DEFAULT deposit, amount INTEGER, currency MYR, provider dev billplz, status required paid failed refunded, provider_ref, provider_url, created_at, updated_at)</code> with <code>UNIQUE(booking_id,type)</code> <code>uq_payments_booking_type</code> and <code>CASCADE</code> semantics <code>src/server/db.ts:318</code> . <code>PaymentRecord</code> type <code>src/server/payments.ts:32</code> .
Invariants	One deposit and at most one balance per booking, never two of same type (DB unique). <code>amount</code> stored in sen. <code>provider_url</code> is Billplz hosted page URL (nullable).
Acceptance	✓ Duplicate <code>INSERT (booking_id,type)</code> violates unique index. ✓ Deleting booking cascades delete payments.

FR-PAY-02 — CREATE DEPOSIT PAYMENT (PAY-FEE)

FIELD	DETAIL
Endpoint	<code>POST /api/bookings/[id]/pay-fee {turnstileToken?}</code> — customer who owns booking; booking <code>status==='accepted'</code> ; quotation <code>pending</code> or already expired? (re-quote path).
Processing	Verify ownership + <code>email_verified</code> (gate at payment, see <code>src/app/api/bookings/route.ts:161</code> comment — unverified allowed at request but blocked or warned at fee); <code>getBookingFeeSen()</code> default 5000 sen; check no existing <code>payments(type=deposit,status IN (required,paid))</code> already; call <code>createBookingPayment(bookingId,'deposit',amountSen)</code> which branches on <code>activePaymentProvider()</code> (<code>BILLPLZ_API_KEY && COLLECTION_ID ? billplz:dev</code>). <code>billplzPayment</code> : fetch owner name/email, POST <code>{BILLPLZ_API}/bills</code> with Basic auth, fields <code>collection_id,name,email,mobile,amount,description,callback_url,redirect_url,reference_1=bookingId</code>; parse <code>{id,url}</code> ; <code>insertPayment({... status='required',provider='billplz',provider_ref=id,provider_url=url})</code> ; return <code>{id,amount,provider,reference,url}</code> . <code>devPayment</code> : synthesize <code>provider_ref=dev_...</code> , insert <code>status='required'</code> . For dev/e2e, immediately <code>markBillPaid(billId)</code> + <code>handlePaymentPaid</code> to auto-confirm. Rate-limit + optional Turnstile verify.
Responses	<code>200 {payment:{amount,type,status,provider,reference,url}}</code> plus <code>booking:{status}</code> after dev auto-settlement (<code>confirmed</code>).
Acceptance	✓ Amount equals <code>getBookingFeeSen()</code> (not client-provided). ✓ Billplz failure (<code>!res.ok !body.id</code>) throws "Failed to create payment" and logs (<code>pino.error</code>). ✓ Dev path is synchronous await (row durable before webhook reads). ✓ Second <code>pay-fee</code> on same booking returns 409 ("deposit already required/paid").

FR-PAY-03 — CREATE BALANCE PAYMENT (PAY-BALANCE)

FIELD	DETAIL
Endpoint	<code>POST /api/bookings/[id]/pay-balance</code> — owner, booking <code>status==='confirmed'</code> , active quotation <code>total</code> exists and not <code>expired</code> .
Processing	Mirror <code>pay-fee</code> but <code>type='balance'</code> and <code>amount = max(0, quotation.total - bookingFeeSen)</code> . If <code>balanceAmount===0</code> → return informative error or treat as no-op paid. Create <code>payments(type=balance)</code> via <code>createBookingPayment</code> . Dev auto-settles similarly and triggers <code>handlePaymentPaid</code> → <code>createPayoutForBooking</code> .
Acceptance	✓ Amount derived (<code>quotation.total - deposit</code>) never client-supplied. ✓ No active quotation or expired → 400. ✓ 0 balance edge case handled (commission + fee covers total).

FR-PAY-04 — BILLPLZ WEBHOOK

FIELD	DETAIL
Endpoint	<code>POST /api/payments/webhook</code> — public, but HMAC-protected
Processing	Read raw body string (NOT parsed JSON) for <code>verifyBillplzSignature(rawBody, X-Billplz-Signature, API_KEY)</code> <code>src/server/payments.ts:381</code>: check header present, hex64 regex, <code>HMAC-SHA256(apiKey, rawBody).hex</code>, <code>timingSafeEqual</code> buffers; if false → <code>401</code> and no state change. On pass: parse JSON fields, <code>billId = body.id</code>, <code>getPaymentForBill(billId)</code> else <code>404</code>; <code>markBillPaid(billId)</code> (<code>UPDATE payments SET status='paid' where provider_ref=billId</code>); <code>handlePaymentPaid(payment)</code>: if <code>deposit</code> → <code>confirmOnFeePaid(booking.status)</code> → <code>UPDATE bookings SET status='confirmed'</code> if ok; if <code>balance</code> → <code>UPDATE quotations SET status='paid'</code> + <code>createPayoutForBooking(...)</code>. Idempotent (re-delivery no-op). Log via <code>pino</code>.
Acceptance	✓ Missing/bad signature → <code>401</code>; no DB write. ✓ Forged <code>paid:true</code> without valid HMAC is ignored. ✓ Replay of same bill id is idempotent (second <code>markBillPaid</code> still true but status stays paid). ✓ Signature over raw body (not pretty-printed JSON) — test with exact byte string.

FR-PAY-05 — REFUND (BALANCE ONLY)

FIELD	DETAIL
Endpoint	<code>POST /api/bookings/[id]/refund</code> — customer who owns booking, or admin; booking <code>status==='cancelled'</code>
Processing	<code>getPaymentForBooking(bookingId, 'balance')</code>; guard <code>type==='balance'</code> else "Only balance..."; guard <code>status==='paid'</code> else "Only paid balances..."; <code>amountSen=payment.amount</code>; if <code>provider==='billplz' && BILLPLZ_API_KEY && provider_ref</code> → <code>POST \$ {BILLPLZ_API}/bills/{provider_ref}/refund {amount, reason:"Booking cancelled"}</code> — on failure throw; then <code>UPDATE payments SET status='refunded'</code>. <code>provider='dev'</code> just marks refunded. Return updated payment.
Acceptance	✓ Deposit row never refunded (guard). ✓ Unpaid balance → error. ✓ Billplz refund failure rolls back (not marked refunded).

9. FR-06 Payouts & Commission

FR-PAYO-01 — COMMISSION COMPUTATION

FIELD	DETAIL
Pure Function	<pre>computeCommission(totalSen, rateBps, waiverSen) → {totalSen, waived, commissionSen, artistNetSen} src/ server/settings.ts:100 — total=max(0,round(totalSen)), waived=total<waiver, commission=waived?0:round(total*rate/10000), artistNetSen=total-commission.</pre>
Settings	<pre>getCommissionRateBps() clamped 0–5000; getCommissionWaiverSen() ≥0; DEFAULT_COMMISSION_RATE_BPS=1000, DEFAULT_COMMISSION_WAIVER_SEN=10000 src/server/ settings.ts:21 ; 30s cache src/server/settings.ts:25.</pre>
Acceptance	✓ <code>computeCommission(50_00,1000,10000)</code> → waived true, commission 0. ✓ <code>computeCommission(100000,1000,10000)</code> → 10000 commission. ✓ Negative inputs clamped to 0. ✓ Rate 6000 clamped to 5000.

FR-PAYO-02 — PAYOUT CREATION

FIELD	DETAIL
Function	<pre>createPayoutForBooking(bookingId, {artistId,eventDate,quoteTotalSen}) src/server/payouts.ts:56 — idempotent: if SELECT * FROM payouts WHERE booking_id=? exists return it; else fetch rateBps,waiverSen,depositSen in parallel; breakdown=computeCommission(...); artistReceives=max(0, breakdown.artistNetSen - depositSen); settleableAt = eventDate? eventDateT00:00+24h ISO : null; artistUserId = SELECT user_id FROM artist_profiles WHERE artist_id=? else null; INSERT payouts(id,artist_user_id,booking_id,gross,commission,net,status=pending,settleable_at,settled_at null, notes=waived?"Commission waived":null, created_at); pino.info.</pre>
Trigger	Called only from <code>handlePaymentPaid</code> when <code>type==='balance'</code> is paid (never from deposit).
Acceptance	✓ Second call with same bookingId returns existing row (no duplicate). ✓ <code>net = gross - commission - deposit</code> (e.g. 100k-10k-5k=85k). ✓ Unclaimed artist → <code>artist_user_id=null</code> still creates payout. ✓ <code>settleable_at</code> exactly 24h after event date midnight UTC.

FR-PAYO-03 — PAYOUT ADMINISTRATION

FIELD	DETAIL
Endpoints	<code>GET /api/admin/payouts?status=</code> (optional filter), <code>PATCH /api/admin/payouts {payoutId, status:'settled' 'failed', notes?}</code> — admin only
Processing	<code>listPayouts(status?)</code> joins <code>payouts</code> JOIN <code>bookings</code> for display; <code>updatePayoutStatus(id, status, notes)</code> does <code>UPDATE payouts SET status=?, settled_at=(settled? now:null), notes=COALESCE(?,notes) WHERE id=?</code> ; returns row or 404. Caller (<code>/api/admin/payouts</code> route) calls <code>logAdminAction</code> after.
Acceptance	✓ Non-admin → 403. ✓ Missing payout → 404. ✓ Settled row gets <code>settled_at</code> ISO; <code>failed</code> gets <code>settled_at null</code>.

10. FR-07 Reviews & Ratings

FR-REV-01 — ADD REVIEW

FIELD	DETAIL
Endpoint	<code>POST /api/artists/[slug]/reviews {rating 1-5, text, event?}</code> — auth required? (customer)
Types	<code>Review</code> <code>src/lib/types.ts:15</code> <code>{id,author,rating,date,text,event}</code> ; <code>NewReviewInput</code> <code>src/server/catalog.ts:580</code> adds <code>entityType,entityId,bookingId,userId</code>
Processing	<code>addEntityReview(input)</code> <code>src/server/catalog.ts:633</code> : if <code>bookingId</code> provided, guard <code>SELECT id FROM reviews WHERE booking_id=?</code> → throw <code>ALREADY_REVIEWED</code> if exists; <code>INSERT reviews(id,entity_type,entity_id,booking_id,user_id,author_name,rating,event,text,created_at)</code> with <code>rating=round(rating)</code> ; then <code>blendAggregate(entityType,entityId,rating)</code> ; fetch back and map via <code>rowToPublicReview</code> .
Acceptance	✓ Duplicate <code>bookingId</code> review → error <code>ALREADY_REVIEWED</code> (409). ✓ Rating rounded to int server-side. ✓ Review appears via <code>listEntityReviews(entityType,entityId)</code> ordered <code>created_at DESC</code>.

FR-REV-02 — GATE REVIEW ELIGIBILITY

FIELD	DETAIL
Function	<pre>findReviewableBooking(userId, entityType, entityId)</pre> <code>src/server/catalog.ts:678</code>: currently only <code>entityType==='artist'</code> supported (studio returns null). Query: <pre>SELECT b.id FROM bookings b LEFT JOIN reviews r ON r.booking_id=b.id WHERE b.user_id=? AND b.artist_id=? AND b.status='completed' AND r.id IS NULL ORDER BY b.date DESC LIMIT 1.</pre>
Acceptance	✓ Completed booking without review → row returned. ✓ Pending/accepted/confirmed booking → null. ✓ Already-reviewed booking → null (JOIN finds r.id). ✓ Studio entity → null (explicit).

FR-REV-03 — ATOMIC RATING BLEND

FIELD	DETAIL
Function	<pre>blendAggregate(entityType, entityId, rating)</pre> <code>src/server/catalog.ts:615</code>: <code>UPDATE artists studios SET review_count=review_count+1, rating=ROUND(((rating*review_count)+?)/(review_count+1),2), updated_at=now WHERE id=?</code> — single statement.
Acceptance	✓ Concurrent reviews don't lose updates (no read-modify-write). ✓ Rating rounded 2dp via <code>ROUND(...,2)</code> (both PG + SQLite). ✓ New <code>review_count</code> is old+1, new rating correctly weighted.

11. FR-08 Messaging & Chat

FR-MSG-01 — PERSIST MESSAGES

FIELD	DETAIL
Table	<code>messages(id TEXT PK, booking_id FK CASCADE, sender_id FK CASCADE, body TEXT, created_at TEXT)</code>
Endpoints	<code>GET /api/bookings/[id]/messages</code> (participants only) and <code>POST /api/bookings/[id]/messages {body}</code>
Processing	Authz: <code>user_id == booking.user_id</code> OR <code>artist_id ∈ claimedArtists/user</code> ; if not participant → 403. POST sanitizes <code>body</code> trim, non-empty, ≤2000?; <code>INSERT messages(randomUUID, booking_id, sender_id=user.id, body, created_at=now)</code> ; publish to <code>chat-bus</code> for SSE. GET returns ordered <code>created_at ASC</code> .
Acceptance	✓ Third user who is neither owner nor claimed artist is blocked (403). ✓ Empty body rejected 400. ✓ Message appears immediately on GET.

FR-MSG-02 — LIVE STREAMING (SSE)

FIELD	DETAIL
Endpoint	<code>GET /api/bookings/[id]/messages/stream</code> — <code>text/event-stream</code>
Processing	Authz same as FR-MSG-01; handler replays history <code>(SELECT * FROM messages WHERE booking_id=? ORDER BY created_at ASC)</code> as <code>data: JSON{ id, senderName, body, createdAt }</code> events, then subscribes to <code>src/server/chat-bus.ts</code> in-memory pub/sub for new messages on that booking channel; keepalive every 30s.
Acceptance	✓ New browser joining stream receives history then live frames. ✓ Disconnect + reconnect replays full history (no lost messages). ✓ Connection is authenticated (cookie same as HTTP routes).

12. FR-09 Profile Claim & Self-Service

FR-CLAIM-01 — CLAIM A CATALOG PROFILE

FIELD	DETAIL
Endpoint	<code>POST /api/artist-profiles {artistId}</code> — role must be <code>artist studio</code> ; one claim per <code>user_id</code> .
Table	<code>artist_profiles(user_id PK FK users, artist_id UNIQUE, claimed_at)</code> + parallel <code>studio_profiles</code>
Processing	Check <code>role ∈ (artist,studio)</code> else 403; validate <code>artistId</code> exists in <code>artists</code> (or studios for studio role); check not already claimed globally (<code>uq_artist_profiles_artist_id</code>); then <code>INSERT</code> and return <code>profile</code> . Emits claim event for admin.
Acceptance	✓ Customer role cannot claim (403). ✓ Already claimed <code>artist_id</code> returns 409. ✓ Second claim by same <code>user_id</code> violates PK → 409 (one profile per account).

FR-CLAIM-02 — FETCH CLAIMED PROFILE

FIELD	DETAIL
Endpoint	<code>GET /api/artist-profiles</code> — authed
Processing	<code>SELECT artist_id FROM artist_profiles WHERE user_id=?</code> (and studio equivalent); if studio role prefer <code>studio_profiles</code> fallback to <code>artist_profiles</code> (shared table history). Return <code>profile:null</code> when none claimed; else <code>{artistId, artistName}</code> .
Acceptance	✓ Customer with no claim → <code>{profile:null}</code> (not 404). ✓ Claimed artist sees own name.

FR-CLAIM-03 — SELF-SERVICE PROFILE EDITS

FIELD	DETAIL
Endpoint	<code>PATCH /api/artist-profiles {updates}</code> — claimed artist only
Processing	Load claimed <code>artistId</code> for caller; apply via <code>updateArtist(artistId, updates)</code> with whitelisted <code>ARTIST_UPDATE_FIELDS</code> subset for self-service (exclude <code>verified</code> / <code>referralEarnings</code>). Validate fields server-side (e.g. <code>priceFrom</code> 0–10M). Update <code>updated_at</code> .
Acceptance	✓ Unclaimed artist → 403. ✓ Field <code>verified</code> silently ignored (cannot self-verify). ✓ Admin edits via <code>/api/admin/artists</code> can set all fields including <code>verified</code> .

13. FR-10 Admin Panel & Audit

FR-ADM-01 — GUARD

FIELD	DETAIL
Server guard	<code>requireAdmin(request)</code> <code>src/server/admin-auth.ts</code> — extracts <code>leish_session</code> , <code>verifySessionToken</code> , loads <code>users</code> row, checks <code>role==='admin'</code> ; returns <code>401</code> <code>jsonError("Not authenticated")</code> or <code>403</code> <code>jsonError("Forbidden")</code> .
Layout guard	<code>src/app/admin/layout.tsx</code> does same server-side before render and redirects non-admins; <code>AdminShell</code> provides sidebar + drawer.
Seed	<code>scripts/seed-admin.ts</code> idempotent: <code>ADMIN_EMAIL/ADMIN_PASSWORD</code> creates user with <code>role='admin'</code> or upgrades existing <code>users.role</code> to admin.
Acceptance	✓ Non-admin direct API call blocked (not just UI hidden). ✓ Audit of failed admin attempt not required but recommendation logged.

FR-ADM-02 — ADMIN CRUD (OVERVIEW)

RESOURCE	ENDPOINTS (UNDER <code>SRC/APP/API/ADMIN/*</code>)	ALLOWED OPERATIONS	AUDIT
Dashboard	<code>GET /api/admin</code>	Read aggregates: users by role, bookings by status, payments total/paid/required/revenue, <code>artistProfiles</code> count, recentBookings (10), recentAudit (10)	No
Users	<code>GET/POST /api/admin/users</code> , <code>PATCH/DELETE /api/admin/users/[id]</code>	Full CRUD (never expose password hash)	Yes — every mutation
Artists	<code>GET/POST /api/admin/artists</code> , <code>PATCH /api/admin/artists/[id]</code>	Create via <code>createArtist</code> , update via <code>updateArtist</code> (all whitelisted fields + <code>verified</code>)	Yes
Studios	Mirrors Artists	Same via <code>createStudio</code> / <code>updateStudio</code>	Yes
Bookings	<code>GET /api/admin/bookings</code> , <code>PATCH /api/admin/bookings/[id]</code>	Admin status override + notes	Yes
Payments	<code>GET /api/admin/payments</code>	List + filter by booking/user	No
Payouts	<code>GET/PATCH /api/admin/payouts</code>	See FR-PAYO-03	Yes
Quotations	<code>GET /api/admin/quotations</code>	List + expire sweep trigger	No
Messages	<code>GET /api/admin/messages</code>	Read all booking threads	No
Emails	<code>GET /api/admin/emails</code>	View <code>email_outbox</code> + retries	No
Audit	<code>GET /api/admin/audit</code>	List <code>admin_audit_log</code> ordered desc	No
Settings	<code>GET/PATCH /api/admin/settings</code>	<code>platform_settings</code> CRUD	Yes
Analytics	<code>GET /api/admin/analytics</code>	Agnost rollups if configured	No

Conventions — Admin pages (`src/app/admin/*`) are "use client" and fetch from `/api/admin/*` via `.then()` chains (never synchronous `setState` in effect body per `react-hooks/set-state-in-effect` rule).

FR-ADM-03 — AUDIT LOG

FIELD	DETAIL
Table	<code>admin_audit_log(id TEXT PK, admin_user_id FK users ON DELETE SET NULL, action TEXT, target_table TEXT, target_id TEXT, details TEXT JSON default '{}', created_at TEXT)</code> with indexes <code>idx_audit_log_admin</code> , <code>idx_audit_log_created</code> .
Helper	<code>logAdminAction({adminUserId, action, targetTable, targetId, details})</code> — called after every successful admin mutation; failure to log shall not roll back the mutation but shall emit <code>pino.error</code> .
FK Note	<code>admin_user_id</code> has real FK to <code>users(id)</code> — integration tests must seed a user row before writing audit entries; otherwise foreign key violation.
Acceptance	✓ Admin <code>PATCH /api/admin/artists/[id]</code> creates audit row with <code>action='update'</code> + <code>target_table='artists'</code>. ✓ Deleting the admin user sets <code>admin_user_id</code> to null (<code>SET NULL</code>) not cascade delete.

FR-ADM-04 — SETTINGS ADMINISTRATION

FIELD	DETAIL
Keys	<code>booking_fee_sen</code> , <code>commission_rate_bps</code> , <code>commission_waiver_sen</code> (+ extensible others like <code>platform_name</code> , <code>support_email</code> may be added).
Endpoint	<code>PATCH /api/admin/settings {key,value}</code> — validate <code>value</code> is numeric within allowed ranges; write <code>platform_settings(value, updated_by=adminUserId, updated_at=now)</code> with <code>INSERT ... ON CONFLICT(key) DO UPDATE</code> . Invalidate cache <code>clearSettingsCache()</code> effect via next read (30s TTL expiry).
Acceptance	✓ Invalid key/value returns 400 validation error. ✓ Changes take effect within 30s (cache TTL) without redeploy. ✓ Setting read failure always falls back to default (never breaks payments).

14. FR-11 Email & Notifications

FR-MAIL-01 — PROVIDER ABSTRACTION

FIELD	DETAIL
Module	<code>src/server/email.ts</code> <code>sendEmail({to, subject, text, html})</code> — selects <code>EMAIL_PROVIDER</code> env (default <code>dev</code>): <code>dev</code>: <code>INSERT email_outbox(id,to_email,subject,text,html,created_at)</code> and return dev link. <code>resend</code>: POST Resend API with <code>RESEND_API_KEY</code>, <code>EMAIL_FROM</code>. <code>postmark</code>: POST Postmark with <code>POSTMARK_SERVER_TOKEN</code>. <code>brevo</code>: POST Brevo with <code>BREVO_API_KEY</code>. Missing creds for selected provider → fallback to dev outbox + <code>pino.warn</code> (never silent).
Acceptance	✓ Unknown <code>EMAIL_PROVIDER</code> falls back to dev. ✓ <code>EMAIL_FROM</code> default is <code>Leish! <no-reply@leish.my></code>. ✓ Dev outbox viewable at <code>/dev/emails</code> (only non-production).

FR-MAIL-02 — BOOKING EMAILS

FIELD	DETAIL
Module	<code>src/server/booking-emails.ts</code> composes all transactional copy: <code>notifyBookingCreated</code>, <code>notifyQuotationSent</code>, <code>notifyInvoiceSent</code>, <code>notifyQuotationExpiry</code>, <code>notifyBalanceReminder</code>, <code>notifyStatusChanged</code>.
Acceptance	✓ New booking triggers <code>booking_created</code> email to relevant party. ✓ Quotation triggers <code>quotation_sent</code>. ✓ Balance reminder (<code>POST /api/bookings/[id]/remind</code> from confirmed bookings) respects <code>email_preferences.balance_reminder</code>.

FR-MAIL-03 — PREFERENCES & RETRIES

FIELD	DETAIL
Preferences	<code>email_preferences(user_id PK FK, booking_created, quotation_sent, invoice_sent, quotation_expiry, balance_reminder, status_changed INTEGER 0/1, updated_at)</code> default <code>1</code> for each; <code>PATCH /api/email/preferences</code> upserts.
Retries	<code>email_retries(id,to_email,subject,text,html,attempts,max_attempts=3,next_retry,last_error,created_at)</code> — on send failure, row inserted with <code>next_retry = now + backoff</code>; cron retries via <code>src/app/api/cron/*</code> guarded by <code>CRON_SECRET</code>.
Acceptance	✓ User can toggle each preference off and stop receiving that class. ✓ Retry attempts capped at 3, then dropped with logged error.

15. FR-12 Dashboard & UX

FR-DASH-01 — CLIENT DASHBOARD

FIELD	DETAIL
Route	<code>/dashboard</code> — client-only (<code>"use client"</code>), requires auth else CTA "Please sign in".
Data	Fetches <code>GET /api/bookings</code> + <code>GET /api/catalog/artists</code> + <code>GET /api/artist-profiles</code> in effects with <code>.then()</code> chains; shows banners (verify email), stats (customer stats hidden; MUA stats: requests count, quotations open, confirmed, earnings paid/pending).
Views	For each booking card: avatar, service · price · eventType, status badges (<code>requested</code> amber, <code>accepted</code> sky, <code>confirmed</code> emerald, <code>completed</code> stone, <code>cancelled</code> red), payment badge, fee <code>fee {status}</code> , quotation block (lines + totals + 24h expiry), chat thread, actions: cancel / pay-fee / pay-balance / refund / invoice / send-reminder (role-aware). Invoices link to HTML + PDF.
Acceptance	✓ Unverified customer sees amber banner with resend (idle/sending/sent). ✓ <code>GET /api/bookings</code> failure shows empty state with correct CTA per role. ✓ Stats compute correctly from <code>bookings + payments + quotations</code>.

FR-DASH-02 — ARTIST CLAIM UI

FIELD	DETAIL
Block	Visible when <code>role ∈ (artist,studio)</code> ; reads <code>GET /api/artist-profiles</code> ; if null shows <code><select></code> populated from <code>/api/catalog/artists</code> and <code>POST /api/artist-profiles</code> on submit; after claim shows "You are managing "...". + refresh.
Acceptance	✓ Claimed profile id shown correctly. ✓ Select disabled until catalog loaded. ✓ Claim error surfaced as <code>role="alert"</code>.

16. FR-13 Invoicing & Documents

FR-INV-01 — PRINTABLE INVOICE (HTML) & PDF

FIELD	DETAIL
Endpoints	<code>GET /api/bookings/[id]/invoice</code> (HTML) and <code>GET /api/bookings/[id]/invoice.pdf</code> (PDF via <code>pdf-lib</code>)
Authz	Owner or claimed artist/studio only; same guard as messaging.
Content	Header (Leish! branding), billing to client, artist + service + date/time + venue, quotation line items (base/travel/early-call/accommodation + extras), <code>booking_fee_sen</code> line, total, payment status per type, commission note when relevant. PDF rendered server-side (no browser print-sim needed).
Acceptance	✓ Unauthorized returns 403. ✓ Invoice reflects server-computed <code>quotation.total</code>, not cached client price. ✓ PDF downloads with correct <code>Content-Disposition</code>.

17. FR-14 Compliance & Data Rights

FR-COMP-01 — DATA EXPORT

FIELD	DETAIL
Endpoint	<code>GET /api/me/export</code> — authed
Processing	Aggregates <code>users</code> public fields, <code>bookings</code> , <code>messages</code> , <code>payments</code> , <code>reviews</code> , <code>artist_profiles</code> , <code>email_preferences</code> ; returns <code>application/json</code> blob with <code>Content-Disposition: attachment</code> named <code>leish-data-<id>.json</code> ; used by <code>/dashboard</code> Export button.
Acceptance	✓ Result is valid JSON containing all owned records. ✓ No other users' data leaked.

FR-COMP-02 — ACCOUNT DELETION

FIELD	DETAIL
Endpoint	<code>DELETE /api/me?confirm=1</code> — authed, <code>confirm</code> required to avoid accidental hit
Processing	Confirm <code>confirm=1</code> ; cascade delete via FK <code>ON DELETE CASCADE</code> for bookings/payments/quotations/messages/sessions; <code>ON DELETE SET NULL</code> for audit entries; remove user row; clear cookie; return 200. Dashboard confirm dialog requires explicit user confirmation.
Acceptance	✓ Without <code>confirm=1</code> returns 400. ✓ After delete, login with same email fails (user gone). ✓ Audit log retains entry with <code>admin_user_id=null</code> (not lost).

18. FR-15 Platform Settings & Ops

FR-OPS-01 — SETTINGS RUNTIME

FIELD	DETAIL
Table	<code>platform_settings(key TEXT PK, value TEXT, updated_by FK, updated_at TEXT)</code>
Accessors	<code>getSetting/getNumberSetting/getBookingFeeSen/getCommissionRateBps/getCommissionWaiverSen/clearSettingsCache</code> <code>src/server/settings.ts</code> . Cache 30s process-local.
Acceptance	✓ Cache clears after <code>clearSettingsCache()</code> (used in tests). ✓ Read failure never throws — falls back to default and warns.

FR-OPS-02 — HEALTH & CRON

FIELD	DETAIL
Health	<code>GET /api/health</code> → <code>{ok:true, db: "postgres" "sqlite", env: "production" ...}</code> ; Docker HEALTHCHECK <code>http://localhost:3000/api/health</code> .
Cron	<code>GET/POST /api/cron/*</code> (e.g. <code>expire-quotations</code> , <code>retry-emails</code>) guarded by <code>CRON_SECRET</code> bearer; Vercel Cron or external scheduler calls them.
Acceptance	✓ Health returns 200 even when <code>DATABASE_URL</code> unset (sqlite mode). ✓ Cron without valid secret returns 401.

FR-OPS-03 — ANALYTICS (AGNOST AI)

FIELD	DETAIL
Server	<code>src/instrumentation.ts</code> initializes <code>agnostai</code> SDK with <code>AGNOST_ORG_ID</code> ; every key route wraps with <code>agnost.begin({userId, agentName, input})</code> → <code>interaction.end(output, success)</code> : register, login, booking-create, booking-status update, quotation, payment webhook, profile edit.
Client	<code>src/lib/agnost-client.ts</code> <code>trackEvent</code> , <code>trackArtistView</code> , <code>trackSearch</code> , <code>trackBookingForm</code> called from <code>ArtistsBrowser</code> and booking/calendar components when <code>NEXT_PUBLIC_AGNOST_ORG_ID</code> set.
Dashboard	<code>https://app.agnost.ai/projects/3e27e121-654d-4746-ba55-7996f21bb351</code>
Acceptance	✓ When <code>AGNOST_ORG_ID</code> missing, instrumentation is no-op (no crash). ✓ Tracking never leaks PII beyond userId/bookingId (hashed where needed).

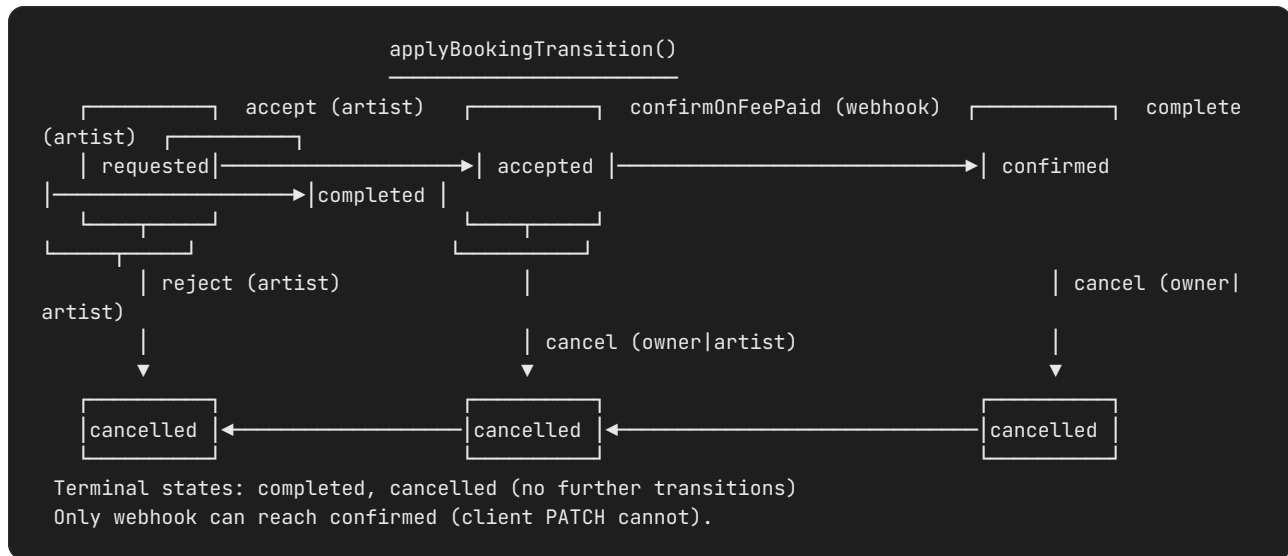
19. Summary Table & MoSCoW

MODULE	FR IDS	COUNT	MUST	SHOULD	COULD
Auth	FR-AUTH-01..07	7	7	—	—
Catalog	FR-CAT-01..04	4	4	—	—
Booking	FR-BOOK-01..04	4	4	—	—
Quotation	FR-QUO-01..02	2	2	—	—
Payments	FR-PAY-01..05	5	5	—	—
Payouts	FR-PAYO-01..03	3	3	—	—
Reviews	FR-REV-01..03	3	3	—	—
Messaging	FR-MSG-01..02	2	2	—	—
Profile Claim	FR-CLAIM-01..03	3	3	—	—
Admin	FR-ADM-01..04	4	4	—	—
Email	FR-MAIL-01..03	3	2	1	—
Dashboard	FR-DASH-01..02	2	1	1	—
Invoicing	FR-INV-01	1	—	1	—
Compliance	FR-COMP-01..02	2	2	—	—
Ops	FR-OPS-01..03	3	2	1	—
Total		48	42	4	—

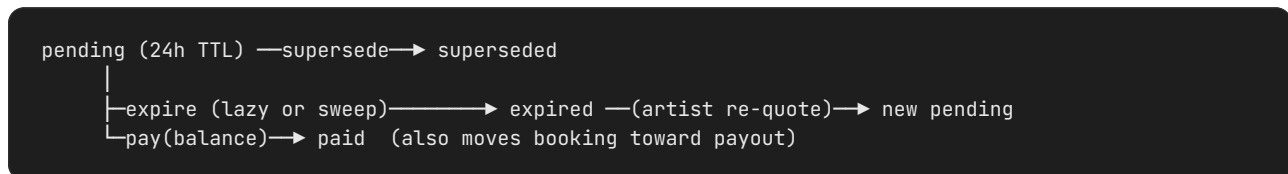
All **Must** items constitute the launch gate. **Should** items are highly desired for launch but may slip to v2.1 without blocking.

20. Appendix — State Machines & Flows

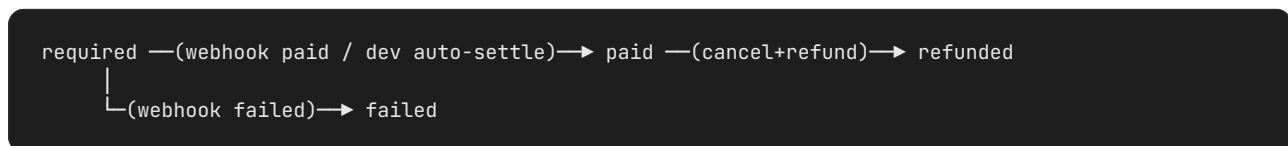
20.1 BOOKING STATUS STATE MACHINE (FROM `SRC/SERVER/BOOKINGS.TS:1`)



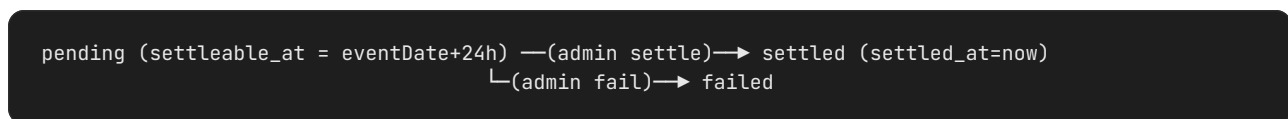
20.2 QUOTATION STATES



20.3 PAYMENT STATUS



20.4 PAYOUT STATUS



Next: `04-NFR-Non-Functional-Requirements.md` for measurable quality attributes, and `05-Use-Cases-and-User-Stories.md` for scenario-level acceptance.

04-NFR-Non-Functional-Requirements

NFR — Non-Functional Requirements (Supplementary Specification)

Leish! v2 — Beauty Booking Marketplace

1. Purpose
2. Performance Efficiency
3. Reliability
4. Security
5. Usability
6. Maintainability & Supportability
7. Portability & Compatibility
8. Compliance (Cross-Cut)
9. Operations & Observability
10. Constraints & Assumptions (NFR View)
11. Verification Matrix (Sampling)
12. Trade-offs & Rationale

NFR — Non-Functional Requirements (Supplementary Specification)

Leish! v2 — Beauty Booking Marketplace

FIELD	VALUE
Document ID	LEISH-NFR-v2.0
Version	2.0.0
Date	2026-08-29
Status	Baseline
Predecessor	SRS (02) §7
Standard	ISO/IEC 25010 (SQuaRE) quality model + OWASP ASVS

1. Purpose

Quantifies how well the system shall perform, not what it does. Every NFR here is **measurable** and has a stated **fit-criterion** (pass/fail) and **verification method**.

Categories follow ISO 25010: Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability — plus Compliance & Operations.

2. Performance Efficiency

NFR-P-01 — RESPONSE TIME (LATENCY)

REQUIREMENT	TARGET (FIT-CRITERION)	PRIORITY	VERIFICATION
Public catalog <code>GET /api/artists</code> (cached 300s) p95	≤500 ms on Vercel + PG; ≤300 ms on cold-cache SQLite dev	Must	k6 / Vercel Analytics p95
<code>POST /api/bookings</code> (create, incl. slot check) p95	≤400 ms excluding async email send	Must	Load test 50 VUs
<code>GET /api/bookings</code> (scoped list, 20 rows) p95	≤350 ms	Must	Load test
<code>POST /api/payments/webhook</code> (verify+mark+settle) p95	≤300 ms	Must	Webhook replay bench
<code>PATCH /api/bookings/[id]</code> (state transition) p95	≤250 ms	Must	Unit bench
Admin dashboard aggregates <code>GET /api/admin</code> p95	≤800 ms (aggregated counts)	Should	Bench

Measurement: from Cloudflare/Vercel edge timing (`x-vercel-cache` , server-timing header if added) or `pino` request duration field.

NFR-P-02 — THROUGHPUT

REQUIREMENT	TARGET	VERIFICATION
Sustained bookings	10 rps (sustained) without error	Must
Webhook ingestion	20 rps burst for 60s (Billplz spike)	Must
Concurrent users (catalog)	500 concurrent (95% <500 ms)	Should

NFR-P-03 — CAPACITY & SCALABILITY

REQUIREMENT	TARGET	IMPLEMENTATION
Catalog growth	10k artists, 5k studios without schema change	Must
DB pool	PG_MAX 10 by default, configurable to 50 (Neon limits)	Must
Storage	Images via Supabase/Vercel Blob, not DB	Must
Horizontal	Stateless handlers (no session affinity); Redis optional for rate-limit	Must

NFR-P-04 — RESOURCE UTILIZATION

REQUIREMENT	TARGET	VERIFICATION
Vercel function memory	≤512 MB per invocation	Must
PG connections	P95 ≤ 80% of PG_MAX ; idle 10s timeout prevents pool exhaustion	Must
SQLite WAL	Enabled (PRAGMA journal_mode=WAL) for concurrent readers	Must

3. Reliability**NFR-R-01 — AVAILABILITY**

REQUIREMENT	TARGET	IMPLEMENTATION
Production uptime (monthly)	≥99.5% (≤3.6h downtime/mo) measured at GET / api/health externally	Must
Health endpoint	Responds <1s even when email/Billplz down (degraded mode)	Must

NFR-R-02 — FAULT TOLERANCE

REQUIREMENT	TARGET
DB pool error does not crash process	Must — <code>pool.on("error")</code> logs <code>pino.error</code> but keeps serving
Email provider failure degrades to <code>email_outbox</code>	Must — never throws from <code>sendEmail()</code> to caller route
Settings read failure falls back to defaults	Must — <code>getSetting()</code> catches and warns, never breaks payment math
Billplz API timeout → payment marked <code>required</code> (not <code>failed</code>) until webhook confirms	Must

NFR-R-03 — RECOVERABILITY

REQUIREMENT	TARGET
SQLite PG migration failure logs and throws (forces deploy rollback)	Must — <code>pgReady</code> catch logs and rethrows
Session JTI lookup failure (infra) is fail-open (accept valid signature) but logs warning — explicit <code>revoked=1</code> always wins	Must

NFR-R-04 — DATA INTEGRITY

REQUIREMENT	TARGET
Money never stored as float	Must — all <code>INTEGER</code> <code>sen</code>
Unique slot constraint enforced even under race	Must — partial index + friendly 409 catch
Rating blend without lost updates under concurrent reviews	Must — single UPDATE

4. Security

NFR-S-01 — AUTHENTICATION

ID	REQUIREMENT	FIT-CRITERION	VERIFICATION
S-01.01	Passwords bcrypt-hashed with random salt, timing-safe compare, optional pepper HMAC	No plaintext in logs/DB; <code>PEPPER_SECRET</code> rotates via <code>PEPPER_VERSION</code>	Code review + <code>src/server/auth.ts</code>
S-01.02	JWT HS256 with <code>SESSION_SECRET</code> 32-byte base64 in prod; 7-day TTL; JTI revocation table	<code>validateEnv()</code> throws if weak secret in prod; <code>revokeSession</code> clears JTI	Pentest + session tests
S-01.03	Cookie <code>leish_session</code> flags: <code>httpOnly</code> always, <code>Secure</code> in prod, <code>SameSite=lax</code> , <code>Path=/</code>	Header inspection	OWASP ZAP
S-01.04	No user enumeration on login/forgot-password	Constant-time response regardless of user existence	Auth tests

NFR-S-02 — AUTHORIZATION

ID	REQUIREMENT	FIT-CRITERION
S-02.01	<code>requireAdmin</code> guards every <code>/api/admin/*</code> ; <code>/admin/*</code> layout also server-guards before render	Must — 401/403 tests; <code>src/app/api/admin/list-routes.test.ts</code> audits coverage
S-02.02	Booking ownership: customer only sees <code>WHERE user_id=?</code> ; artist only sees claimed ids	Must — scope tests
S-02.03	Message SSE: participants only (<code>owner</code> OR <code>claimedArtist</code>)	Must — 403 for third party

NFR-S-03 — INPUT VALIDATION & INJECTION

ID	REQUIREMENT	FIT-CRITERION
S-03.01	All API inputs validated with <code>zod</code> via <code>registerSchema/loginSchema/bookingSchema/quotationSchema/artistsQuerySchema</code>	Must — zod rejects invalid before any DB touch
S-03.02	No SQL string concatenation for table names; <code>applyUpdate</code> field-map whitelist; values always via placeholders (<code>? / @name → \$n</code>)	Must — <code>ARTIST_UPDATE_FIELDS</code> whitelist
S-03.03	Billplz webhook verified via <code>verifyBillplzSignature</code> timing-safe before any state mutation	Must — reject 401 on bad sig; test suite covers tampered body

NFR-S-04 — SECURITY HEADERS & TRANSPORT

HEADER	VALUE	SOURCE
<code>Strict-Transport-Security</code>	<code>max-age=63072000; includeSubDomains; preload (prod)</code>	<code>next.config.ts</code>
<code>X-Frame-Options</code>	<code>DENY</code>	<code>next.config.ts</code>
<code>X-Content-Type-Options</code>	<code>nosniff</code>	<code>next.config.ts</code>
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	<code>next.config.ts</code>
<code>Content-Security-Policy</code>	Per-request nonce, no <code>unsafe-inline</code> for scripts; inline theme script uses nonce/hashed	<code>next.config.ts</code> + <code>layout.tsx</code>
<code>Content-Type</code> charset	<code>utf-8</code> always	Next default

NFR-S-05 — RATE LIMITING & BOT PROTECTION

REQUIREMENT	FIT-CRITERION
Sliding window per IP on <code>register/login/forgot/verify-email/bookings</code> webhook — Upstash Redis when <code>UPSTASH_REST_URL</code> set, else in-memory	Must — 429 + <code>Retry-After</code> header
Turnstile widget (<code>NEXT_PUBLIC_TURNSTILE_SITE_KEY</code>) required for <code>register/login/forgot/bookings pay-*</code> when configured	Should — skip silently in dev/e2e when unset
<code>ALLOWED_ORIGINS</code> extra origins checked for state-changing requests	Should

NFR-S-06 — VULNERABILITY MANAGEMENT

REQUIREMENT	FIT-CRITERION
<code>npm audit</code> clean criticals at build; Dependabot auto-PRs	Must
ESLint <code>next</code> rules clean (<code>npm run lint</code> 0 warnings)	Must (CI gate)
Secret scanning: <code>github-run-secret-scanning</code> on PR	Must

5. Usability

NFR-U-01 — ACCESSIBILITY

REQUIREMENT	TARGET	VERIFICATION
WCAG 2.1 AA: color contrast $\geq 4.5:1$, keyboard navigation, screen-reader labels on filters/search	Must	Lighthouse a11y ≥ 90 ; manual axe-core
Form validation messages via zod surfaced inline; no raw zod dump	Must	Component tests
Dark/light theme preserved in <code>localStorage</code> with inline anti-CLS script; <code>prefers-reduced-motion</code> respected	Should	<code>src/lib/theme.tsx</code> + <code>layout.tsx</code>

NFR-U-02 — UX & CONTENT

REQUIREMENT	TARGET
Booking request completable in <90s (discovery → submit)	Should — measured via analytics funnel
Empty states with role-appropriate CTA ("Browse artists" vs "Claim a profile")	Must
Currency displayed via <code>formatRM(sen)</code> <code>src/lib/utls.ts</code> as <code>RM</code> <code>X.XX</code> consistently	Must
404 pages return true 404 (not 200 shell)	Must — no root <code>loading.tsx</code>

NFR-U-03 — INTERNATIONALIZATION

REQUIREMENT	TARGET
Date/times <code>en-MY</code> locale (<code>toLocaleDateString("en-MY")</code>)	Must — <code>formatDate</code> helper
Currency MYR only; English only at launch	Accept — i18n deferred

6. Maintainability & Supportability

NFR-M-01 — CODE QUALITY

REQUIREMENT	FIT-CRITERION
TypeScript <code>tsc --noEmit</code> 0 errors (<code>npm run typecheck</code>)	Must (CI gate)
Prettier formatted (<code>npm run format:check</code>)	Must (CI gate)
Coverage $\geq 80\%$ (<code>npm run test:coverage</code> HTML)	Must (quality-gate.yml)
<code>vitest</code> suite green; <code>playwright</code> e2e green on <code>chromium</code>	Must (CI + <code>ci.yml</code>)

NFR-M-02 — MODULARITY & DOCUMENTATION

REQUIREMENT	FIT-CRITERION
Money logic isolated to 4 modules only: <code>settings.ts</code> , <code>payments.ts</code> , <code>payouts.ts</code> , <code>quotations.ts</code> — route handlers must not compute amounts inline	Should — enforced by code review
Catalog source of truth clearly documented: <code>src/lib/data.ts</code> is seed-only , never runtime; runtime is <code>src/server/catalog.ts</code>	Must — AGENTS.md §1
Absolute imports <code>@/</code> enforced	Should — <code>tsconfig paths</code>

NFR-M-03 — OPERABILITY

REQUIREMENT	FIT-CRITERION
<code>LOG_LEVEL</code> env governs <code>pino</code> verbosity; JSON lines in prod, pretty in dev	Must
<code>reportError(err, ctx)</code> always logs structurally and optionally forwards to Sentry/webhook	Must
PG & SQLite schemas kept in sync; <code>detectSchemaDrift()</code> warns on divergence	Should
<code>npm run db:migrate</code> idempotent and also backfills additive columns (e.g. <code>payments.type</code>)	Must

7. Portability & Compatibility

REQUIREMENT	TARGET	VERIFICATION
Node ≥ 22 required (for <code>node:sqlite</code>) — <code>engines.node</code> in <code>package.json</code>	Must	<code>npm install</code> warns below 22
Runs on Vercel serverless (primary) and Docker <code>node:20-alpine</code> <code>standalone</code>	Must	<code>Dockerfile</code> multi-stage build
Browsers: last 2 evergreen; mobile-first responsive (<640px works)	Must	Manual device matrix
Env portability: same <code>.env.example</code> works for local (SQLite) and prod (PG) — behavior switches via <code>DATABASE_URL</code> presence	Must	<code>isPostgres()</code> check

8. Compliance (Cross-Cut)

AREA	NFR	FIT-CRITERION
PDPA	Consent recorded (<code>consent</code> + <code>consent_timestamp</code>), exportable, deletable	Must — see <code>docs/PDPA_RETENTION_GUIDELINES.md</code>
Financial	Settlements auditable for ≥7 years (payments/ payouts/quotations) even after user deletion (anonymized log)	Should
Privacy	No PII in logs (user names/emails redacted; IDs only)	Must
Accessibility	PDPA/Human rights — data never exposed to wrong user (scope queries)	Must

9. Operations & Observability

NFR-O-01 — LOGGING

REQUIREMENT	FIT-CRITERION
<code>logger</code> (<code>pino</code> 10.3) JSON in prod, pretty via <code>pino-pretty</code> in dev	Must
<code>LOG_WEBHOOK_URL</code> forwards JSON batch: 20 lines / 50ms debounce	Should
Every request logs route + userId (hashed if sensitive) + duration on error	Should

NFR-O-02 — MONITORING & ALERTING

REQUIREMENT	FIT-CRITERION
Sentry envelope POST when <code>SENTRY_DSN</code> set, else <code>ERROR_WEBHOOK_URL</code> POST fallback	Should — <code>src/server/errors.ts</code>
Agnost events for business funnel (register/login/booking/quotation/payment)	Should — dashboard <code>3e27e121-654d-4746-ba55-7996f21bb351</code>
<code>GET /api/health</code> polled by Docker + UptimeRobot (1m interval)	Must

NFR-O-03 — DEPLOYMENT

REQUIREMENT	FIT-CRITERION
<code>npm run build</code> succeeds with no errors (sets <code>NODE_ENV=production</code>)	Must — CI includes <code>ci.yml</code> build step
<code>npm run env:check</code> (production mode) reports missing <code>SESSION_SECRET</code> , <code>DATABASE_URL</code> , <code>POSTGRES_URL</code> , Billplz keys	Must — pre-deploy checklist AGENTS.md §Production Deployment
<code>npm run db:migrate</code> + <code>npm run db:seed-catalog</code> run successfully against target PG before first traffic	Must

10. Constraints & Assumptions (NFR View)

Inherited from SRS §2.5/2.6. Additional NFR constraints:

- Client `NEXT_PUBLIC_*` vars are the only ones exposed to browser; no secret may be `NEXT_PUBLIC_*` — enforced by review.
- Images must come from `*.supabase.co` (locked `images.remotePatterns`) — no arbitrary external domain.
- No background job runtime in serverless — must use stateless cron routes (`src/app/api/cron/*`) with `CRON_SECRET` guard.

11. Verification Matrix (Sampling)

NFR ID	METHOD	TOOL	PASS CRITERION
P-01	Load test	k6, Vercel analytics	p95 ≤ target
P-04	Config audit	Code review	<code>PG_MAX</code> , <code>idleTimeoutMillis</code> , WAL pragma present
S-03.03	Unit / contract	Vitest <code>verifyBillplzSignature</code> suite; ZAP	0 bypass cases
S-04	Header scan	<code>curl -I</code> + CSP evaluator	All headers present, no <code>unsafe-inline</code>
U-01	Accessibility	Lighthouse CI + axe	≥90 a11y, 0 critical
M-01	CI gate	GitHub Actions <code>quality-gate.yml</code>	All jobs green
O-02	Smoke	<code>curl /api/health</code> + Sentry dashboard	200 + no unreported error

12. Trade-offs & Rationale

DECISION	TRADE-OFF	RATIONALE
PG-first + SQLite fallback	Drift risk vs zero local setup friction	Chosen: drift detection + shared facade keeps both backends cheap; local DX wins
HS256 symmetric vs RS256 asymmetric sessions	Must share secret vs PKI rotation complexity	HS256 sufficient single-service platform; rotation via <code>SESSION_SECRET</code> redeploy
In-memory fallback for rate-limit when Upstash absent	Multi-instance inaccuracy vs deploy simplicity	Acceptable for preview/dev; prod sets Upstash
pino batch webhook vs structured OTEL	Simpler vs richer traces	Chose webhook forwarding to allow DataDog/ELK without SDK lock-in
<code>force-dynamic</code> for catalog vs ISR revalidation	Always-fresh vs CDN cache hits	Freshness wins for admin edits; public catalog still benefits from <code>s-maxage=300</code> on API layer

Next: `05-Use-Cases-and-User-Stories.md` maps these NFRs to user-visible scenarios with step-by-step flows.

05-Use-Cases-and-User-Stories

- Use Cases & User Stories — Leish! v2
- 1. Use-Case Summary (Umbrella)
 - 2. Detailed Use Cases
 - 3. User Stories (INVEST) — Backlog Format
 - 4. Non-Functional Stories (Supplementary)
 - 5. Trace to FRS

Use Cases & User Stories — Leish! v2

FIELD	VALUE
Document ID	LEISH-UCS-v2.0
Version	2.0.0
Date	2026-08-29
Predecessor	BRS (01), SRS (02), FRS (03)
Notation	UC-XX use cases (Cockburn), US-XX stories (INVEST)

1. Use-Case Summary (Umbrella)



ID	USE CASE	PRIMARY ACTOR	PRIORITY
UC-01	Register & Verify Email	Guest	Must
UC-02	Login & Manage Session	Guest/Customer/Artist/Admin	Must
UC-03	Reset Forgotten Password	Guest	Must
UC-04	Discover Artists / Studios	Guest/Customer	Must
UC-05	View Artist/Studio Detail & Reviews	Guest	Must
UC-06	Submit Booking Request	Customer	Must
UC-07	Accept / Reject Booking	Artist	Must
UC-08	Build & Send Quotation	Artist	Must
UC-09	Pay Deposit (Booking Fee) & Confirm	Customer	Must
UC-10	Pay Balance (T-3d) & Trigger Payout	Customer	Must
UC-11	Cancel Booking / Request Refund	Customer / Artist	Must
UC-12	Complete Booking & Review	Customer / Artist	Must
UC-13	Chat Per-Booking (Live SSE)	Customer / Artist	Must
UC-14	Claim Artist/Studio Profile	Artist / Studio	Must
UC-15	Manage Own Artist Profile (Self-Service)	Artist / Studio	Should
UC-16	Admin Dashboard & User Management	Admin	Must
UC-17	Admin Catalog & Booking Overrides	Admin	Must
UC-18	Admin Payout Settlement	Admin	Must
UC-19	Export Data & Delete Account (PDPA)	Customer	Must
UC-20	Manage Email Preferences / View Outbox	Customer / Admin	Should

2. Detailed Use Cases

UC-01 — REGISTER & VERIFY EMAIL

Primary actor: Guest **Preconditions:** No active session; `LEISH_DB_PATH` reachable or PG available. **Success guarantee:** Account created with role, `email_verified=0`, verification token stored hashed.

STEP	ACTOR	ACTION
1	Guest	Opens <code>/register</code> , chooses role (Client/Artist/Studio toggles), enters name/email/password, checks PDPA consent, optionally solves Turnstile.
2	Guest	Submits; client validates inline via <code>registerSchema</code> .
3	System	Validates zod, lowercases email, checks uniqueness, hashes with scrypt+pepper, inserts <code>users</code> + <code>email_verifications</code> (hashed token), sets <code>leish_session</code> JWT cookie, enqueues verification email (or writes <code>email_outbox</code>).
4	System	Redirects to <code>/dashboard</code> ; banner "Verify your email" with resend button.
5	Guest	Opens email, clicks <code>/verify-email?token=...</code>
6	System	Validates single-use, non-expired; sets <code>email_verified=1</code> , marks <code>used_at</code> , redirects with success.

Alternate flows: - 1a Duplicate email → 409 generic error (no enumeration). - 3a Email provider down → dev outbox + warn log; dashboard still shows `devVerifyUrl`. - 5a Expired token → friendly "Link expired, request a new one."

Acceptance: See FRS FR-AUTH-01/05.

UC-02 — LOGIN & SESSION

Primary actor: Any authed role **Includes:** login, session refresh via middleware `proxy.ts`, logout/revocation.

STEP	ACTION
1	User opens <code>/login</code> (or redirected from gated route with <code>?redirect=/dashboard</code>).
2	Enters email/password (+ Turnstile when configured).
3	System verifies scrypt+pepper, creates <code>sessions(jti,...)</code> , signs JWT, sets cookie. <code>agnost.begin(login)</code> ends success=true.
4	Redirect to <code>redirect</code> or <code>/dashboard</code> with role label.
5	Later: visits any route — <code>verifySessionToken</code> checks sig+expiry+revoked; protected routes (<code>/dashboard</code> , <code>/onboarding</code> , <code>/admin/*</code>) enforce via middleware layout.
6	Click Log out → <code>POST /api/auth/logout</code> → <code>UPDATE sessions SET revoked=1</code> + clear cookie → redirect <code>/</code> .

Alternate: Wrong creds → 401 generic; rate-limited → 429 with Retry-After; revoked JTI → redirect to login.

UC-03 — RESET FORGOTTEN PASSWORD

STEP	ACTION
1	Guest on <code>/forgot-password</code> enters email (+ Turnstile).
2	System: rate-limit by IP, lookup user (always reply success-ish), if exists generate <code>password_resets</code> hashed token 1h, send email.
3	Guest clicks <code>/reset-password?token=...</code> , enters new password (8–128).
4	System verifies single-use + expiry, script new hash, marks <code>used_at</code> , invalidates other reset tokens. Can now log in.

Alternate: Expired token → error; reused token → `used_at` blocks.

UC-04 — DISCOVER ARTISTS / STUDIOS

Actors: Guest / Customer **Entry:** `/`, `/artists`, `/studios`

STEP	ACTION
1	Page server-fetches <code>ListAllArtists()</code> (DB-backed, <code>force-dynamic</code>) and renders <code>ArtistsBrowser</code> .
2	User interacts: type query (name/tagline/specialties), pick <code>state</code> (cascades <code>area</code> list from <code>AREAS_BY_STATE</code>), pick <code>bridal</code> / <code>nonBridal</code> tag, set <code>budget</code> , pick <code>date</code> . <code>trackSearch/trackArtistView</code> fire via <code>agnost-client</code> when configured.
3	System: <code>artistsQuerySchema</code> validates → <code>ListArtists(filters)</code> SQL pre-filters <code>state/area/budget</code> then <code>filterArtists</code> for remaining predicates — returns ranked by <code>rating DESC</code> .
4	User sees filtered cards (<code>ArtistCard</code>) with image, rating, priceFrom, area; clicks → <code>/artists/[slug]</code> .
5	Empty result → friendly “No artists found, try adjusting filters.”

UC-05 — VIEW ARTIST/STUDIO DETAIL

STEP	ACTION
1	User opens <code>/artists/aisha-azman</code> (slug=legacy id). Server resolves <code>resolveArtist(slug)</code> → <code>rowToArtist</code> + <code>ListEntityReviews</code> .
2	Page shows bio, specialties tags, services table (name/price/duration), portfolio carousel, availability slots, reviews (<code>RatingStars</code>), "Book Now" CTA.
3	"Book Now" requires auth → if guest → redirect <code>/login?redirect=/artists/[slug]/book</code> .

Alternate: Unknown slug → `notFound()` true 404 (no shell 200).

UC-06 — SUBMIT BOOKING REQUEST

Primary actor: Customer (verified for next step, but request allowed even if unverified per FRS FR-BOOK-01)

Entry: `/artists/[slug]/book` or booking modal on detail page.

STEP	ACTION
1	Customer picks <code>service</code> (from artist.services), <code>date</code> (≥today, datepicker), <code>time</code> (slot), enters <code>eventType</code> (select—bridal/nonBridal taxonomy), <code>venue</code> , <code>guestCount</code> , <code>notes</code> .
2	Validates <code>bookingSchema</code> ; client price hidden (server resolves).
3	Submits <code>POST /api/bookings</code> ; system pre-checks slot, inserts <code>bookings(status=requested)</code> , emails artist, Agnost <code>booking-create</code> .
4	Redirect to <code>/dashboard</code> where booking appears <code>requested</code> amber badge.

Alternates: - 3a Slot taken (unique index race) → 409 "just been taken, pick another time". - 3b Past date → 400 "Date cannot be in the past". - 3c Invalid service → 400 "Service not available for this artist".

UC-07 — ACCEPT / REJECT BOOKING

Primary actor: Artist (claimed)

STEP	ACTION
1	Artist sees <code>requested</code> bookings in dashboard filtered to claimedIds.
2	Clicks Accept → <code>PATCH /api/bookings/[id] {action:"accept"} →</code> <code>applyBookingTransition(requested,accept,{role:artist})</code> checks <code>isArtist</code> true and <code>current≡requested</code> → <code>accepted</code> ; or Reject → <code>cancelled</code> (terminal).
3	System updates row, optionally emails customer, shows new badge (sky <code>accepted</code> or red <code>cancelled</code>).

Alternates: Customer trying accept → "Only artists..."; accept on non-requested → error.

UC-08 — BUILD & SEND QUOTATION

Primary actor: Artist (claimed), booking must be `accepted` and not already pending quote (re-quote supersedes).

STEP	ACTION
1	Artist on booking card <code>accepted</code> without quotation sees "Build & send quotation"; clicks opens form.
2	Enters <code>baseFee</code> (required), optional <code>travelFee/earlyCallFee/accommodationFee</code> , adds ≤10 extras (label+amount), optional <code>artistNote</code> . Preview total auto-summed.
3	POST <code>/api/bookings/[id]/quotation</code> → system computes <code>quotationTotal()</code> , supersedes prior <code>pending</code> , inserts with <code>expires_at=now+24h</code> .
4	Customer dashboard now shows quotation breakdown, total, expiry countdown; server computed <code>balanceAmount = total - fee</code> and <code>balanceDueDate = eventDate - 3d</code> .

Alternate: Re-quote while `pending` → new row replaces, old becomes `superseded` ; quote after expiry (status `expired`) → new `pending` again allowed.

UC-09 — PAY DEPOSIT (BOOKING FEE) & CONFIRM

Primary actor: Customer

STEP	ACTION
1	Customer sees <code>accepted</code> + <code>quotation pending</code> + <code>Deposit status required</code> + "Pay RM 200 booking fee →" (+ Turnstile).
2	Click <code>Pay</code> → <code>POST /api/bookings/[id]/pay-fee</code> → <code>createBookingPayment('deposit', getBookingFeeSen())</code> ; system creates <code>payments(type=deposit, status=required)</code> ; if Billplz, returns <code>provider_url</code> Billplz hosted page.
3a	Dev: auto <code>markBillPaid</code> + <code>handlePaymentPaid</code> → <code>UPDATE bookings SET status='confirmed'</code> immediately; dashboard reloads → emerald <code>confirmed</code> + "✓ Booking fee paid".
3b	Billplz: open <code>provider_url</code> in new tab; after Billplz checkout, webhook <code>POST /api/payments/webhook</code> verified → same state change. Redirect landing <code>/dashboard</code> or <code>/booking/success</code> reads real status.
4	Fee is disclosed non-refundable throughout.

Alternate: Quotation expired → `totalPrice` null; pay-fee blocked until re-quoted.

UC-10 — PAY BALANCE (T-3D) & TRIGGER PAYOUT

STEP	ACTION
1	On <code>confirmed</code> booking, dashboard shows "Balance RM X due by <date (eventDate-3d)>" and "Pay balance" (when <code>balancePayment.status !== paid</code>).
2	Click <code>Pay balance (RM ...)</code> → <code>POST /api/bookings/[id]/pay-balance</code> → <code>amount = quotation.total - deposit</code> ; creates <code>payments(type=balance, status=required)</code> ; same Billplz/dev branching.
3	Dev auto-settle → <code>handlePaymentPaid(balance)</code> does <code>UPDATE quotations SET status='paid'</code> + <code>createPayoutForBooking</code> ; payouts row appears <code>pending</code> in admin.
4	Billplz webhook same logic; then payout <code>settleable_at = eventDate + 24h</code> .

Alternate: Zero balance (tiny quotation) → no payment needed / informative state.

UC-11 — CANCEL BOOKING / REQUEST REFUND

ACTOR	ACTION
Customer	On <code>requested accepted confirmed</code> sees Cancel → <code>PATCH {action:"cancel"}</code> → if <code>completed/cancelled</code> already → "no longer be cancelled"; else → <code>cancelled</code> .
Artist	Same; either side may cancel non-terminal; <code>isOwner</code> or <code>isArtist</code> suffices.
Refund	After <code>cancelled</code> that was <code>confirmed</code> and had <code>balance paid</code> : Customer sees Request refund → <code>POST /api/bookings/[id]/refund</code> → system refunds <code>balance</code> only; deposit stays. <code>dev</code> marks <code>refunded</code> directly; <code>billplz</code> calls refund API.

UC-12 — COMPLETE BOOKING & REVIEW

STEP	ACTION
1	Artist on <code>confirmed</code> clicks Complete → <code>PATCH {action:"complete"}</code> → <code>applyBookingTransition(confirmed,complete)</code> → <code>completed</code> (terminal).
2	Customer sees booking <code>completed</code> ; eligible for review: <code>findReviewableBooking</code> returns it (most recent completed without review).
3	Customer posts <code>POST /api/artists/[slug]/reviews {rating 1-5, text, event}</code> → <code>addEntityReview</code> guards duplicate, <code>INSERT reviews</code> , <code>blendAggregate</code> atomically updates artist <code>rating/review_count</code> .
4	New review appears on artist detail via <code>listEntityReviews</code> ordered desc.

Alternate: Studio bookings currently not reviewable (returns null). Second review for same booking → `ALREADY_REVIEWED` 409.

UC-13 — CHAT PER-BOOKING (LIVE SSE)

STEP	ACTION
1	On booking card, either party clicks "Chat" → collapsible <code>ChatThread</code> opens.
2	Component mounts SSE: <code>new EventSource(/api/bookings/[id]/messages/stream)</code> ; server replays history ordered ASC, then streams live via <code>chat-bus</code> pub/sub.
3	User types draft, POST <code>/api/bookings/[id]/messages {body}</code> → persisted + published → other participant receives event without refresh.
4	Authz: non-participant GET/POST returns 403.

UC-14 — CLAIM ARTIST/STUDIO PROFILE

STEP	ACTION
1	Artist/Studio registers → sees dashboard block "Your artist profile" + <code><select></code> of all artists from <code>/api/catalog/artists</code> + Claim profile button.
2	POST <code>/api/artist-profiles {artistId}</code> → checks role, uniqueness, inserts <code>artist_profiles(user_id,artist_id)</code> .
3	Future <code>GET /api/bookings</code> for that user is scoped to claimed ids; bookings for unclaimed artists remain invisible.

Alternate: Same `artist_id` already claimed by another user → 409; same `user_id` tries second profile → PK violation 409.

UC-16 — ADMIN DASHBOARD & USER MANAGEMENT

Actor: Admin **Entry:** `/admin` (layout guard redirects non-admins).

STEP	ACTION
1	Admin logs in (<code>role=admin</code> — seeded via <code>ADMIN_EMAIL/ADMIN_PASSWORD npx tsx scripts/seed-admin.ts</code>).
2	<code>/admin</code> fetch <code>GET /api/admin</code> → stats (users by role, bookings by status, payments revenue, recent lists) render <code>StatCard</code> s + tables.
3	<code>/admin/users</code> → list, create, <code>PATCH /api/admin/users/[id]</code> role/status edit, <code>DELETE</code> — every mutation audited (<code>admin_audit_log</code>).
4	Non-admin direct API call blocked 403 even without UI.

UC-18 — ADMIN PAYOUT SETTLEMENT

STEP	ACTION
1	Admin opens <code>/admin/payouts</code> ; <code>GET /api/admin/payouts?status=pending</code> lists payouts joined with booking+artist name.
2	Click Settle or Fail → <code>PATCH /api/admin/payouts {payoutId, status, notes}</code> → <code>updatePayoutStatus</code> sets <code>settled_at</code> when settled, logs audit.
3	Payout starts <code>pending</code> with <code>settleable_at = eventDate+24h</code> dispute window; settling before window shows warning but is allowed.

UC-19 — EXPORT DATA & DELETE ACCOUNT (PDPA)

STEP	ACTION
1	Any authed user on <code>/dashboard</code> "Your data" section clicks Export my data → <code>GET /api/me/export</code> downloads <code>leish-data-...json</code> (bookings+messages+payments+profile+preferences).
2	Click Delete my account → confirm dialog → <code>DELETE /api/me?confirm=1</code> → cascade via FK deletes owned data; anchor audit rows kept with <code>admin_user_id=null</code> ; cookie cleared; redirect <code>/</code> .

3. User Stories (INVEST) — Backlog Format

Stories complement use cases for sprint planning. Acceptance is Given/When/Then.

EPIC 1: DISCOVERY

ID	STORY	POINTS	PRIORITY	ACCEPTANCE
US-01	As a Guest , I want to browse featured artists on the home page, so I can quickly see top MUAs.	3	Must	Given  loads, when page renders, then top 3 by rating shown with hero/category/how-it-works/CTA.
US-02	As a Customer , I want to filter artists by state/area/budget/event , so I can find local affordable options.	5	Must	G/W filter state Selangor Then <code>ListArtists(state=Selangor)</code> returns only Selangor; area dropdown cascades.
US-03	As a Customer , I want to search artists by name/tagline/specialty text, so I can find known MUAs.	2	Must	G/W query "Aisha" Then results include Aisha Azman.

EPIC 2: ACCOUNT

ID	STORY	POINTS	ACCEPTANCE
US-04	As a Guest , I want to register as customer/artist/studio with PDPA consent, so I can transact.	3	See UC-01; zod errors inline; scrypt hashed.
US-05	As a User , I want to verify my email via link and have a resend button, so I can trust my account.	3	See UC-01 steps 5–6; links single-use hashed.
US-06	As a User , I want to log in/out and stay signed in 7 days across tabs, so I'm not re-authing constantly.	2	JWT cookie httpOnly; revoked JTI rejected.
US-07	As a User , I want to reset forgotten password without leaking which emails exist.	5	Forgot always 200-ish + 1h single-use token.
US-08	As an Artist , I want to claim my profile to receive relevant bookings only.	3	See UC-14; scoped <code>GET /api/bookings</code> .

EPIC 3: BOOKING & MONEY

ID	STORY	POINTS	ACCEPTANCE
US-09	As a Customer , I want to request a booking with service/date/time/event details and have the slot locked , so I don't lose it.	5	Slot unique index + 409; price derived server-side.
US-10	As an Artist , I want to accept/reject requests for my claimed profile.	2	<code>applyBookingTransition</code> enforced.
US-11	As an Artist , I want to quote (base/travel/early/accommodation+extras) with 24h window and re-quote if needed.	8	<code>quotationTotal</code> sum; <code>expires_at=now+24h</code> ; superseded.
US-12	As a Customer , I want to pay deposit (RM50) via Billplz/dev and see booking confirmed immediately (webhook).	8	<code>pay-fee</code> creates Billplz bill; webhook verified timing-safe.
US-13	As a Customer , I want to pay balance (total-deposit) due 3 days before the event.	5	<code>balanceDueDate</code> = eventDate-3d; payout created on paid.
US-14	As a Customer , I want to cancel and get balance refunded (deposit non-refundable) if needed.	3	<code>refundBalancePayment</code> guards balance-only.
US-15	As an Artist , I want my payout computed correctly (waiver, commission) and settleable after 24h .	5	<code>computeCommission</code> + <code>net</code> = <code>total-commission-deposit</code> .

EPIC 4: SOCIAL & COMMS

ID	STORY	POINTS	ACCEPTANCE
US-16	As a Customer/Artist , I want per-booking chat that updates live without refresh.	5	SSE stream replay+live; 403 for outsiders.
US-17	As a Customer touching confirmed→completed , I want to review (once) and see rating update.	3	Gated on completed+unreviewed; atomic blend 2dp.
US-18	As a User , I want to control email prefs (toggle types) and not be spammed.	2	email_preferences toggles honored.
US-19	As a Customer , I want invoice HTML + PDF for completed/confirmed bookings.	3	GET /invoice + /invoice.pdf for owner/claimed only.

EPIC 5: ADMIN

ID	STORY	POINTS	ACCEPTANCE
US-20	As an Admin , I want a dashboard with user/booking/payment stats + recent activity.	3	GET /api/admin aggregates.
US-21	As an Admin , I want to CRUD users/artists/studios/bookings with all changes audited .	5	admin_audit_log on every mutation.
US-22	As an Admin , I want to settle/fail payouts after the dispute window.	3	PATCH /api/admin/payouts .
US-23	As an Admin , I want to tune platform settings (fee/commission/waiver) without redeploying.	2	PATCH /api/admin/settings + 30s cache.

EPIC 6: COMPLIANCE

ID	STORY	POINTS	ACCEPTANCE
US-24	As a Customer , I want to export my data as JSON and delete my account fully.	3	PDPA; cascade semantics.

4. Non-Functional Stories (Supplementary)

ID	STORY	ACCEPTANCE
NFR-US-01	As Engineering , I want CI to gate every PR on lint/typecheck/test/coverage≥80%/format, so debt doesn't ship.	<code>quality-gate.yml</code> required.
NFR-US-02	As Ops , I want logs structured then forwarded to webhook in batches, so I can pipe to ELK/Datadog.	<code>LOG_WEBHOOK_URL</code> 20/50ms batch.
NFR-US-03	As Security , I want security headers + CSP nonce + HMAC webhook, so attacks are rejected early.	<code>next.config.ts</code> headers + <code>verifyBillplzSignature</code> timing-safe.

5. Trace to FRS

UC	FRS IDS
UC-01	FR-AUTH-01, FR-AUTH-05
UC-02	FR-AUTH-02, FR-AUTH-03, FR-AUTH-04, FR-AUTH-07
UC-03	FR-AUTH-06
UC-04	FR-CAT-01, FR-CAT-03
UC-05	FR-CAT-02
UC-06	FR-BOOK-01, FR-DASH-01
UC-07	FR-BOOK-03
UC-08	FR-QUO-01, FR-QUO-02
UC-09	FR-PAY-02, FR-PAY-04
UC-10	FR-PAY-03, FR-PAY-04, FR-PAYO-02
UC-11	FR-BOOK-03, FR-PAY-05
UC-12	FR-BOOK-03, FR-REV-01..03
UC-13	FR-MSG-01, FR-MSG-02
UC-14	FR-CLAIM-01, FR-CLAIM-02
UC-16..18	FR-ADM-01..04, FR-PAYO-03
UC-19	FR-COMP-01, FR-COMP-02
UC-20	FR-MAIL-03, FR-ADM-02 (emails)

Full traceability lives in `docs/08-Traceability-Matrix.md`.

Next: `docs/06-Data-Model-and-ERD.md` specifies the storage for every entity referenced above.

06-Data-Model-and-ERD

Data Model & ERD — Leish! v2

1. Overview & Conventions

2. Entity-Relationship Diagram

3. Table Dictionaries

4. Constraints & Indexes (Summary)

5. Money Logic & Derived Fields

6. Seeding & Migration

7. Data Retention & Lifecycle

8. Verification of Model

Data Model & ERD — Leish! v2

FIELD	VALUE
Document ID	LEISH-DATA-v2.0
Version	2.0.0
Date	2026-08-29
Source of Truth	<code>src/server/db.ts</code> — <code>PG_SCHEMA</code> (line 207) + <code>SQLITE_SCHEMA</code> (line 477)
Access Facade	<code>getDb(): DbFacade</code> — <code>prepare(sql) → Statement {get,all,run}</code> + <code>exec(sql)</code>
Migration	<code>scripts/migrate.ts</code> (<code>npm run db:migrate</code>) idempotent; SQLite <code>migrateSqlite()</code> auto-runs

1. Overview & Conventions

- **Two-backend facade:** `isPostgres()` → `pg.Pool` (production, `DATABASE_URL`) else `node:sqlite DatabaseSync` (`LEISH_DB_PATH` or `data/leish.db` , `WAL` + `foreign_keys=ON`). Placeholders `@name` / `?` translated to `$n` for PG (`compilePlaceholders`).
- **IDs:** `TEXT PK` — `randomUUID()` / `crypto.randomUUID()` .

- **Times:** `TEXT` ISO-8601 `new Date().toISOString()` everywhere (no `TIMESTAMPZ` variance across backends).
- **Money:** `INTEGER` sen (cents) — never float. Only `artists.rating` is `REAL`.
- **Booleans:** `INTEGER 0/1` (`verified`, `email_verified`, `consent`, `revoked`, `email_preferences` flags).
- **JSON arrays:** `TEXT` containing `JSON.stringify(...)` (`specialties`, `services`, `portfolio`, `extras`, etc.).
- **Cascade:** `bookings/payments/quotations/messages` `CASCADE` on parent delete (PDPA);
`admin_audit_log.admin_user_id` `SET NULL` (preserve audit).

drift guard: `detectSchemaDrift()` warns when `PG_SCHEMA` and `SQLITE_SCHEMA` diverge on tables/columns (dev-time only).

2. Entity-Relationship Diagram



```
admin_audit_log PK(id) FK admin_user_id→users SET NULL
```

Legacy removed scaffold: catalog_overrides (kept for migration but folded into real columns at seed)

3. Table Dictionaries

3.1 USERS

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	TEXT	PK, randomUUID()	Public user id (toPublicUser strips password)
email	TEXT	UNIQUE, lowercased	Login key; idx_users_email
name	TEXT	NOT NULL	Display name (2–80 validated)
role	TEXT	CHECK customer artist studio admin	See src/lib/types.ts:1
password	TEXT	NOT NULL	scrypt hash (+ pepper HMAC if PEPPER_SECRET)
email_verified	INTEGER	0/1 default 0, backfilled via migrateSqlite when missing	Verification gate
consent	INTEGER	0/1 default 0	PDPA consent flag
consent_timestamp	TEXT	nullable ISO	When consent was given
created_at	TEXT	NOT NULL ISO	

Indexes: idx_users_email(email) . **Backfill:** SQLite adds email_verified, consent, consent_timestamp via ALTER TABLE when absent.

3.2 ARTISTS (RUNTIME CATALOG)

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
<code>id</code>	TEXT	PK	<code>aisha-azman</code> for seeds, UUID for admin-created
<code>slug</code>	TEXT	UNIQUE	URL key; seeded <code>slug=id</code> ; dedup via <code>slugifyName()+suffix</code>
<code>name</code>	TEXT	NOT NULL	
<code>tagline</code>	TEXT	default ""	
<code>bio</code>	TEXT	default ""	
<code>image</code>	TEXT	default ""	URL or <code>/images/...</code>
<code>rating</code>	REAL	default 0	Blended atomically (2dp)
<code>review_count</code>	INTEGER	default 0	
<code>state</code>	TEXT	default ""	<code>MALAYSIA_STATES</code> value
<code>area</code>	TEXT	default ""	within state's <code>AREAS_BY_STATE</code>
<code>price_from</code>	INTEGER	default 0	sen; min service price
<code>verified</code>	INTEGER	0/1 default 0	Admin-only field
<code>years_experience</code>	INTEGER	default 0	
<code>specialties</code>	TEXT	JSON []	string[]
<code>services</code>	TEXT	JSON []	<code>Service[]</code> <code>{name,price,duration}</code>
<code>bridal</code>	TEXT	JSON []	<code>BridalEvent[]</code>
<code>non_bridal</code>	TEXT	JSON []	<code>NonBridalEvent[]</code>
<code>availability</code>	TEXT	JSON []	string[] slots e.g. "Tomorrow, 10:00 AM"
<code>portfolio</code>	TEXT	JSON []	image URLs
<code>referral_code</code>	TEXT	default ""	Assigned via <code>assignReferralCode</code>
<code>referred_by</code>	TEXT	FK artists(id) SET NULL	Referrer
<code>referral_earnings</code>	INTEGER	default 0	sen
<code>created_at</code> , <code>updated_at</code>	TEXT	NOT NULL	

Indexes: `idx_artists_state_area(state,area)`.

3.3 STUDIOS — SAME SHAPE, REPLACES TAGLINE→DESCRIPTION? ACTUALLY BOTH

TAGLINE + DESCRIPTION ; SERVICES IS STRING[] NOT SERVICE[] ; PLUS ADDRESS , HOURS , PHONE .

Full DDL identical to PG_SCHEMA block lines 403–424.

3.4 BOOKINGS

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	TEXT	PK	
user_id	TEXT	FK users CASCADE	Customer owner; idx_bookings_user
artist_id	TEXT	NOT NULL	Catalog artist booked (text, intentionally not strict FK for seed flexibility)
studio_id	TEXT	FK studios SET NULL, nullable	Studio variant (Option B); backfilled
artist_name	TEXT	NOT NULL	Denormalized at booking time for fast display
service	TEXT	NOT NULL	Service name chosen
price	INTEGER	NOT NULL	sen; server-derived from artist.services.find(service).price
date	TEXT	YYYY-MM-DD	Event date (≥ today at creation)
time	TEXT	NOT NULL	Slot label ("10:00 AM" etc)
notes	TEXT	nullable	≤2000 chars
event_type	TEXT	nullable	e.g. "solemnization" (backfilled)
venue	TEXT	nullable	≤200 chars (backfilled)
guest_count	INTEGER	default 0	0–1000 (backfilled)
status	TEXT	CHECK requested accepted confirmed cancelled completed default requested	State machine
balance_reminder_at	TEXT	nullable ISO	When reminder sent (backfilled)
created_at	TEXT	NOT NULL	

Critical constraint:

```
CREATE UNIQUE INDEX IF NOT EXISTS uq_bookings_slot
ON bookings(artist_id, date, time)
WHERE status IN ('requested', 'accepted', 'confirmed');
```

Prevents double-booking live slots; cancelled/completed don't block.

3.5 QUOTATIONS

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
<code>id</code>	TEXT	PK	
<code>booking_id</code>	TEXT	FK bookings CASCADE	
<code>base_fee</code>	INTEGER	default 0	sen
<code>travel_fee</code>	INTEGER	default 0	
<code>early_call_fee</code>	INTEGER	default 0	
<code>accommodation_fee</code>	INTEGER	default 0	
<code>extras</code>	TEXT	JSON <code>[]</code>	<code>ExtraItem[] {label, amount sen}</code> ≤10
<code>artist_note</code>	TEXT	nullable	≤1000
<code>total</code>	INTEGER	NOT NULL	<code>quotationTotal()</code> computed sen
<code>status</code>	TEXT	CHECK <code>pending paid expired superseded</code> default pending	
<code>created_at</code> , <code>expires_at</code>	TEXT	NOT NULL	<code>expires_at = created_at + 24h</code>

3.6 PAYMENTS (HYBRID)

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
<code>id</code>	TEXT	PK	<code>PaymentRecord.id</code>
<code>booking_id</code>	TEXT	FK bookings CASCADE	<code>idx_payments_booking</code>
<code>type</code>	TEXT	CHECK <code>deposit balance</code> default deposit	
<code>amount</code>	INTEGER	NOT NULL	sen
<code>currency</code>	TEXT	default MYR	
<code>provider</code>	TEXT	default dev	<code>dev billplz</code>
<code>status</code>	TEXT	CHECK <code>required paid failed refunded</code> default required	
<code>provider_ref</code>	TEXT	nullable	Billplz <code>id</code> or <code>dev_...</code>
<code>provider_url</code>	TEXT	nullable	Billplz hosted page URL
<code>created_at</code> , <code>updated_at</code>	TEXT	NOT NULL	

Unique: `CREATE UNIQUE INDEX uq_payments_booking_type ON payments(booking_id,type)` — one deposit + one balance per booking. **Migration:** legacy single-`UNIQUE(booking_id)` tables rebuilt to this shape; backfill adds `type` and `provider_url` if missing.

3.7 `PAYOUTS`

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
<code>id</code>	TEXT	PK	
<code>artist_user_id</code>	TEXT	FK users SET NULL	Claimed artist user (nullable when unclaimed)
<code>booking_id</code>	TEXT	FK bookings CASCADE	one payout per booking
<code>gross_sen</code>	INTEGER	NOT NULL	quotation.total
<code>commission_sen</code>	INTEGER	NOT NULL	computed via <code>computeCommission</code>
<code>net_sen</code>	INTEGER	NOT NULL	<code>max(0, gross - commission - deposit)</code>
<code>status</code>	TEXT	CHECK <code>pending settled failed</code> default pending	
<code>settleable_at</code>	TEXT	nullable ISO	<code>eventDate+24h</code>
<code>settled_at</code>	TEXT	nullable ISO	set when <code>settled</code>
<code>notes</code>	TEXT	nullable	"Commission waived (small booking)" etc
<code>created_at</code>	TEXT	NOT NULL	

Indexes: `idx_payouts_status`, `idx_payouts_artist`. **Cardinality:** ≤1 per `booking_id` (app guard; could be made `UNIQUE` in future).

3.8 `SESSIONS` (JTI REVOCATION)

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
<code>jti</code>	TEXT	PK	JWT ID
<code>user_id</code>	TEXT	FK users CASCADE	owner; <code>idx_sessions_user</code>
<code>revoked</code>	INTEGER	0/1 default 0	revoked=1 means blocked
<code>expires_at</code>	TEXT	NOT NULL	<code>exp</code> mirrored for cleanup
<code>created_at</code>	TEXT	NOT NULL	

3.9 REVIEWS

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
<code>id</code>	TEXT	PK	
<code>entity_type</code>	TEXT	CHECK (<code>artist studio</code>)	polymorphic target
<code>entity_id</code>	TEXT	NOT NULL	<code>artists.id</code> or <code>studios.id</code>
<code>booking_id</code>	TEXT	UNIQUE FK bookings CASCADE	gates: one review per booking
<code>user_id</code>	TEXT	FK users SET NULL	author; nullable for seed rows
<code>author_name</code>	TEXT	NOT NULL	denormalized display name
<code>rating</code>	INTEGER	CHECK 1-5	rounded int
<code>event</code>	TEXT	nullable	service/event label
<code>text</code>	TEXT	NOT NULL	review body
<code>created_at</code>	TEXT	NOT NULL	

Index: `idx_reviews_entity(entity_type,entity_id)`.

3.10 REFERRALS

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
<code>id</code>	TEXT	PK	
<code>referrer_type</code>	TEXT	CHECK (<code>artist studio</code>)	who referred
<code>referrer_id</code>	TEXT	NOT NULL	
<code>referee_type</code>	TEXT	CHECK (<code>artist studio</code>)	who was referred
<code>referee_id</code>	TEXT	NOT NULL	
<code>status</code>	TEXT	CHECK (<code>pending qualified paid</code>) default pending	
<code>reward_sen</code>	INTEGER	default 0	earned
<code>qualified_at</code> , <code>paid_at</code>	TEXT	nullable ISO	
<code>created_at</code>	TEXT	NOT NULL	

Indexes: `idx_referrals_referrer`, `idx_referrals_referee`, `idx_referrals_status`.

3.11 MESSAGES

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
<code>id</code>	TEXT	PK	
<code>booking_id</code>	TEXT	FK bookings CASCADE	<code>idx_messages_booking</code>
<code>sender_id</code>	TEXT	FK users CASCADE	
<code>body</code>	TEXT	NOT NULL	message text
<code>created_at</code>	TEXT	NOT NULL	ordered ASC for replay

3.12 COMMUNICATION & EMAIL TABLES

TABLE	KEY COLUMNS	PURPOSE
<code>email_outbox</code>	<code>id, to_email, subject, text, html, created_at</code>	de pr sin vie at em
<code>email_retries</code>	<code>id, to_email, subject, text, html, attempts, max_attempts=3, next_retry, last_error, created_at</code>	As re qu to at
<code>email_preferences</code>	<code>user_id PK FK, booking_created, quotation_sent, invoice_sent, quotation_expiry, balance_reminder, status_changed INTEGER 0/1 default1, updated_at</code>	Pe to
<code>email_verifications</code>	<code>id, user_id FK CASCADE, token_hash, expires_at, used_at, created_at</code>	Si us ha to ve
<code>password_resets</code>	<code>id, user_id FK CASCADE, token_hash, expires_at, used_at, created_at</code>	Si 1h re

3.13 OPERATIONAL TABLES

TABLE	KEY COLUMNS	PURPOSE
<code>artist_profiles</code>	<code>user_id</code> PK FK CASCADE, <code>artist_id</code> TEXT <code>UNIQUE</code> , <code>claimed_at</code>	Artist/studio claims — scope filter
<code>studio_profiles</code>	<code>user_id</code> PK FK CASCADE, <code>studio_id</code> TEXT <code>UNIQUE</code> , <code>claimed_at</code>	Same for studios
<code>platform_settings</code>	<code>key</code> PK, <code>value</code> TEXT, <code>updated_by</code> FK SET NULL, <code>updated_at</code>	Runtime knobs: <code>booking_fee_sen</code> , <code>commission_rate_bps</code> , <code>commission_waiver_sen</code>
<code>admin_audit_log</code>	<code>id</code> PK, <code>admin_user_id</code> FK SET NULL, <code>action</code> , <code>target_table</code> , <code>target_id</code> , <code>details</code> JSON '{}', <code>created_at</code>	Audit trail; indexes on <code>admin_user_id</code> , <code>created_at</code>
<code>catalog_overrides</code>	<code>id</code> PK, <code>entity_type</code> , <code>entity_id</code> , <code>field</code> , <code>value</code> , <code>updated_by</code> , <code>created_at</code> , <code>updated_at</code> , <code>UNIQUE(entity_type,entity_id,field)</code>	Legacy — folded into real columns at seed; kept for migration but <code>idx_catalog_overrides_entity</code> still indexed

Note on FKs in tests: `admin_audit_log.admin_user_id → users(id)` has real FK; tests must seed a user row before inserting audit.

4. Constraints & Indexes (Summary)

CONSTRAINT / INDEX	SQL EXCERPT	PURPOSE
<code>uq_bookings_slot</code>	<code>UNIQUE(artist_id,date,time) WHERE status IN (...)</code>	Anti double-booking
<code>uq_payments_booking_type</code>	<code>UNIQUE(booking_id,type)</code>	One deposit + one balance
<code>uq_artist_profiles_artist_id</code>	<code>UNIQUE(artist_id)</code>	One owner per artist
<code>uq_studio_profiles_studio_id</code>	<code>UNIQUE(studio_id)</code>	One owner per studio
<code>reviews.booking_id UNIQUE</code>	Column constraint	One review per booking
Role checks	<code>CHECK role IN (...)</code> etc	Enforce vocabularies
State checks	<code>CHECK status IN (...)</code>	Enforce booking/quote/payment/payout states
Range checks	<code>CHECK rating BETWEEN 1 AND 5</code>	Validate rating
Foreign keys	<code>REFERENCES ... ON DELETE CASCADE/SET NULL</code>	PDPA & audit semantics
Area index	<code>idx_artists_state_area(state,area)</code>	Fast catalog pre-filter
Booking user index	<code>idx_bookings_user(user_id)</code>	Customer booking list
Payment booking index	<code>idx_payments_booking(booking_id)</code>	Serialization join

5. Money Logic & Derived Fields

Quoting (`src/server/quotations.ts:44`):

```
function quotationTotal(input): number {
  return input.baseFee + (travelFee??0) + (earlyCallFee??0) + (accommodationFee??0) + sum(extras.amou
nt)
```

Stored in `quotations.total` ; never float.

Commission (`src/server/settings.ts:100`):

```
function computeCommission(totalSen, rateBps, waiverSen) {
  const waived = total < waiverSen;
  const commissionSen = waived ? 0 : Math.round(total * rateBps / 10000);
  return { totalSen, waived, commissionSen, artistNetSen: total - commissionSen };
}
```

Payout net (`src/server/payouts.ts:79`): `artistReceivesSen = max(0, artistNetSen - depositSen)` where `depositSen = getBookingFeeSen()`.

Booking serialization (`src/app/api/bookings/route.ts:39`): `balanceDueDate = eventDate - 3 days`, `balanceAmount = max(0, total - fee)` when quotation exists and not expired.

6. Seeding & Migration

6.1 CATALOG SEEDING (IDEMPOTENT)

- Source: `SEED_ARTISTS` (7 entries) + `SEED_STUDIOS` (4) `src/lib/data.ts`.
- Runner: `ensureCatalogSeeded()` called lazily before first catalog read AND explicitly via `npm run db:seed-catalog` (`scripts/seed-catalog.ts`).
- Logic: Inserts new `artists/studios` rows when missing; legacy `catalog_overrides` rows are **folded** into real columns and deleted (so old overrides don't drift).
- Placeholders: Admin-created artists without image get `/images/hero.jpg` to satisfy `next/image` requirements.

6.2 ADMIN SEEDING

- `scripts/seed-admin.ts` : reads `ADMIN_EMAIL/ADMIN_PASSWORD`, upserts `users` (`role='admin'`), hash via `scrypt`; idempotent upgrade of existing non-admin user.

6.3 SCHEMA MIGRATIONS

- **Postgres**: `PG_SCHEMA` applied lazily on `getDb()` first query via `pool.query(PG_SCHEMA)` in `pgReady`; explicitly via `scripts/migrate.ts`.
- **SQLite**: `SQLITE_SCHEMA` + `migrateSqlite(db)` additive `ALTER TABLE` + partial-index creation; supports aging DBs lacking newer columns (`email_verified`, `provider_url`, `type`, etc.).
- Both backends apply `CREATE UNIQUE INDEX IF NOT EXISTS uq_payments_booking_type` separately to avoid rollback of entire schema on legacy databases.

7. Data Retention & Lifecycle

Per `docs/PDPA_RETENTION_GUIDELINES.md`:

ENTITY	RETENTION	DELETION
<code>users</code>	Until account deletion; then anonymized audit log retained with <code>admin_user_id→null</code>	<code>DELETE /api/me</code> cascades bookings/ messages/payments via FK <code>CASCADE</code>
<code>bookings/quotations/ payments</code>	7 years for financial audit (configurable)	Cascade on user delete; admin may override status but never hard-delete for audit
<code>messages</code>	Same as booking	Cascade
<code>reviews</code>	Indefinite (anonymized if user deleted → <code>user_id null</code> kept)	No hard delete of public proof
<code>sessions</code>	Expire after <code>expires_at</code> ; gc via cron or lazy cleanup	<code>revoked</code> rows inert
<code>email_outbox</code>	Dev only; not retention-bound	Ephemeral

8. Verification of Model

CHECK	METHOD
PG_SCHEMA vs SQLite drift	<code>extractSchemaTables()</code> + <code>detectSchemaDrift()</code> warning (dev) — run <code>npm run db:migrate</code> and compare <code>PG_SCHEMA</code> / <code>SQLITE_SCHEMA</code> tables+columns
Placeholder translation	Unit tests <code>compilePlaceholders/resolveParams</code>
FK enforcement	Integration tests with <code>:memory:</code> SQLite <code>foreign_keys=ON</code>
Money integer	Lint grep for float amounts in <code>src/server/ {payments,quotations,payouts}.ts</code>
Index coverage	<code>EXPLAIN QUERY PLAN</code> on <code>listArtists(state,area)</code> must use <code>idx_artists_state_area</code>

Next: `docs/07-API-Specification.md` details HTTP contracts over this model; `docs/08-Traceability-Matrix.md` maps requirements to tables and endpoints.

07-API-Specification

API Specification — Leish! v2

1. Conventions
2. Endpoint Index
3. Authentication & Sessions
4. Catalog APIs
5. Booking APIs
6. Messaging APIs
7. Payments Webhook
8. Profile Claim
9. Reviews
10. Data Rights
11. Email Preferences
12. Upload
13. Admin APIs (all behind `requireAdmin`)
14. Health & Cron
15. Error Handling
16. Security Notes (API Consumers)
17. Example: End-to-End Booking Flow via API

API Specification — Leish! v2

FIELD	VALUE
Document ID	LEISH-API-v2.0
Version	2.0.0
Date	2026-08-29
Base URL	<code>https://leish.my</code> (prod) / <code>http://localhost:3000</code> (dev) — <code>NEXT_PUBLIC_SITE_URL</code>
Auth	<code>Cookie: leish_session=<JWT></code> (httpOnly, SameSite=lax); no Bearer header
Content-Type	<code>application/json</code> unless noted (<code>text/event-stream</code> , <code>application/pdf</code> , <code>text/html</code>)
Errors	JSON envelope <code>{error: string}</code> with appropriate HTTP status; zod validation returns first issue message
Helpers	<code>src/server/http.ts</code> — <code>jsonError(message,status)</code> , <code>readJson(req)</code> , <code>tryRoute</code> / <code>statefulRoute</code> wrappers that log <code>pino</code> + normalize errors

1. Conventions

ITEM	RULE
Status codes	200 OK, 201 Created, 400 Bad Request (validation), 401 Unauthorized (missing/bad session), 403 Forbidden (role mismatch), 404 Not Found, 409 Conflict (duplicate/slot), 429 Too Many Requests, 500 Internal
Timestamps	ISO-8601 strings <code>YYYY-MM-DDTHH:mm:ss.sssZ</code> (UTC)
Money	Integer sen (cents). Presented as <code>formatRM(sen) → "RM X.XX"</code> client-side
Pagination	<code>?limit=1..100 default 20, ?offset=0.. default 0</code> → response <code>{pagination: {total, limit, offset, hasMore}}</code> + array
Rate limit	<code>429</code> + <code>Retry-After: <seconds></code> when exceeded; Sliding window via Upstash or in-memory fallback
CSR	<code>ALLOWED_ORIGINS</code> checked for state-changing (<code>POST/PATCH/DELETE</code>) cross-origin; otherwise standard <code>SRS-F-12</code>
Validation	Zod schemas in <code>src/server/validation.ts</code> enforce on every mutating endpoint

2. Endpoint Index

GROUP	METHOD	PATH	AUTH	ROUTE HANDLER
Health	GET	/api/health	No	src/app/api/health/route.ts
Auth — Register	POST	/api/auth/register	No	src/app/api/auth/register/route.ts
Auth — Login	POST	/api/auth/login	No	src/app/api/auth/login/route.ts
Auth — Logout	POST	/api/auth/logout	Cookie	src/app/api/auth/logout/route.ts
Auth — Me	GET	/api/auth/me	Cookie (soft)	src/app/api/auth/me/route.ts or GET /api/me variant
Auth — Verify Email	GET	/api/auth/verify-email?token=	No	src/app/api/auth/verify-email/route.ts
Auth — Resend Verify	POST	/api/auth/resend-verification	Cookie	src/app/api/auth/resend-verification/route.ts
Auth — Forgot	POST	/api/auth/forgot-password	No (rate-limited)	src/app/api/auth/forgot-password/route.ts
Auth — Reset	POST	/api/auth/reset-password	No	src/app/api/auth/reset-password/route.ts
Auth — Errors	POST	/api/errors	No (rate-limited)	src/app/api/errors/route.ts
Catalog — Artists	GET	/api/artists?query&state&area&bridal&nonBridal&budget	No	src/app/api/artists/route.ts
Catalog — Artists (client)	GET	/api/catalog/artists	No	src/app/api/catalog/artists/route.ts
Catalog — Studios	GET	/api/catalog/studios (or /api/studios)	No	src/app/api/catalog/studios/route.ts

GROUP	METHOD	PATH	AUTH	ROUTE HANDLER
Catalog — Artist by slug	GET	/artists/[slug] (RSC, not API)	No	src/app/artists/[slug]/page.tsx (resolveArtist)
Bookings — List	GET	/api/bookings?limit&offset	Cookie	src/app/api/bookings/ route.ts:86
Bookings — Create	POST	/api/bookings	Cookie	src/app/api/bookings/ route.ts:151
Bookings — Transition	PATCH	/api/bookings/[id]	Cookie	src/app/api/bookings/[id]/ route.ts
Bookings — Quotation	POST	/api/bookings/[id]/quotation	Cookie (claimed artist)	src/app/api/bookings/[id]/ quotation/ route.ts
Bookings — Pay Fee	POST	/api/bookings/[id]/pay-fee	Cookie (owner)	src/app/api/bookings/[id]/ pay-fee/ route.ts
Bookings — Pay Balance	POST	/api/bookings/[id]/pay-balance	Cookie (owner)	src/app/api/bookings/[id]/ pay-balance/ route.ts
Bookings — Remind	POST	/api/bookings/[id]/remind	Cookie (claimed artist)	src/app/api/bookings/[id]/ remind/route.ts
Bookings — Refund	POST	/api/bookings/[id]/refund	Cookie (owner)	src/app/api/bookings/[id]/ refund/route.ts
Bookings — Invoice HTML	GET	/api/bookings/[id]/invoice	Cookie (owner artist)	
Bookings — Invoice PDF	GET	/api/bookings/[id]/invoice.pdf	Cookie (owner artist)	
Messages — List	GET	/api/bookings/[id]/messages	Cookie (participant)	src/app/api/bookings/[id]/ messages/ route.ts
Messages — Create	POST	/api/bookings/[id]/messages	Cookie (participant)	src/app/api/bookings/[id]/ messages/ route.ts

GROUP	METHOD	PATH	AUTH	ROUTE HANDLER
Messages — Stream	GET	<code>/api/bookings/[id]/messages/stream</code>	Cookie (participant)	<code>src/app/api/bookings/[id]/messages/stream/route.ts</code> (SSE)
Payments — Webhook	POST	<code>/api/payments/webhook</code>	HMAC (<code>X-Billplz-Signature</code>)	<code>src/app/api/payments/webhook/route.ts</code>
Artists — Claimed	GET	<code>/api/artist-profiles</code>	Cookie	<code>src/app/api/artist-profiles/route.ts</code>
Artists — Claim	POST	<code>/api/artist-profiles</code>	Cookie (artist/studio)	<code>src/app/api/artist-profiles/route.ts</code>
Artists — Edit Self	PATCH	<code>/api/artist-profiles</code>	Cookie (claimed)	<code>src/app/api/artist-profiles/route.ts</code>
Studios — Claim	POST/ GET/ PATCH	<code>/api/studio-profiles</code>	Cookie	<code>src/app/api/studio-profiles/route.ts</code>
Reviews — Add	POST	<code>/api/artists/[slug]/reviews</code>	Cookie	<code>src/app/api/artists/[slug]/reviews/route.ts</code> (or via <code>src/server/catalog.ts</code> helper)
Me — Export	GET	<code>/api/me/export</code>	Cookie	<code>src/app/api/me/export/route.ts</code>
Me — Delete	DELETE	<code>/api/me?confirm=1</code>	Cookie	<code>src/app/api/me/route.ts</code>
Email — Preferences	GET/ PATCH	<code>/api/email/preferences</code>	Cookie	<code>src/app/api/email/preferences/route.ts</code>
Email — Outbox (admin)	GET	<code>/api/admin/emails</code>	Cookie admin	<code>src/app/api/admin/emails/route.ts</code>
Upload	POST	<code>/api/upload</code>	Cookie	<code>src/app/api/upload/route.ts</code>
Cron — Expire	POST/ GET	<code>/api/cron/...</code>	<code>CRON_SECRET</code>	<code>src/app/api/cron/*</code>
Admin — Dashboard	GET	<code>/api/admin</code>	Cookie admin	<code>src/app/api/admin/route.ts</code>

GROUP	METHOD	PATH	AUTH	ROUTE HANDLER
Admin — Users	GET/ POST/ PATCH/ DELETE	<code>/api/admin/users...</code>	admin	<code>src/app/api/admin/users/*</code>
Admin — Artists/ Studios	GET/ POST/ PATCH	<code>/api/admin/artists</code> , <code>/api/admin/studios</code>	admin	<code>src/app/api/admin/artists/*</code>
Admin — Bookings	GET/ PATCH	<code>/api/admin/bookings...</code>	admin	<code>src/app/api/admin/bookings/*</code>
Admin — Payments	GET	<code>/api/admin/payments</code>	admin	<code>src/app/api/admin/payments/route.ts</code>
Admin — Payouts	GET/ PATCH	<code>/api/admin/payouts</code>	admin	<code>src/app/api/admin/payouts/route.ts</code>
Admin — Quotations	GET	<code>/api/admin/quotations</code>	admin	<code>src/app/api/admin/quotations/route.ts</code>
Admin — Messages	GET	<code>/api/admin/messages</code>	admin	<code>src/app/api/admin/messages/route.ts</code>
Admin — Audit	GET	<code>/api/admin/audit</code>	admin	<code>src/app/api/admin/audit/route.ts</code>
Admin — Settings	GET/ PATCH	<code>/api/admin/settings</code>	admin	<code>src/app/api/admin/settings/route.ts</code>
Admin — Analytics	GET	<code>/api/admin/analytics</code>	admin	<code>src/app/api/admin/analytics/route.ts</code>
Dev — Emails	GET	<code>/dev/emails</code> (RSC page)	No (dev-only guard)	<code>src/app/dev/emails/page.tsx</code>
Errors ingestion	POST	<code>/api/errors</code>	rate-limited	<code>src/app/api/errors/route.ts</code>

Routes shown are exhaustive for v2 baseline; exact file layout under `src/app/api/` matches table. See `src/app/api/admin/list-routes.test.ts` for automated coverage of admin surface.

3. Authentication & Sessions

POST /API/AUTH/REGISTER

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Nurul Huda",
  "email": "nurul@example.my",
  "password": "Str0ng!Pass99",
  "role": "customer",          // customer | artist | studio
  "consent": true,
  "consentTimestamp": "2026-08-29T03:00:00.000Z"
}
```

STATUS	BODY
201	{user:{id,email,name,role,emailVerified}, devVerifyUrl?: string} + Set-Cookie: leish_session=...; HttpOnly; Path=/; SameSite=Lax; Secure@prod
400	{error:"Name must be at least 2 characters"} (first zod issue)
409	{error:"An account with this email already exists"}
429	{error:"Too many requests"} + Retry-After

Validation: `registerSchema` `src/server/validation.ts:5`.

POST /API/AUTH/LOGIN

```
POST /api/auth/login
{ "email":"nurul@example.my", "password":"Str0ng!Pass99" }
```

- 200 + cookie on success; 401 generic on failure; 429 when rate-limited. Agnost `login` tracked.

POST /API/AUTH/LOGOUT

- Reads `Cookie: leish_session=...`, revokes JTI (`UPDATE sessions SET revoked=1`), clears cookie. Always 200.

GET /API/AUTH/ME AND GET /API/ME ALIAS

- 200 `{user: PublicUser | null}` — null when unauthenticated (soft check, not 401).

POST /API/AUTH/RESEND-VERIFICATION

- Cookie required; rate-limited per `user_id` (or IP). 200 on success (`{message:"Verification email sent"}`) + email queued; 401 when unauthenticated; 429 when rate-limited.

POST /API/AUTH/FORGOT-PASSWORD

```
POST /api/auth/forgot-password
{ "email": "nurul@example.my", "turnstileToken": "" }
```

- Always `200 {message: "If an account exists ..."} (no enumeration)`. Token `password_resets` hashed sha256; 1h expiry. Dev response may include `devResetUrl`.

POST /API/AUTH/RESET-PASSWORD

```
POST /api/auth/reset-password
{ "token": "raw-token-from-email", "newPassword": "NewStr0ng!Pass" }
```

- 200 on success; 400 on invalid/expired/used/reused; single-use (`used_at` set).

4. Catalog APIs

GET /API/ARTISTS

```
Query: ?
query=string(<=100)&state=State&area=string(<=80)&bridal=BridalEvent&nonBridal=NonBridalEvent&budget=int
```

```
GET /api/artists?state=Selangor&area=Cyberjaya&bridal=solemnization&budget=70000&query=aisha
→ 200
{
  "artists": [ Artist, ... ] // Artist {id,slug,name,tagline,bio,image,rating,reviewCount,state,area,
                             // priceFrom,specialties,services,bridal,nonBridal,availability,portfolio,
                             // verified,yearsExperience,reviews[]}
}
```

- Validates `artistsQuerySchema` `src/server/validation.ts:52`. Implements `listArtists(filters)` with SQL pre-filter then `filterArtists`.
- Caching: public; CDN `Cache-Control: public, s-maxage=300, stale-while-revalidate=60` (set on route).
- Errors: 400 on invalid enum (e.g. unknown `bridal` id).

GET /API/CATALOG/ARTISTS & GET /API/CATALOG/STUDIOS

- Same semantics; separate route wrappers used by `ArtistsBrowser` client components that fetch with `.then()`.

GET /API/ARTISTS/[SLUG]/REVIEWS (OR SERVER HELPER)

- `GET` returns `listEntityReviews("artist", id) → Review[]`.
- `POST` body `{rating:1-5, text: string, event?:string, bookingId?:string}` → `addEntityReview` + `blendAggregate`; see FRS FR-REV-01.

5. Booking APIs

GET /API/BOOKINGS

```
GET /api/bookings?limit=20&offset=0
Cookie: leish_session=...

→ 200
{
  "bookings":[
    {
      "id":"uuid",
      "artistId":"aisha-azman",
      "studioId": null,
      "artistName":"Aisha Azman",
      "service":"Reception Makeup",
      "price":68000, // sen
      "date":"2026-09-10",
      "time":"10:00 AM",
      "notes":"Garden venue",
      "status":"accepted", // requested|accepted|confirmed|cancelled|completed
      "eventType":"reception",
      "venue":"Cyber 5",
      "guestCount":80,
      "quotation": {
        "id":"q-uuid",
        "baseFee":88000,"travelFee":8000,"earlyCallFee":0,"accommodationFee":0,
        "extras":[{"label":"Hair styling","amount":5000}],
        "artistNote":"Early call included",
        "total":101000,
        "status":"pending", // pending|paid|expired|superseded
        "createdAt":"2026-08-29T02:00:00.000Z",
        "expiresAt":"2026-08-30T02:00:00.000Z"
      } | null,
      "totalPrice":101000 | null, // null when no/active quotation expired
      "balanceDueDate":"2026-09-07", // ISO date (event - 3d)
      "balanceAmount":96000, // max(0, total - fee) | null
      "payment": { "amount":5000,"type":"deposit","status":"required|paid|...", "provider":"dev|
billplz","reference":"dev_...","url":null} | null,
      "balancePayment":{"amount":
96000,"type":"balance","status":"...","provider":"...","reference":"...","url":"..."} | null
    }
  ],
  "pagination":{"total":12,"limit":20,"offset":0,"hasMore":false}
}
```

- **Auth:** 401 when missing/bad session.
- **Scoping:** customer → `WHERE user_id=?`; artist/studio → claimed `artist_id / studio_id` set (empty → `[]`).
- **Errors:** 401 unauthenticated.

POST /API/BOOKINGS

```

POST /api/bookings
Cookie: leish_session=...
{
  "artistId": "aisha-azman",
  "service": "Reception Makeup",
  "date": "2026-09-10",
  "time": "10:00 AM",
  "notes": "Garden venue, 30 guests",
  "eventType": "reception",
  "venue": "12-G, Jalan Teknokrat 5",
  "guestCount": 30
}

```

STATUS	BODY
201	{booking:SerializedBooking, user:PublicUser}
400	{error:"Service not available for this artist" zod message}
401	{error:"Not authenticated"}
404	{error:"Artist not found"}
409	{error:"Sorry, this time slot has just been taken..."}
429	Too many requests

See FRS FR-BOOK-01 for server-derived price, pre-check + unique index race guard.

PATCH /API/BOOKINGS/[ID]

```

PATCH /api/bookings/bk_123
Cookie: leish_session=...
{ "action": "accept" } // accept | reject | complete | cancel
→ 200 { booking:SerializedBooking }
→ 400 {error:"Only requested bookings can be accepted"}
→ 401 | 403 | 404

```

Rules per `applyBookingTransition` + ownership checks (FRS FR-BOOK-03). Agnost tracks status updates.

POST /API/BOOKINGS/[ID]/QUOTATION

```
POST /api/bookings/bk_123/quotation
Cookie: leish_session=... (claimed artist)
{
  "baseFee": 88000,
  "travelFee": 8000,
  "earlyCallFee": 0,
  "accommodationFee": 0,
  "extras": [{"label": "Hair styling", "amount": 5000}],
  "artistNote": "Early call included, travel within 30km"
}
→ 201 {quotation: SerializedQuotation}
```

- 403 when not claimed artist; 400 when booking not `accepted` or validation fails; previous pending becomes `superseded`.

POST /API/BOOKINGS/[ID]/PAY-FEE & POST /API/BOOKINGS/[ID]/PAY-BALANCE

```
POST /api/bookings/bk_123/pay-fee
{ "turnstileToken": "..." } // optional when Turnstile configured
→ 200 {payment: {amount: 5000, type: "deposit", status: "required|paid", provider: "dev|
billplz", reference: "...", url: "https://www.billplz.com/bills/xxx" | null},
      booking: {status: "accepted|confirmed"}}
```

- Deposit amount is `getBookingFeeSen()` (not body). Balance amount is `quotation.total - fee`.
- `provider_url` is Billplz hosted page — client opens via `window.open(url, "_blank")`.
- When `provider=dev`, route auto-settles synchronously (feasible for e2e): `markBillPaid` + `handlePaymentPaid` + status `confirmed`.

POST /API/BOOKINGS/[ID]/REMINDE

- Claimed artist only, booking must be `confirmed`. Sends `balance_reminder` email respecting `email_preferences.balance_reminder`. Rate-limited.

POST /API/BOOKINGS/[ID]/REFUND

```
POST /api/bookings/bk_123/refund
→ 200 {payment: {...status: "refunded"}, message: "Balance refund issued"}
→ 400 {error: "Only balance payments are refundable" | "Only paid balances..."}
→ 401/403
```

- Refund endpoints call `refundBalancePayment()`; `billplz` path `POST /bills/{id}/refund`.

INVOICE

```
GET /api/bookings/[id]/invoice → 200 text/html (printable)
GET /api/bookings/[id]/invoice.pdf → 200 application/pdf + Content-Disposition: attachment;
filename="leish-invoice-<id>.pdf"
```

- Authz: owner or claimed artist only. Content built server-side from quotation+fees.

6. Messaging APIs

GET /API/BOOKINGS/[ID]/MESSAGES

```
GET /api/bookings/bk_123/messages
→ 200 {messages:[{id, senderName, body, createdAt}]}
```

POST /API/BOOKINGS/[ID]/MESSAGES

```
POST /api/bookings/bk_123/messages
{ "body":"Hello, please confirm venue." }
→ 201 {message:{id, senderName:"...", body:"...", createdAt:"..."}}
→ 400 empty body
→ 403 not participant
```

GET /API/BOOKINGS/[ID]/MESSAGES/STREAM (SSE)

```
GET /api/bookings/bk_123/messages/stream
Accept: text/event-stream
Cookie: leish_session=...

→ 200 Content-Type: text/event-stream
    Cache-Control: no-cache
    Connection: keep-alive

data: {"id":"m1","senderName":"Nurul","body":"Hi","createdAt":"..."}
data: {"id":"m2","senderName":"Aisha","body":"Hello!","createdAt":"..."}

: keepalive
```

- Replays history `ORDER BY created_at ASC` then streams live via `src/server/chat-bus.ts`. Requires participant auth; drop+reconnect replays.

7. Payments Webhook

POST /API/PAYMENTS/WEBHOOK

```
POST /api/payments/webhook
Content-Type: application/json
X-Billplz-Signature: 64-hex-hmac-sha256-of-raw-body-with-API_KEY
X-Billplz-Paid: true|false (inside body alternative field depending on Billplz version)
{
  "id":"billplz-bill-id",
  "collection_id":"...",
  "paid": true,
  "paid_amount": 5000,
  "amount": 5000,
  "state": "paid"
}
```

STATUS	BODY	EFFECT
200	<code>{ok:true}</code>	Processed: <code>markBillPaid</code> + <code>handlePaymentPaid</code> routed by <code>type</code>
401	<code>{error:"Invalid signature"}</code>	No DB write
404	Bill id not found	No booking confirmed

Verification `verifyBillplzSignature(rawBody, header, apiKey)` `src/server/payments.ts:381`: hex64 regex, `createHmac("sha256",apiKey).update(rawBody).digest("hex")`, `timingSafeEqual`.

Callers must send **raw body bytes** exactly as Billplz did (not pretty-printed).

8. Profile Claim

GET /API/ARTIST-PROFILES

```
GET /api/artist-profiles
Cookie: leish_session=...
→ 200 {profile:{artistId:"aisha-azman", artistName:"Aisha Azman"} | null}
→ 401 unauthenticated
```

POST /API/ARTIST-PROFILES

```
POST /api/artist-profiles
{ "artistId":"aisha-azman" }
→ 201 {profile:{artistId, artistName}}
→ 403 customer role cannot claim
→ 409 artistId already claimed OR user already has profile (PK violation)
→ 404 artist not found
```

PATCH /API/ARTIST-PROFILES

```
PATCH /api/artist-profiles
{ "bio":"New bio...", "priceFrom": 75000, "state":"Selangor" }
→ 200 {artist: Artist}
→ 400 validation on priceFrom range
→ 403 unclaimed cannot edit
```

- Whitelisted fields only; `verified` / `referralEarnings` ignored for self-service.

GET/POST/PATCH /API/STUDIO-PROFILES — MIRRORS ARTIST; SCOPES `STUDIO_ID`.

9. Reviews

POST /API/ARTISTS/[SLUG]/REVIEWS (EXAMPLE; SOME IMPL CALLS **SRC/SERVER/CATALOG.TS** VIA DEDICATED HANDLER)

```
POST /api/artists/aisha-azman/reviews
Cookie: leish_session=...
{
  "rating": 5,
  "text": "Aisha made me look amazing!",
  "event": "Reception",
  "bookingId": "bk_123" // optional; when given gates on that booking
}
→ 201 {review:{id,author,rating,date,text,event}}
→ 400 rating out of 1-5
→ 409 {error:"ALREADY_REVIEWED"} when same booking_id already has review
→ 401
```

10. Data Rights

GET /API/ME/EXPORT

```
GET /api/me/export
Cookie: leish_session=...
→ 200 Content-Type: application/json
  Content-Disposition: attachment; filename="leish-data-a1b2c3d4.json"
  { user, bookings:[], messages:[], payments:[], artistProfiles:[], ...}
→ 401
```

DELETE /API/ME?CONFIRM=1

```
DELETE /api/me?confirm=1
Cookie: leish_session=...
→ 200 {message:"Account deleted"}
→ 400 missing confirm=1
→ 401
```

11. Email Preferences

GET /API/EMAIL/PREFERENCES

→ 200 {preferences:{booking_created, quotation_sent, invoice_sent, quotation_expiry, balance_reminder, status_changed}} all 0/1.

PATCH /API/EMAIL/PREFERENCES

```
PATCH /api/email/preferences
{ "balance_reminder":0 }
→ 200 {preferences:{...}}
```

12. Upload

POST /API/UPLOAD

- Multipart/form-data; image only; routed to Vercel Blob or S3 (`BLOB_READ_WRITE_TOKEN` or AWS). Returns `{url}`. Auth required; size limites configurable.

13. Admin APIs (all behind `requireAdmin`)

All responses are JSON; every mutating handler calls `LogAdminAction()` after success.

GET /API/ADMIN — DASHBOARD AGGREGATES

```
GET /api/admin
→ 200 {
  stats:{
    users:{total, customers, artists, studios, admins},
    bookings:{total, requested, accepted, confirmed, completed, cancelled},
    payments:{total, paid, required, totalRevenue}, // totalRevenue in sen
    artistProfiles: number
  },
  recentBookings:[{id,artist_name,service,date,time,status,created_at}, ...10],
  recentAudit:[{id,action,target_table,target_id,created_at}, ...10]
}
```

GET /API/ADMIN/USERS (+ POST /API/ADMIN/USERS, PATCH /API/ADMIN/USERS/[ID], DELETE)

- `GET ?limit=&offset=&role=` paginated listing; `POST {name,email,password,role}` create; `PATCH {name,role,emailVerified}` update; `DELETE` remove (audit). Password never returned.

GET /API/ADMIN/ARTISTS (+ POST, PATCH /API/ADMIN/ARTISTS/[ID])

- Mirrors catalog mutability with full whitelist (including `verified`, `referralEarnings`). `POST {name,slug?,tagline,bio,state,area,priceFrom,...}` → `createArtist`.

GET /API/ADMIN/STUDIOS — MIRRORS ARTISTS VIA `CREATESTUDIO` / `UPDATESTUDIO`.

GET /API/ADMIN/BOOKINGS + PATCH /API/ADMIN/BOOKINGS/[ID]

- Admin status override + notes (bypass state machine for ops, but still audited).

GET /API/ADMIN/PAYMENTS — LISTS PAYMENTS JOINED WITH BOOKINGS (FILTER `?STATUS=&TYPE=&BOOKINGID=`).

GET /API/ADMIN/PAYOUTS + PATCH /API/ADMIN/PAYOUTS

```
GET /api/admin/payouts?status=pending
→ 200 {payouts:
  [{id,artist_user_id,booking_id,gross_sen,commission_sen,net_sen,status,settleable_at,settled_at,notes
  ,created_at, artist_name, service, event_date},...]}

PATCH /api/admin/payouts
{ "payoutId":"p_...", "status":"settled", "notes":"DuitNow ref XYZ" }
→ 200 {payout: PayoutRow}
→ 404 payout not found
→ 403 non-admin
```

GET /API/ADMIN/QUOTATIONS — LIST QUOTATIONS WITH BOOKING JOIN, FILTER BY STATUS.

GET /API/ADMIN/MESSAGES — LIST MESSAGES WITH BOOKING SCOPE.

GET /API/ADMIN/EMAILS — RETURNS `EMAIL_OUTBOX` (DEV) / RETRY QUEUE STATUS.

GET /API/ADMIN/AUDIT — PAGINATED `ADMIN_AUDIT_LOG` ORDERED `CREATED_AT DESC`.

GET /API/ADMIN/SETTINGS + **PATCH /API/ADMIN/SETTINGS**

```
GET /api/admin/settings
→ 200 {settings:{booking_fee_sen:"5000", commission_rate_bps:"1000", commission_waiver_sen:"10000"}}

PATCH /api/admin/settings
{ "key":"booking_fee_sen", "value":"7000" } // or batch [{key,value}]
→ 200 {setting:{key,value,updated_by,updated_at}}
→ 400 invalid key/value range
```

GET /API/ADMIN/ANALYTICS — AGNOST ROLLUPS (IF CONFIGURED).

14. Health & Cron

GET /API/HEALTH

```
GET /api/health
→ 200 {ok:true, db:"postgres"|"sqlite", env:"production"|"development", uptime:123.4}
→ 503 if DB unreachable
```

Docker `HEALTHCHECK` hits this.

POST /API/CRON/* (E.G. `/API/CRON/EXPIRE-QUOTATIONS`)

- Guard: `Authorization: Bearer <CRON_SECRET>` (Vercel Cron sends it); otherwise 401.
- Handlers: sweep `findExpiredQuotations()` + `markQuotationExpired`, flush `email_retries` (`next_retry < now && attempts < max`).

15. Error Handling

CONDITION	STATUS	BODY EXAMPLE
Zod validation	400	<code>{error:"Name must be at least 2 characters"}</code>
Missing session	401	<code>{error:"Not authenticated"}</code>
<code>requireAdmin</code> non-admin	403	<code>{error:"Forbidden"}</code>
Unknown artist/booking	404	<code>{error:"Artist not found"}</code>
Duplicate email / profile / slot / payment type	409	<code>{error:"An account with this email already exists"}</code> or slot message
Rate limit	429	<code>{error:"Too many requests"}</code> + <code>Retry-After</code>
Webhook bad sig	401	<code>{error:"Invalid signature"}</code>

Client error ingestion: `POST /api/errors {message, stack?, context}` — rate-limited, `reportError()`.

16. Security Notes (API Consumers)

- Never send `SESSION_SECRET` to client; all `NEXT_PUBLIC_*` are safe.
- Billplz webhook **must** send `X-Billplz-Signature` as hex64 of `HMAC-SHA256(apiKey, rawBody)`; JSON body is the Billplz bill payload (amount in sen).
- All mutating requests should include `Content-Type: application/json` unless multipart; missing `Cookie` returns 401 (not redirect) for API routes; pages handle redirects.
- Rate limits are per-IP + per-user where relevant; clients should respect `429` and back off.

17. Example: End-to-End Booking Flow via API

```
# 1. Register + verify (dev: grab devVerifyUrl from response)
curl -X POST http://localhost:3000/api/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"name":"Nurul","email":"nurul@example.my","password":"Str0ng!
Pass99","role":"customer","consent":true}'

# 2. Create booking
curl -X POST http://localhost:3000/api/bookings \
  -H 'Content-Type: application/json' \
  --cookie "leish_session=..." \
  -d '{"artistId":"aisha-azman","service":"Reception Makeup","date":"2026-09-20","time":"10:00
AM","eventType":"reception","venue":"Cyber 5","guestCount":40}'

# 3. Artist claims + accepts (as artist user)
curl -X POST http://localhost:3000/api/artist-profiles --cookie "leish_session=<artist>" -d '{"artist
Id":"aisha-azman"}'
curl -X PATCH http://localhost:3000/api/bookings/BK_ID --cookie "leish_session=<artist>" -d '{"action
":"accept"}'

# 4. Artist quotes
curl -X POST http://localhost:3000/api/bookings/BK_ID/quotation --cookie "leish_session=<artist>" \
  -d '{"baseFee":88000,"travelFee":8000,"extras":[{"label":"Hair styling","amount":5000}]}'

# 5. Customer pays deposit (dev → auto confirmed)
curl -X POST http://localhost:3000/api/bookings/BK_ID/pay-fee --cookie "leish_session=<customer>" -d
'{}'
# → {payment:{url:null, status:"paid"|"required"}, booking:{status:"confirmed"}}

# 6. Customer pays balance (dev → payout created)
curl -X POST http://localhost:3000/api/bookings/BK_ID/pay-balance --cookie
"leish_session=<customer>" -d '{}

# 7. Artist completes
curl -X PATCH http://localhost:3000/api/bookings/BK_ID --cookie "leish_session=<artist>" -d '{"action
":"complete"}'

# 8. Customer reviews
curl -X POST http://localhost:3000/api/artists/aisha-azman/reviews --cookie "leish_session=<customer>
" \
  -d '{"rating":5,"text":"Amazing!","event":"Reception","bookingId":"BK_ID"}'
```

Next: [docs/08-Traceability-Matrix.md](#) maps each BR → SRS → FRS → API → Table → Test.

08-Traceability-Matrix

Traceability Matrix — Leish! v2

- 1. End-to-End Trace (BR → FRS → Implementation → Test)
- 2. Business Rule → Test Matrix (BRL → Enforcement → Test)
- 3. NFR → Gate Matrix
- 4. Reverse Trace — Code → Requirement (Orphan Check)
- 5. Change History Linkage
- 6. Usage Notes

Traceability Matrix — Leish! v2

FIELD	VALUE
Document ID	LEISH-RTM-v2.0
Version	2.0.0
Date	2026-08-29
Purpose	Bidirectional trace: Business Requirement → SRS → FRS → API/UI → Data → Verification. Ensures no orphan requirement and no unvalidated feature.

1. End-to-End Trace (BR → FRS → Implementation → Test)

BR	BRS §	SRS- F	FRS IDS	APIS / PAGES	TABLES / LOGIC
BR-01 Public catalog discovery	6.1	F-02	FR-CAT-01	GET /api/artists , GET /api/catalog/artists , GET /artists , /studios , /	artists, studios + listArtists / AREAS_BY_STATE
BR-02 Filtered artists	6.1	F-02	FR-CAT-01, FR-CAT-03	GET /api/artists? state&area&bridal&nonBridal&budget&query	artistsQuerySchema , idx_artists_state_area
BR-03 Detail view	6.1	F-02	FR-CAT-02	GET /artists/[slug] , GET /studios/[slug] (RSC resolveArtist)	artists, studios, reviews via listEntityReviews
BR-04 Gated reviews	6.1	F-07	FR-REV-01, FR-REV-02	POST /api/artists/[slug]/reviews	reviews.booking_id UNIQUE + findReviewableBooking
BR-05 Atomic rating blend	6.1	F-07	FR-REV-03	(internal)	blendAggregate() single UPDATE ROUND(...,2)
BR-06 Registration	6.2	F-01	FR- AUTH-01, FR- AUTH-07	POST /api/auth/register , /register	users , registerSchema , crypt+pepper
BR-07 Email verification gate	6.2	F-01	FR- AUTH-05, FR- AUTH-02	GET /api/auth/verify-email , POST /api/auth/ resend-verification , /verify-email	email_verifications hashed token, users.email_verified , banner in /dashboard
BR-08 Password reset	6.2	F-01	FR- AUTH-06	POST /api/auth/forgot-password , POST /api/ auth/reset-password , /forgot-password , / reset-password	password_resets hashed 1h single-use
BR-09 Session 7d + revocation	6.2	F-01	FR- AUTH-03, FR- AUTH-04	POST /api/auth/login , POST /api/auth/logout , Cookie leish_session	sessions JTI, SESSION_TTL_SECONDS , verifySessionToken
BR-10 Export / delete (PDPA)	6.2	F-13	FR- COMP-01, FR- COMP-02	GET /api/me/export , DELETE /api/me? confirm=1 , /dashboard buttons	FK CASCADE / SET NULL
BR-11 Booking request with event details	6.3	F-03	FR- BOOK-01	POST /api/bookings , /artists/[slug]/book	bookingSchema , bookings + server-derived price , notifyBookingCreated
BR-12 Slot uniqueness	6.3	F-03	FR- BOOK-01	POST /api/bookings 409 path	uq_bookings_slot partial index + pre-check + race catc

BR	BRS §	SRS- F	FRS IDS	APIS / PAGES	TABLES / LOGIC
BR-13 Accept/ reject/ complete	6.3	F-03	FR- BOOK-03	<code>PATCH /api/bookings/[id] {accept\ reject\ complete\ cancel}</code>	<code>applyBookingTransition</code> state machine
BR-14 Quotation 24h + re- quote	6.3	F-04	FR-QUO-01, FR-QUO-02	<code>POST /api/bookings/[id]/quotation</code> , <code>/dashboard</code> quote block	<code>quotations</code> <code>status</code> <code>pending/superseded/expired</code> <code>paid</code> , <code>quotationTotal</code> , <code>QUOTATION_TTL_MS</code>
BR-15 Quotation display + balance due	6.3	F-03, F-04	FR- BOOK-04	<code>GET /api/bookings</code> <code>serializeBooking</code> (quotation,totalPrice,balanceDueDate,balanceAmount)	<code>getBookingFeeSen()</code> , <code>balanceDueDate = date -30</code>
BR-16 Pay deposit	6.4	F-05	FR-PAY-02	<code>POST /api/bookings/[id]/pay-fee</code>	<code>payments(type=deposit)</code> , <code>createBookingPayment</code> , Billplz <code>POST /bills</code>
BR-17 Pay balance T-3d	6.4	F-05	FR-PAY-03	<code>POST /api/bookings/[id]/pay-balance</code>	<code>payments(type=balance)</code> , <code>amount=total-fee</code>
BR-18 Webhook HMAC verified	6.4	F-05	FR-PAY-04	<code>POST /api/payments/webhook</code> + <code>X-Billplz-Signature</code>	<code>verifyBillplzSignature</code> timingSafe, <code>markBillPaid</code> , <code>handlePaymentPaid</code>
BR-19 Webhook routes + confirms / creates payout	6.4	F-05, F-06	FR-PAY-04, FR- PAYO-02	same webhook	<code>confirmOnFeePaid</code> vs <code>createPayoutForBooking</code> routed by <code>type</code>
BR-20 Commission computed	6.4	F-06	FR- PAYO-01	(internal, admin settings)	<code>platform_settings</code> , <code>computeCommission</code> , rate clamp 0..5000
BR-21 Payout settle/fail	6.4	F-06	FR- PAYO-03	<code>GET/PATCH /api/admin/payouts</code> + <code>/admin/payouts</code>	<code>payouts.status</code> <code>pending→settled/failed</code> , <code>settleable_at = event+24h</code>
BR-22 Refund balance only	6.4	F-05	FR-PAY-05	<code>POST /api/bookings/[id]/refund</code>	<code>refundBalancePayment</code> guard <code>type=balance</code> + <code>status=paid</code>
BR-23 Chat live	6.5	F-08	FR- MSG-01, FR-MSG-02	<code>GET/POST /api/bookings/[id]/messages</code> , <code>GET .../messages/stream</code> SSE	<code>messages</code> + <code>chat-bus</code> pub sub

BR	BRS §	SRS- F	FRS IDS	APIS / PAGES	TABLES / LOGIC
BR-24 Email preferences	6.5	F-11	FR-MAIL-03	<code>GET/PATCH /api/email/preferences</code>	<code>email_preferences</code> flags
BR-25 Email retries + outbox	6.5	F-11	FR-MAIL-01, FR-MAIL-02	<code>POST /api/bookings/*</code> email side-effects, <code>/dev/emails</code>	<code>email_outbox</code> , <code>email_retries</code> , <code>sendEmail</code> fallback
BR-26 Admin CRUD	6.6	F-10	FR-ADM-02	<code>GET/POST/PATCH/DELETE /api/admin/users</code> , <code>.../artists</code> , <code>.../studios</code> , <code>GET/PATCH .../bookings</code> , <code>GET .../payments</code> , <code>GET .../quotations</code> , <code>GET .../messages</code>	<code>artists, studios, bookings</code> , via <code>updateArtist</code>
BR-27 Audit trail	6.6	F-10	FR-ADM-03	all admin mutators	<code>admin_audit_log</code> + <code>logAdminAction()</code> + <code>idx_audit_log_*</code>
BR-28 Tunable settings	6.6	F-10, F-15	FR-ADM-04, FR-OPS-01	<code>GET/PATCH /api/admin/settings</code> , <code>/admin/settings</code>	<code>platform_settings</code> , <code>getBookingFeeSen()</code> etc + 30s cache
BR-29 Dashboard metrics	6.6	F-10	FR-ADM-02 (dashboard)	<code>GET /api/admin</code>	Aggregated counts <code>users by role</code> , <code>bookings by status</code> , <code>payments revenue</code>
BR-30 Structured logs + forwarding	6.7	F-12	FR-OPS-01 (NFR)	<code>src/server/logger.ts</code>	<code>pino</code> + <code>LOG_WEBHOOK_URL</code> batch 20/50ms
BR-31 Error reporting	6.7	F-12	(NFR-S)	<code>src/server/errors.ts</code>	<code>reportError</code> → Sentry / <code>ERROR_WEBHOOK_URL</code>
BR-32 Analytics events	6.7	F-12	FR-OPS-03	<code>src/server/agnost.ts</code> , <code>src/lib/agnost-client.ts</code> , <code>src/instrumentation.ts</code>	<code>agnost.begin/end</code> , <code>trackEvent</code>
BRL-*	BRS §7	SRS §6-7	—	—	Enforced by DB CHECK + code guards
NFR-P/R/S/U/M/O	BRS §10-12	SRS §7, NFR doc	—	Headers, CSP, rate-limit, <code>next.config.ts</code>	—

2. Business Rule → Test Matrix (BRL → Enforcement → Test)

BRL	RULE	ENFORCEMENT (DB/CODE)	TEST LOCATION / SPEC
BRL-01	Slot atomic	<code>uq_bookings_slot</code> partial index <code>src/server/db.ts:470</code> ; service guard <code>POST /api/bookings</code>	Booking concurrency test (cohort 2 par requests → 1 409)
BRL-02	Price authority	<code>serviceDef.price</code> server derive <code>src/app/api/bookings/route.ts:183</code> ; <code>quotationTotal()</code> server-only	Booking create test asserts custom price ignored
BRL-03	Sen integers	<code>INTEGER</code> columns + <code>Math.round</code> in <code>quotations.ts</code> / <code>settings.ts</code> / <code>payouts.ts</code>	Lint + payment math tests
BRL-04	Deposit non-refundable	<code>refundBalancePayment()</code> guard <code>type!</code> <code>=balance → throw</code> <code>src/server/payments.ts:322</code>	Refund test attempts deposit → error
BRL-05	Confirm only via webhook	<code>confirmOnFeePaid</code> called only in <code>handlePaymentPaid</code> <code>src/server/payments.ts:246</code> ; PATCH rejects <code>confirmed</code>	Webhook vs PATCH tests
BRL-06	24h quotation TTL	<code>QUOTATION_TTL_MS=86400000</code> <code>src/server/quotations.ts:13</code> ; <code>isQuotationExpired</code> lazy + sweep	Expiry test (mock now+25h → expired)
BRL-07	Review gated	<code>reviews.booking_id UNIQUE</code> + <code>findReviewableBooking</code> <code>src/server/catalog.ts:678</code>	Gate test: uncompleted → null; duplicate review → 409
BRL-08	Commission waiver	<code>computeCommission</code> waived branch <code>src/server/settings.ts:106</code>	Unit test total<waiver → 0
BRL-09	Commission cap	Clamp <code>min(5000,max(0,round(v)))</code> <code>src/server/settings.ts:77</code>	Cap test rate 6000 → 5000
BRL-10	Verification gate	Payment handler checks <code>email_verified</code> (strict flag)	Pay before verified → 403 when enabled
BRL-11	Profile claim scoping	<code>artist_profiles</code> UNIQUE + <code>GET /api/bookings</code> claimedId filter <code>src/app/api/bookings/route.ts:99</code>	Scoping test: artist sees only claimed
BRL-12	Settlement 24h	<code>settleable_at = eventDate+24h</code> <code>src/server/payouts.ts:81</code>	Payout date math test
BRL-13	Audit completeness	<code>logAdminAction()</code> in every admin mutator + <code>src/app/api/admin/list-routes.test.ts</code> coverage audit	Admin mutation test asserts audit row created
BRL-14	Rate limit	<code>src/server/rate-limit.ts</code> sliding window + 429 header	Rate-limit contract test
BRL-15	7d JTI revoke	<code>src/server/session.ts</code> <code>SESSION_TTL_SECONDS=604800</code> + <code>revokeSession</code> wins	Session revoke test

3. NFR → Gate Matrix

NFR ID	GATE TYPE	BLOCKING PR?	MEASUREMENT
NFR-P-01 (latency p95)	Bench on preview deploy; warn if > target	No (advisory for launch, blocking by v2.1)	k6 / Vercel p95
NFR-P-04 (pool, WAL)	Code review required fields present	Yes	Review checklist
NFR-R-02 (degradation)	Test: kill email/Billplz mock → still 200	Yes	Unit with mocked provider
NFR-S-03.03 (webhook sig)	Unit suite includes forged sig → 401	Yes (CI fails if missing)	Vitest
NFR-S-04 (headers)	<code>curl -I</code> assertions on <code>next.config.ts</code> headers	Yes	ZAP / curl
NFR-U-01 (a11y ≥90)	Lighthouse CI step <code>assert: a11y≥90</code>	Advisory pre-launch, blocking post-launch	Lighthouse CI
NFR-M-01 (lint/typecheck/coverage)	<code>quality-gate.yml</code> all green, coverage ≥80%	Yes	GHA required
NFR-O-02 (health)	<code>GET /api/health</code> 200 on PR deploy	Yes	Smoke fetch

4. Reverse Trace — Code → Requirement (Orphan Check)

IMPLEMENTATION	REQUIREMENT(S)	ORPHAN?
All <code>src/server/*</code> modules	See §1 rows	No — mapped
<code>src/components/*</code>	FRS §12 Dashboard, SRS §4.1 UI	Covered
<code>referrals</code> table + <code>referral_code</code> columns	BRS §4.2 deferred (referral MVP scaffold only) — no business rule yet; code is pre-requirement scaffolding for v2.2	Deferred — flag red if exposed before BR; tests exist but feature hidden
<code>products</code> type <code>src/lib/types.ts:89</code>	BRS §4.2 deferred e-commerce	Scaffold, not traced — ok if unused; must not gate launch
<code>src/lib/agnost-client.ts</code> client tracking	BR-32	Traced
<code>src/server/referral.ts</code> helpers	BRS deferred	Scaffold — see above; require BR before enabling rewards

Action: no undocumented production feature that bypasses a BR; scaffolds above are gated behind `referredBy` writes that only run when `referralCode` supplied (optional) — safe.

5. Change History Linkage

DOC VERSION	RTM CHANGE
v1 (2026-08-15)	BR-01..BR-16 only (single payment, no payouts)
v2 (2026-08-29)	Added BR-17..BR-25, BR-27..BR-32; added payments.type hybrid, payouts, messaging, admin audit, Agnost

6. Usage Notes

- **Before adding code:** add row to FRS → map to BR or create new BR (with product consent); never code from Jira alone.
- **Before shipping:** `git diff --stat` against this RTM — every changed table/endpoint must have a row or be scaffolding.
- **Before audit:** `npm run typecheck && npm run lint && npm test` + `src/app/api/admin/list-routes.test.ts` proves admin guard coverage.

This RTM is the single reference for reviewers (security, QA, compliance) to confirm every promised capability is implemented and tested.

09-Glossary

Glossary — Leish! v2

Glossary — Leish! v2

TERM	DEFINITION
Admin	Platform operator with <code>role='admin'</code> ; can access <code>/admin/*</code> and <code>/api/admin/*</code> ; all mutations audit-logged to <code>admin_audit_log</code> .
Agnost AI	Analytics SDK (<code>agnostai</code>) initialized at startup (<code>src/instrumentation.ts</code>) and client (<code>src/lib/agnost-client.ts</code>); register/login/booking/quotation/payment/profile-edit; dashboard <code>3e27e121...</code> .
Artist	Role <code>artist</code> ; MUA serving customers; must claim a catalog profile (<code>artist_profiles</code>) to receive/act on bookings.
Balance	<code>payments.type='balance'</code> — second payment = <code>quotation.total - deposit</code> ; due 3 days before event (<code>balanceDueDate</code>); when paid triggers payout. Non-refundable as deposits are, but balance is refundable on cancelled.
Billplz	Malaysian hosted payment gateway API v3; configured via <code>BILLPLZ_API_KEY</code> , <code>BILLPLZ_COLLECTION_ID</code> , <code>BILLPLZ_...</code> ; bills created at <code>POST /bills</code> with HMAC webhook <code>X-Billplz-Signature</code> . Fallback <code>provider='dev'</code> when not co...
BridalEvent	Enum <code>engagement\ solemnization\ reception\ full-package\ bridal-other</code> ; used in artist tags and <code>BRIDAL_EV...</code> .
Booking	Central entity (<code>bookings</code> table) with <code>id,user_id,artist_id,studio_id,artist_name,service,price,date,time,notes,event_type,venue,guest_coun...</code> ; status state machine <code>requested→accepted→confirmed→completed\ cancelled</code> ; slot guarded by partial unique index.
Booking Fee / Deposit	Flat non-refundable amount <code>booking_fee_sen</code> (default RM50 = 5000 sen) — first payment to confirm an <code>accepted</code> platform-kept; configured in <code>platform_settings</code> .
Budget	Filter on <code>listArtists</code> (<code>budget</code> → SQL <code>price_from ≤ budget</code>); integer sen equivalent client sends MYR.
Catalog	DB tables <code>artists,studios</code> (runtime source of truth); <code>src/lib/data.ts</code> <code>SEED_ARTISTS/STUDIOS</code> are seed-only ; repository <code>src/server/catalog.ts</code> (auto-seeds via <code>ensureCatalogSeeded</code>).
Claim	Linking a platform user to a catalog entity: <code>artist_profiles(user_id→artist_id)</code> or <code>studio_profiles</code> ; scopes (<code>bookings</code> to claimed ids (<code>getClaimedArtistIds</code>).
Client / Customer	Role <code>customer</code> ; external user booking services; only owner of booking can pay/cancel.
Commission	Platform keep computed by <code>computeCommission(totalSen, rateBps, waiverSen)</code> — waived when <code>total < waive...</code> otherwise <code>round(total*rate/10000)</code> ; deducted from payout (not added to client price).
CSP (Content Security Policy)	Per-request nonce CSP set in <code>next.config.ts</code> ; no <code>unsafe-inline</code> for scripts (theme inline uses nonce) plus <code>X-Fr...</code> <code>Options:DENY</code> , etc.
DbFacade	Two-backend DB abstraction <code>getDb(): DbFacade {prepare,exec}</code> hiding PG (<code>pg.Pool</code>) vs SQLite (<code>node:sqlite DatabaseSync</code>); placeholder translation <code>@name/? → \$n</code> ; <code>getDb()</code> synchronous.
Deposit	Alias for Booking Fee (flat RM50). <code>payments.type='deposit'</code> .
Dev Provider	<code>provider='dev'</code> — synthetic payment row with <code>provider_ref=dev_...</code> and no Billplz call; webhook auto-settled in te... never charges real money.
Email Outbox	Table <code>email_outbox</code> sink for <code>EMAIL_PROVIDER=dev</code> ; viewable at <code>/dev/emails</code> (dev only).

TERM	DEFINITION
Event Type	<code>event_type</code> on bookings (bridal/non-bridal taxonomy label); <code>venue</code> , <code>guest_count</code> sibling fields.
FRS	Functional Requirements Specification (<code>docs/03-FRS-...</code>); per-module <code>FR-xxx</code> with acceptance criteria.
Guest	Unauthenticated visitor; can browse catalog but is redirected for booking/auth actions.
BRS / BRD	Business Requirements Specification (<code>docs/01-BRS-...</code>); business-level <code>BR-xx</code> , <code>BRL-xx</code> .
NFR	Non-Functional Requirements (<code>docs/04-NFR-...</code>) covering performance, security, reliability etc.
JTI / JWT	JWT with JTI revocation table <code>sessions(jti,revoked)</code> ; HS256 via <code>jose</code> ; cookie <code>leish_session</code> 7-day TTL.
NonBridalEvent	Enum <code>dinner\ graduation\ ceremony\ corporate\ touch-up\ non-bridal-other</code> ; <code>NON_BRIDAL_EVENTS</code> .
Payout	Settlement row (<code>payouts</code> : <code>gross,commission,net,status pending→settled\ failed,setttable_at=event+24h</code> when balance is paid; <code>net = gross - commission - deposit</code> ; admin settles via <code>/admin/payouts</code>).
PDPA	Malaysia Personal Data Protection Act 2010 — consent (<code>users.consent</code>), export (<code>/api/me/export</code>), delete (<code>DELETE /api/me</code>), retention per <code>docs/PDPA_RETENTION_GUIDELINES.md</code> .
Platform Settings	<code>platform_settings</code> key/value store runtime-configurable by admin: <code>booking_fee_sen</code> , <code>commission_rate_bps</code> (clamp), <code>commission_waiver_sen</code> (default RM100). Cached 30s.
Pre-Check vs Index	Booking slot pre- <code>SELECT</code> friendly 409 + race <code>catch</code> on <code>UNIQUE</code> / <code>duplicate key</code> — hard guarantee is DB partial in <code>uq_bookings_slot</code> .
Quotation	Line-item price breakdown (<code>quotations</code> : base/travel/earlyCall/accommodation + extras) with <code>total</code> sen; <code>status pending expired superseded paid</code> ; TTL 24h (<code>QUOTATION_TTL_MS</code>); re-quote supersedes.
RTM	Requirements Traceability Matrix (<code>docs/08-Traceability-Matrix.md</code>); BR→SRS→FRS→API→DB→test.
SRS	Software Requirements Specification (<code>docs/02-SRS-...</code>); SRS-F features with §4 external interfaces.
SSE	Server-Sent Events <code>text/event-stream</code> for per-booking chat: <code>GET /api/bookings/[id]/messages/stream</code> via <code>ch</code> .
State Machine	<code>applyBookingTransition(current,action,{isOwner,role})</code> + <code>confirmOnFeePaid</code> defining valid transitions (diag §20).
Studio	Role <code>studio</code> and catalog <code>studios</code> table; may have distinct flow in future but today shares Artist flows scoped by <code>st</code> when claimed; seed holds 4 studios.
Terms of Reference	<code>BR</code> , <code>SRS-F</code> , <code>FR</code> , <code>NFR</code> , <code>UC</code> , <code>US</code> , <code>BRL</code> naming convention across docs.
Turnstile	Cloudflare Turnstile bot protection widget (<code>TurnstileWidget</code>) + server verify (<code>TURNSTILE_SECRET_KEY</code>); required on <code>balance</code> when configured.
Use Case / User Story	<code>UC-xx</code> (Cockburn steps) and <code>US-xx</code> (INVEST) in <code>docs/05-Use-Cases-and-User-Stories.md</code> .
Verification	<code>email_verifications</code> — hashed single-use token for email verification; shown in <code>/dev/emails</code> in dev.